

PROJECT PPG
RECOVERABLE ASSETS (RA) & INVENTORY RISK MANAGEMENT (IRM)

MOHAMED ALIF FATHI BIN ABDUL LATIF	A23CS0112
MUHAMMAD ADAM BIN RAZALI	A23CS0116
AFIF SHAQIR IRFAN BIN ARQAM	A23CS0204
YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201
LAU YAN KAI	A23CS0098
LAU YEE WEN	A23CS0099
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164

UNIVERSITI TEKNOLOGI MALAYSIA

ABSTRACT

The purpose of this study is to design and implement a fully automatic end-to-end data engineering pipeline for PPG Coatings (M) Sdn Bhd. The main goal of the system is to solve operational problems related to inventory outage, and to detect the loss of raw materials and the manual record of inactivity of different departments. The project is developed using the Waterfall methodology and divides the data life cycle into three levels based on Medallion Architecture, Bronze, Silver, and Gold layer. Technically, the system uses Azure Data Factory as an arrangement tool to extract six CSV-formatted operational data files into Azure Data Lake Storage Gen2 cloud storage. Subsequently, using the Azure Databricks engine driven by the PySpark programming language, the Silver-level data was cleaned up and the mathematical indicators of the use of raw materials within 12 months were calculated at the Gold level. From the perspective of business logic, this study realises the automatic identification of recoverable assets, that is, the number of inventories that exceeds the actual use of the factory for 12 months. Then, these recyclable assets are classified as high-risk materials belonging to the Magna Carta category, provided that their utilisation rate is zero and the expiration date has exceeded 9 months, as “dead inventory” or the actual shelf life (expired) of the inventory. This structured classification is crucial to help the company's management and finance department automatically calculate the amount of financial reserves or the value of loss. Specifically, the method involves multiplying the quantity of the problem inventory by the unit cost. In addition to financial loss control, this integrated system is also modelled and developed based on the Galaxy Schema in the Azure Synapse Analytics analysis database. The solution design covers multiple fact tables and dimension tables, enabling the system to identify materials that may be at risk of shortage and directly track the impact of supply chain issues on production orders (unfulfilled orders) and customer sales orders. All these comprehensive analysis results will then be transmitted to Microsoft Power BI to generate an accurate interactive visual dashboard, which helps companies change from passive inventory management to more active and fully data-driven risk management.

ABSTRAK

Kajian ini dilakukan bertujuan untuk mereka bentuk dan melaksanakan saluran paip kejuruteraan data (*data engineering pipeline*) hujung-ke-hujung yang diautomasikan sepenuhnya bagi PPG Coatings (M) Sdn Bhd. Objektif utama sistem ini adalah untuk menyelesaikan masalah operasi berkaitan keusangan inventori (*inventory obsolescence*) serta pengesanan bahan mentah luput dan tidak aktif yang sebelum ini direkodkan secara manual merentasi jabatan yang berbeza. Pembangunan projek ini menggunakan metodologi Waterfall bagi menyokong senibina Rangka Kerja Medallion (*Medallion Architecture*), yang menyusun kitaran hayat data kepada lapisan Gangsa (*Bronze*), Perak (*Silver*) dan Emas (*Gold*). Secara teknikal, sistem ini mengekstrak enam fail data operasi berformat CSV ke dalam storan awan *Azure Data Lake Storage Gen2* dengan menggunakan *Azure Data Factory* sebagai alat orkestrasi. Selepas itu, enjin *Azure Databricks* yang dikuasakan oleh bahasa pengaturcaraan PySpark digunakan untuk membersihkan data di lapisan Perak serta melakukan pengiraan matematik bagi metrik penggunaan bahan mentah untuk tempoh 12 bulan di lapisan Emas. Bagi aspek logik perniagaan, kajian ini mengautomatiskan pengesanan Aset Boleh Pulih (*Recoverable Assets*) iaitu stok yang kuantitinya melebihi penggunaan sebenar kilang selama 12 bulan. Stok RA ini seterusnya diklasifikasikan sebagai bahan berisiko tinggi di bawah kategori Magna Carta sekiranya ia merupakan Stok Mati (*Dead Stock*) sifar penggunaan melebihi 9 bulan tanpa tarikh luput) atau merupakan stok yang telah melepasi tarikh luput fizikal (*Expired*). Pengklasifikasian berstruktur ini amat penting bagi membantu pihak pengurusan syarikat dan jabatan kewangan mengira jumlah peruntukan kewangan (*provision amount*) atau nilai kerugian secara automatik dengan mendarabkan kuantiti stok bermasalah tersebut dengan kos seunit. Selain daripada kawalan kerugian kewangan, sistem integrasi ini turut dibangunkan menggunakan pemodelan Skema Galaksi (*Galaxy Schema*) di dalam pangkalan data analitik *Azure Synapse Analytics*. Reka bentuk skema yang merangkumi pelbagai jadual fakta dan jadual dimensi ini membolehkan sistem mengesan bahan yang berisiko mengalami kehabisan stok (*stockout*), serta menjejaki impak masalah rantai bekalan ini secara langsung ke atas pesanan pengeluaran yang tidak dapat dipenuhi dan pesanan jualan pelanggan yang bakal terjejas. Kesemua output analitik komprehensif ini kemudiannya disalurkan ke *Microsoft Power BI* untuk menghasilkan papan pemuka visual interaktif yang tepat, sekali gus membantu syarikat beralih daripada pengurusan inventori yang reaktif kepada pengurusan risiko yang jauh lebih proaktif dan dipacu sepenuhnya oleh data.

TABLE OF CONTENTS

	TITLE	PAGE
	ABSTRACT	i
	ABSTRAK	ii
	TABLE OF CONTENTS	iii
	LIST OF TABLES	viii
	LIST OF FIGURES	x
	LIST OF ABBREVIATIONS	xiii
CHAPTER 1	INTRODUCTION	1
1.1	Introduction	1
1.2	Problem Background	2
1.3	Problem Aim	2
1.4	Project Objectives	2
1.5	Project Scope	3
1.6	Project Importance	3
1.7	Project Organisation	4
CHAPTER 2	LITERATURE REVIEW	5
2.1	Introduction	5
2.2	Supply Chain Management	5
2.2.1	Introduction and Definition of Supply Chain Management	6
2.2.2	Evolution of Supply Chain Management	7
2.2.3	The Drivers of Supply Chain Coordination	8
2.2.4	The Seven Principles of Supply Chain Management	9
2.2.5	Supply Chain Business Processes and Integration	9
2.2.6	Gaps and Future Research Directions	9
2.3	Enterprise Resource Planning	10
2.4	Medallion Architecture	12

2.5	Visualisation	13
2.6	Comparison of Previous Studies	14
2.6.1	Traditional SCM & Uncertainty	14
2.6.2	Process Integration & Risk	15
2.6.3	Data Engineering & Architecture	15
2.7	Azure	17
2.8	Power BI	18
2.9	Summary	19
CHAPTER 3	RESEARCH METHODOLOGY	21
3.1	Introduction	21
3.2	Methodology Choice and Justification	21
3.3	Phases of the Chosen Methodology	22
3.3.1	Activities and Process Applied in Each Phases	23
3.3.2	Design Modelling	25
3.3.3	Design Tools	26
3.3.4	Gantt Chart	28
3.3.5	Technology Used	29
3.4	System Requirements	29
3.4.1	Software Requirements	30
3.4.2	Hardware Requirements	30
3.5	Summary	31
CHAPTER 4	ANALYSIS AND DESIGN	32
4.1	Introduction	32
4.2	System Analysis	32
4.2.1	Case Study	33
4.2.2	Company Organization Structure	34
4.2.3	Requirements Gathering Technique	36
4.3	System Requirement Analysis	37
4.3.1	Functional Requirements	37
4.3.2	Non-functional requirement	39
4.4	Current System Analysis	40

4.5	System Design	41
4.5.1	System Architecture	41
4.5.2	Explain each component in system architecture	42
4.5.3	System Design (Data Engineering Process)	43
4.5.3.1	Use Case Diagram	43
4.5.3.2	Activity Diagram	45
4.5.4	Database Design	46
4.5.5	Interface Design	56
4.5.5.1	Inventory Health & Financial Risk	57
4.5.5.2	Fulfilment Risk and Customer Impact	58
4.6	Summary	59
CHAPTER 5	IMPLEMENTATION AND TESTING	60
5.1	Introduction	60
5.2	Pipeline Development	60
5.2.1	Data Storage and Transformation Process	61
5.2.2	Analytical Database Development using Azure Synapse Analytics	63
5.3	Coding of System Main Functions	65
5.3.1	Phase 1: Bulk Ingestion and Landing (Source to Bronze)	65
5.3.1.1	Programmatic Data Enrichment and Anomaly Seeding	66
5.3.1.2	Local PowerShell Bulk Ingestion Engine	67
5.3.2	Phase 2: Structural Integrity and Parquet Conversion (Bronze to Silver)	69
5.3.2.1	Data Factory Pipeline Orchestration & Transformation Logic	70
5.3.2.2	Storage Tier Verification (Silver Container Population)	71
5.3.3	Phase 3: Algorithmic Scoring and Business Rules Enforcement (Silver to Gold)	72

5.3.3.1	PySpark Programmatic Transformation Logic	72
5.3.3.2	Cloud Storage Verification (Gold Container Population)	79
5.3.4	Phase 4: Serving Layer Aggregation and Optimization (Gold to Curated)	79
5.3.5	Phase 5: Semantic Modeling and DAX Formula Engineering (Curated to Power BI)	92
5.3.5.1	Semantic Model Galaxy Schema Architecture	92
5.3.5.2	Technical Breakdown of Core DAX Measures	94
5.4	Interfaces of System	97
5.4.1	Ingestion Interface – Azure Data Lake Storage Gen 2	98
5.4.2	Transformation Interface (Silver Layer) - Azure Data Factory Studio	98
5.4.3	Transformation Interface (Gold Layer) - Azure Databricks Workspace	100
5.4.4	Transformation Interface (Curated Layer) – Azure Synapse Studio	100
5.4.5	Serving Layer – Microsoft Power BI Desktop	101
5.5	Main Functions Testing	102
5.5.1	Data Quality and Schema Testing	102
5.5.1.1	Test Approach	102
5.5.1.2	Test Cases	103
5.5.2	Business Logic Testing	106
5.5.2.1	Test approach	106
5.5.2.2	Test cases	106
5.5.2.3	Business Rule Decision Documented	107
5.5.2.4	Results	108
5.5.3	End-to-End (E2E) Integration Testing	109
5.5.3.1	Test Approach	109
5.5.3.2	Test Case	109

5.5.3.3	Result	111
5.5.4	Performance and Scalability Testing	111
5.6	Summary	116
REFERENCES		117

LIST OF TABLES

TABLE NO.	TITLE	PAGE
Table 1.1	Project Organisation	4
Table 2.1	SQL queries to extract different categories of arcs of the GoR built on top of SAP ERP	11
Table 2.2	Comparison of Previous Literature	16
Table 4.1	Requirement Gathering Methodologies and Engineering Outputs	36
Table 4.2	Functional requirements	38
Table 4.3	Non-functional requirements	39
Table 4.4	Relationship for each table	47
Table 4.5	<i>fact_inventory_status</i> fact table	50
Table 4.6	<i>fact_purchase_order</i> fact table	51
Table 4.7	<i>fact_production_order</i> fact table	51
Table 4.8	<i>fact_sales_order_impact</i> fact table	52
Table 4.9	<i>dim_material</i> dimension table	52
Table 4.10	<i>dim_material_category</i> dimension table	53
Table 4.11	<i>dim_supplier</i> dimension table	53
Table 4.12	<i>dim_product</i> dimension table	53
Table 4.13	<i>dim_customer</i> dimension table	54
Table 4.14	<i>dim_date</i> dimension table	54
Table 4.15	<i>dim_sales_order</i> dimension table	54
Table 5.1	Test Case Description	103
Table 5.2	Test Cases and Scenarios	107
Table 5.3	Cross-Layer Record Count Verification	110
Table 5.4	E2E Value Reconciliation	110
Table 5.5	Dashboard Material Classification Verification	110
Table 5.6	Business Rule Invariants	111

Table 5.7	Performance Testing I	112
Table 5.8	Performance Testing II	114
Table 5.9	Performance Testing III	115

LIST OF FIGURES

Figure 2.1	Schematic of a generic supply chain (Thomas D.J. and P.M. Griffin, 1996)	7
Figure 2.2	Uncertainty in the Supply Chain (Davis, 1993)	8
Figure 3.1	Gantt Chart for Project Progress	28
Figure 4.1	Data Architecture in PPG Recoverable Assets & Inventory Risk Management System	41
Figure 4.2	Use Case Diagram for PPG Inventory Risk Management System	44
Figure 4.3	Activity Diagram for PPG Inventory Risk Management System	45
Figure 4.4	Relational Design for Recoverable Assets and Inventory Risk Management Data Warehouse	47
Figure 4.5	Galaxy Schema Design for Recoverable Assets and Inventory Risk Management Data Warehouse	49
Figure 4.6	Galaxy Schema Implementation on Actual Pipeline	55
Figure 4.7	Dashboard for PPG Magna Carta and Inventory Health	57
Figure 4.8	Dashboard for PPG Fulfilment Risk and Customer Impact	58
Figure 5.1	Azure Data Lake Storage Gen2 Containers	61
Figure 5.2	Bronze-to-Silver and Silver-to-Gold Transformation Pipeline	61
Figure 5.3	Azure Synapse Analytics External Tables in the <i>ppg_curated</i> Database	63
Figure 5.4	Implemented Dimension and Fact Tables in Azure Synapse Analytics	64
Figure 5.5	Python Execution Output for Dataset Expansion and Target Business Anomaly Seeding.	67
Figure 5.6	Windows PowerShell Terminal Verification of Bulk Blobs Data Ingestion into the Bronze Storage Container	68
Figure 5.7	Azure Portal Storage Browser Interface Confirming Successful Data Landing in the Bronze Container	69
Figure 5.8	Azure Data Factory Pipeline Workspace Configuring the Bronze-to-Silver ETL Processing Workflows	70

Figure 5.9	Azure Portal Storage Browser Verifying Clean Structure and Population within the Silver Tier Container	71
Figure 5.10	Python Code Snippet of 12 Month Consumption Vendor	73
Figure 5.11	Python Code Snippet of Stagnation Tracking Engine	73
Figure 5.12	Python Code Snippet of Unfulfilled Production Orders	74
Figure 5.13	Python Code Snippet of Data Integration & Schema Hardening	75
Figure 5.14	Python Code Snippet of Algorithm Classification Engine	76
Figure 5.15	Python Code Snippet of Stockout Risk Identification	77
Figure 5.16	Python Code Snippet of BOM Bridge	78
Figure 5.17	Python Code Snippet of Downstream Customer Impact Trace	78
Figure 5.18	Azure Portal Storage Browser Verifying the Generation of the Gold Delta Table Directory	79
Figure 5.19	Phase A Content Screenshot	81
Figure 5.20	Phase B1 Content Screenshot	82
Figure 5.21	Phase B2 Content Screenshot	83
Figure 5.22	Phase C Content Screenshot	84
Figure 5.23	Phase D Content Screenshot	85
Figure 5.24	Phase E Content Screenshot	87
Figure 5.25	Phase F Content Screenshot	88
Figure 5.26	Phase G Content Screenshot	89
Figure 5.27	Verification Phase Content Screenshot	91
Figure 5.28	Azure Portal Storage Browser Verifying Clean Structure and Population within the Curated Tier Container	91
Figure 5.29	Power BI Semantic Model Galaxy Schema Blueprint	93
Figure 5.30	Code for Counting Below Reorder Level	94
Figure 5.31	Code for Counting Dead Stock Quantity	94
Figure 5.32	Code for Counting Expired Stock Quantity	95
Figure 5.33	Code for Counting Recoverable Assets Quantity	95
Figure 5.34	Code for Counting Total Provision Amount	96

Figure 5.35	Code for Counting Impacted Customer	96
Figure 5.36	Final Power BI Dashboard	97
Figure 5.37	Azure Storage Browser in ADLS Gen2	98
Figure 5.38	Azure Data Factory Studio: Pipeline1	99
Figure 5.39	Azure Data Factory Studio: Monitor Hub	99
Figure 5.40	Azure Databricks Workspace: SilverToGold Notebook Editor	100
Figure 5.41	Azure Synapse Studio Develop Hub: Curated SQL Scripts	101
Figure 5.42	Azure Synapse Studio SQL Editor: Results Grid and Messages	101
Figure 5.43	Inspecting input and output schemas within the ADF data flow to validate correct data type casting for material dataset	103
Figure 5.44	Test on Category	104
Figure 5.45	Configuring the Aggregate transformation in Azure Data Factory to group by primary keys and remove the duplicate records from the inventory snapshot.	104
Figure 5.46	Data Flow execution metrics confirming the successful filtering of duplicate and null records before writing to the Silver Parquet sink	105
Figure 5.47	Unit Test Passed	108
Figure 5.48	Reconciliation Test Passed	108

LIST OF ABBREVIATIONS

PPG	-	Pittsburgh Plate Glass Company
RA	-	Recoverable Assets
MC	-	Magna Carta
ERP	-	Enterprise Resources Planning
SAP	-	System Applications and Products in Data Processing
CRM	-	Customer Relationship Management
UTM	-	Universiti Teknologi Malaysia
SCM	-	Supply Chain Management
IRM	-	Inventory Risk Management
ADF	-	Azure Data Factory
DAX	-	Data Analysis Expression

CHAPTER 1

INTRODUCTION

1.1 Introduction

Managing inventory has never been a simple task, especially in manufacturing. Stock materials for too long and you are burning through storage costs while watching them inch closer to their expiry date. Wait too long to reorder and you are staring at a halted production line with customer deadlines passing you by. For PPG, this balancing act centers on two specific problem areas. The first is what the company calls "Recoverable Assets," which are materials sitting untouched for more than a year. The second is "Magna Carta" inventory, stock that has essentially reached the end of its usable life. Both represent real money walking out the door.

The bigger problem is how these materials are currently tracked, or rather, how they are not. Teams are piecing together information manually across separate procurement, manufacturing, and sales systems. It is slow, it leaves room for mistakes, and by the time anyone flags a concern, the loss has usually already happened. Management ends up reacting rather than planning.

This project is built to change that. The idea is to build a fully automated data pipeline using Microsoft Azure, one that pulls in raw daily data from all the relevant sources and processes it in stages. Along the way, the system applies PPG's own inventory rules to flag risky stock automatically, calculates the financial impact in real terms, and identifies which customer orders are at risk of delay due to material shortfalls. The output feeds directly into a Microsoft Power BI dashboard, giving the team a clear, live picture of where things stand without anyone having to chase down spreadsheets or cross-reference reports by hand. The goal is simple: less manual effort, fewer surprises, and better decisions before problems get expensive.

1.2 Problem Background

PPG is a paint manufacturer that requires efficient inventory control of its basic materials. The business manages huge amounts of data from a variety of sources, including purchase orders, production usage, monthly inventory records and consumer sales orders. Accurately identifying raw materials with excess stock that exceeds 12 months of consumption as Recoverable Asset, then materials categorized as Magna Carta dead stock that has not been used for nine months or longer and Magna Carta expired for materials that is already expired based on lot expiration dates is challenging. It is also difficult to determine the overall financial backup needed for outdated products. Additionally, it is still difficult to identify which open customer sales orders will be affected by future shortages and to identify products that are at risk of stockout.

This project addresses these challenges by designing and implementing an end-to-end data engineering pipeline using Microsoft Azure and Power BI. The pipeline will process the raw data, apply the necessary business logic for inventory analysis and produce an analytics-ready dashboard to support better Recoverable Assets and Inventory Risk Management.

1.3 Problem Aim

The aim of this project is to design and implement end-to-end data engineering pipelines for PPG to identify inventory obsolescence and expired materials.

1.4 Project Objectives

- (a) To provide actionable insights into stock utilization, financial management and impact of material shortages on customer sales orders by applying the Recoverable Assets (RA) and Magna Carta (MC) business logic.

- (b) To create analytical ready data by developing a three-tier data pipeline (Bronze, Silver and Gold).
- (c) To enhance financial monitoring by calculating the total provision amount of Magna Carta materials.
- (d) To have an accurate data-driven visualization by having an effective dashboard.

1.5 Project Scope

- (a) This project will use 6 synthetic CSV files that simulate PPG's operational system, which are material.csv, inventory_snapshot.csv, purchase_order.csv, production_orders.csv, supplier.csv, and sales_order.csv.
- (b) The pipeline is limited to three-layer structure. The Bronze layer will import the processed raw data into data lake and conduct basic data quality inspection. The Silver layer involves data transformation to calculate 12 months of consumption, apply RA/MC business logic and track the impact of sales orders. The Gold layer will load the transformed data into a data warehouse that uses star schema that centred on the fact_inventory_status fact table.
- (c) This project will use Microsoft Azure for data engineering tasks and Microsoft Power BI for data visualisation development.
- (d) The final output of this project will be an analysis dashboard which is designed to answer specific business questions from identifying materials below the reorder level to calculating the total financial write-down amount.

1.6 Project Importance

- (a) This project is very important because it can assist the companies in transforming raw data into useful information which can be used for making better business decisions. The main one would be the ability to automatically detect expired or long dormant "Magna Carta" materials, which improves

financial accuracy by helping enterprises to accurately determine the amount it should recognise as losses.

- (b) Secondly, the project helps to save money by identifying "Recoverable Assets" which are the materials that have been used for more than 12 months. Once the situation is known, enterprises can cut down on unnecessary inventories and allocate the money to other uses. In addition, the system can also issue a inventory shortage risk warning to managers before the materials are about to run out. Finally, the system allows companies to easily identify orders from their customers that will be delayed due to material shortages, thereby enhancing customer service and planning.

1.7 Project Organisation

Table 1.1 shows this report is organised in six distinct chapters.

Table 1.1 Project Organisation

Chapter 1	Introducing the project case study, covering the problem background, aims, objectives, and the end-to-end data pipeline proposal
Chapter 2	Describes a detailed literature review of related studies, fundamental theories, and technology that will be used in this project
Chapter 3	Details the methodology, explaining the chosen approach, the phases of the framework development, and the hardware and software requirements needed for execution.
Chapter 4	Focuses on analysis and design, data architecture breakdown, requirement analysis, and the database and interface designs
Chapter 5	Implementing the solution, testing the prototype, and code reporting of the system to ensure the frameworks work correctly.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter analyses in detailed the literature related to the establishment of end-to-end data pipelines for inventory risk management. It is important to go beyond specific technologies and focus on the basic concept that control the end-to-end data lifecycle in order to create a solid foundation (Reis & Housley, 2022). Therefore, this overview will review the key theories of data engineering and define data engineering as the development, implementation and maintenance of systems and processes that accept raw data and provide high quality and consistent data for analysis use cases. It will explore data modelling concepts such as star schema as well as the evolution of the field from managing complex overall big data clusters to using highly abstract simplified cloud architecture. By studying these technical sources, this chapter establishes the theoretical framework required to design the proposed bronze, silver and gold three-layer data pipeline and the analytical dashboard for the detection of PPG inventory out.

2.2 Supply Chain Management

Supply chain management has emerged as one of the most significant strategic concepts in modern business. Historically, D.J. Thomas and P.M. Griffin, (1996) mentioned the three fundamental stages of the supply chain, which are procurement, production, and distribution, have been managed independently, buffered by large inventories. However, the landscape has shifted considerably. One of the most significant paradigms of shifts of modern business management is that individual businesses no longer compete as solely autonomous entities, but rather as supply chains; business management has entered the era of internetwork competition.

2.2.1 Introduction and Definition of Supply Chain Management

Defining SCM precisely has proven challenging, as the literature offers multiple perspectives. The literature is replete with buzzwords such as integrated purchasing strategy, integrated logistics, supplier integration, and supply chain synchronization to address elements of this new management philosophy. Among the more comprehensive definitions, the Global Supply Chain Forum defines SCM as the integration of key business processes from end user through original suppliers that provides products, services, and information that add value for customers and other stakeholders.

From a broader perspective, SCM encompasses the entire value chain, from the extraction of raw materials to its end of useful life and may even extend to include recycling or re-use. More operationally, SCM as described by Thomas D.J. and P.M. Griffin (1996) is the management of material and information flows both in and between facilities, such as vendors, manufacturing and assembly plants, and distribution centers as shown visually through Figure 2.1.

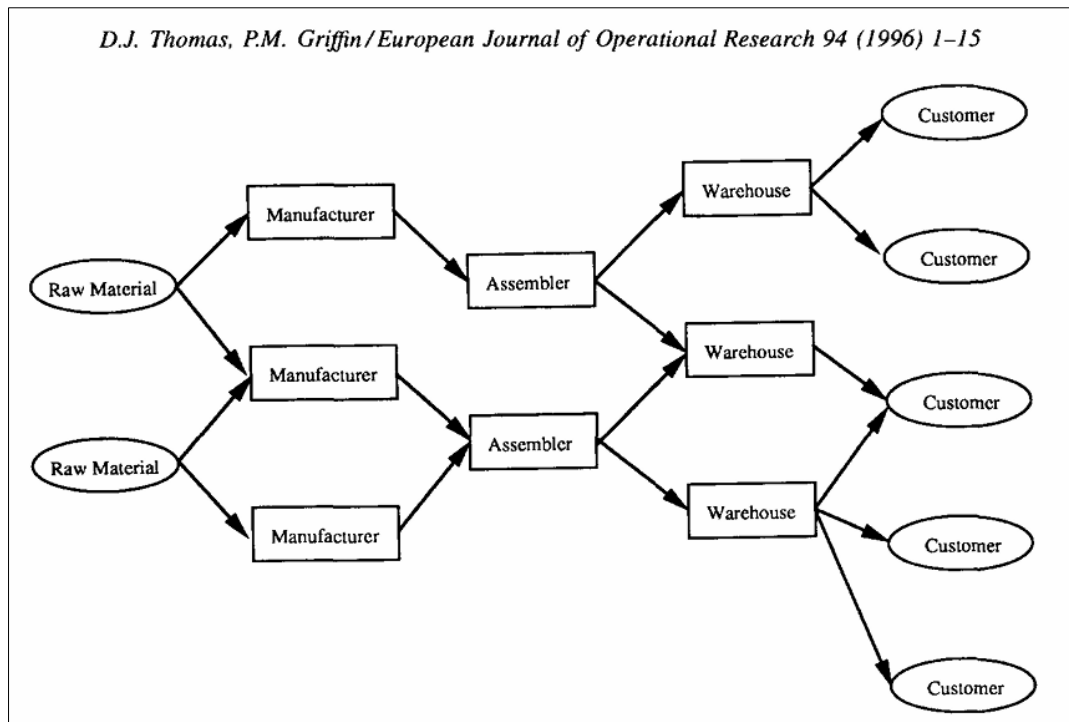


Figure 2.1 Schematic of a generic supply chain (Thomas D.J. and P.M. Griffin, 1996)

2.2.2 Evolution of Supply Chain Management

The development of SCM has followed a distinct historical trajectory rooted in two separate disciplines. As mentioned by K.C. Tan (2001), in the 1950s and 1960s, most manufacturers emphasized mass production to minimize unit production cost with little product or process flexibility. New product development was slow and relied exclusively on in-house technology and capacity.

The intense global competition in the 1980s forced organizations to offer low-cost, high-quality, and reliable products with greater design flexibility. Manufacturers utilize just-in-time (JIT) and other management initiatives to improve manufacturing efficiency. It was in this environment that SCM emerged as manufacturers began experimenting with strategic partnerships with immediate suppliers.

The evolution continued into the 1990s as organizations further extended best practices to include strategic suppliers and logistics in the value chain. From a literature standpoint, SCM developed two separate paths, purchasing and supply, and

transportation and logistics which eventually merged into a unified body of literature with a common goal of waste elimination and increased efficiency.

2.2.3 The Drivers of Supply Chain Coordination

A central motivation for SCM stated by Thomas D.J. and Griffin P.M., (1996) is cost reduction through coordination. In the United States, annual expenditures on non-military logistics are estimated at \$670 million, representing over 11% of the Gross National Product. With logistics costs of 30% of cost of goods sold not uncommon for U.S. manufacturing firms, potential savings in coordination cannot be ignored.

Beyond cost, managing uncertainty is a critical driver. In Figure 2.2, three distinct sources of uncertainty of plague supply chains: suppliers, manufacturing, and customers. Davis (1993) demonstrated this through Hewlett-Packard's experience, noting that inventory exists more or less as simple insurance against uncertainty. A systems-level view is necessary to properly diagnose these uncertainties, as a fragmented approach can lead to downstream inventory reductions being canceled by upstream increases.

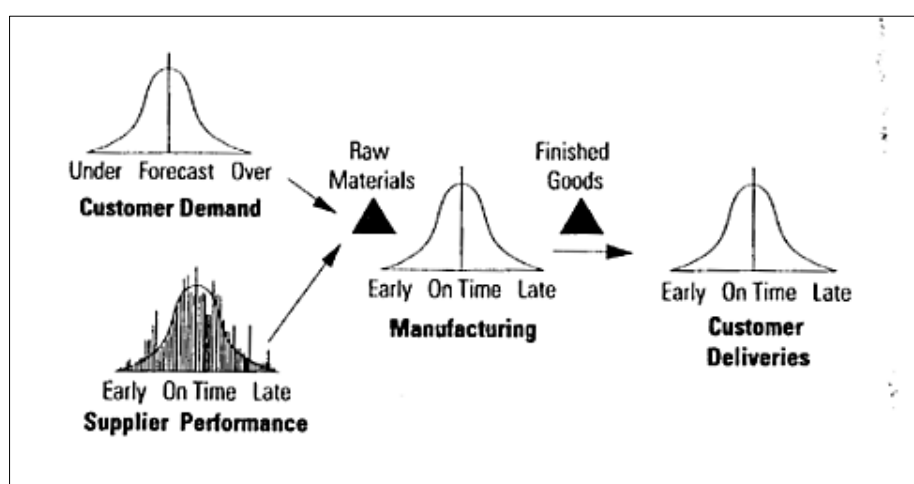


Figure 2.2 Uncertainty in the Supply Chain (Davis, 1993)

2.2.4 The Seven Principles of Supply Chain Management

Practical guidance on the supply chain was concluded into seven core guidelines by Anderson, Britt, and Favre (1997). Effective supply chain leaders conceptualize the supply chain from their suppliers' suppliers to their customers' customers, striving for material results centered on revenue growth, asset utilization, and cost.

- (a) **Principle 1 (Segmentation):** This principle emphasizes segmenting customers by their needs to develop a portfolio of tailored services.
- (b) **Principle 6 (Technology):** Firms need a supply chain-wide technology strategy that integrates day-to-day transaction handling, mid-term planning, and long-term strategic analysis.
- (c) **Principle 7 (Performance):** Excellent managers measure service in terms of the "perfect order," viewing performance from the customer's perspective.

2.2.5 Supply Chain Business Processes and Integration

Successful SCM necessitates a change from functional thinking to process integration, according to Lambert and Cooper (2000). Customer relationship management, demand management, and production flow management are the three main supply chain processes that the Global Supply Chain Forum has recognized. Importantly, not every process link requires the same level of management; managers can allocate scarce resources by identifying four essentially distinct types of business process links.

2.2.6 Gaps and Future Research Directions

This literature review on SCM has exposed the topic to some unresolved problems that exist:

- (a) **Transportation Cost:** Although many study models use unrealistic linear cost functions, Thomas and Griffin (1996) pointed out that transportation costs frequently account for more than half of total logistics costs.
- (b) **Environmental Sustainability:** The impact of supply chain operations must be modeled as environmental regulations tighten, using SCM concepts for recycling and reverse logistics.
- (c) **Integration Guidance:** To advise practitioners on the kind and degree of integration that ought to be applied to each process link, Lambert and Cooper (2000) advocated for normative research.

The literature examined here shows how SCM evolved from autonomous functional management to an integrated, cross-organizational field. A recurring element appears in all these sources: the performance of a company's complete supply chain network is becoming increasingly crucial to its competitive advantage.

2.3 Enterprise Resource Planning

Based on Simović et al. (2021), ERP system is an integrated suite of applications that is designed to manage the major business process activities of an organization. Usually, processes in ERP are purchase order fulfilment and manufacturing. As mentioned by Berti et al. (2023), The primary purpose of applying ERP in an organization is for data centralization where ERP gathers information from many departments such as finance, procurement, sales, inventory, production, HR, etc into a shared database to ensure everyone is accessing from a single source of truth. The next purpose is for process standardization were having consistent procedures across enterprises in codifying best-practice workflows (e.g., order-to-cash, procure-to-pay). Furthermore, it is for real time visibility. Applying an ERP makes an enterprise able to integrate dashboard and reporting where it could provide up-to-date insights into key performance benchmarks. Lastly, applying an ERP can create collaboration and integration with external system such as CRM, supply-chain partners, and analytics tools.

Example in previous study, Based on Berti et al. (2023), SAP ERP stores all transactional and master data in a relational database. To switch this raw data into a usable stream for process mining analytics, building the data pipeline is required. The pipeline runs in four stages. Firstly, process identification where the analysts choose a business process and a representative object type or table. Secondly, table selection stage where from the database, it would generate a Graph of Relationships (GoR). In this graph, it would show tables, attributes, object type, transaction, and the various arcs (foreign-key, primary-key, etc.). Thirdly, extraction and preprocessing where SQL queries are used to retrieve the rows of the selected tables. Users are allowed to filter entries on attributes before extraction. Lastly, Event-log construction (Blueprints) where it would map table entry to a primitive event such as activity, timestamp, related objects. The primitive events are then converted into an OCEL-compliant object-centric event log.

Then, the pipeline is acknowledged by the interactive SAP explorer where the schema is being loaded into an eo4j graph database for fast GoR navigation, a web interface guides table selection, and a Python component executes the extraction queries and builds the OCEL log.

Overall, this stage pipeline mitigates the size and complexity of SAP ERP data, avoids the convergence problem of traditional single-case logs, and generates a more scalable, object-centric view ready for analytics. Below is the example of SQL queries to extract different categories of arcs of the GoR built on top of SAP ERP.

Table 2.1 SQL queries to extract different categories of arcs of the GoR built on top of SAP ERP

Category	SQL query
<i>attribute arcs</i>	SELECT TABNAME, FIELDNAME FROM DD03L WHERE KEYFLAG != 'X' AND CHECKTABLE = ' '
<i>primary key arcs</i>	SELECT TABNAME, FIELDNAME FROM DD03L WHERE KEYFLAG = 'X' AND CHECKTABLE = ' '
<i>foreign key arcs</i>	SELECT TABNAME, FIELDNAME FROM DD03L WHERE CHECKTABLE != ' '
<i>domain arcs</i>	SELECT DISTINCT FIELDNAME, DOMNAME FROM DD03L
<i>object type arcs</i>	SELECT DISTINCT TABNAME, OBJECT FROM TCDOB
<i>transaction arcs</i>	SELECT DISTINCT TABNAME, TCODE FROM CDHDR a JOIN CDPOS b ON a.MANDANT = b.MANDANT AND a.CHANGENR = b.CHANGENR
<i>relationship arcs</i>	SELECT DISTINCT TABNAME, CHECKTABLE FROM DD03L

2.4 Medallion Architecture

The Medallion Architecture is a data design pattern used to organize data into a logical structure as it moves through a data lake. This framework is divided into three distinct layers: Bronze, Silver, and Gold. According to Databricks (2023), the primary goal of this architecture is to improve data quality and structure at each stage before it is used for final reporting.

- (a) **Bronze Layer:** This is the entry point for storing the data in six synthetic CSV files in a raw, unprocessed format. The main focus is to maintain real and raw data, that is, the data remains in its original state in the source system for reprocessing when needed. Simple quality control will be performed, including checking for missing data and consistency of data type.
- (b) **Silver Layer:** This layer is an intermediate layer where data is cleaned, filtered, and joined. In this project the silver layer is responsible for applying “business logic” to calculate the material consumption for 12 months, and classify the materials as “Recoverable Assets” (RA) or “Magna Carta” (MC). This layer is used to transform the actual data into verified data to prepare for subsequent in-depth analysis.

- (c) **Gold Layer:** Gold layer organizes the refined data in a format that is appropriate for high performance reporting. In this project, the data will be loaded into the Galaxy Schema in the data warehouse.
- **Star Schema Modeling:** The Gold layer is using a Galaxy Schema, which enables multiple fact tables to use shared dimension tables. This design can accommodate various business processes and minimise data duplication and enhance analytical performance.
 - **Supply Chain Risk:** Supply chain risk management deals with the identification, evaluation and mitigation of risks that could impact the ability of the supply chain to perform. For this project, it is forecasting materials shortages, and tracking customer orders that could be impacted if there is a risk of materials. Active risk management is essential to move supply chain operations from reactive to proactive in order to avoid affecting customer commitments due to inventory shortage, as Christopher (2016) mentioned.
 - **Data Quality Management:** Data classification and validation are carried out to guarantee good data quality when transforming from Bronze to Silver. The data quality principles are rigorously applied in checking the original data inputs. Business rules can be supported with data quality integration in some Magna Carta business rules, where business decisions are not impacted by incomplete or ambiguous data. Loshin (2010) states that one of the keys to implementing and analysing business rules effectively is the quality of the data available.

2.5 Visualisation

Data visualization is essentially the bridge between what happens in the background and the decisions people actually need to make at work. In supply chain and inventory management, companies are dealing with a constant flood of raw data every single day. Trying to spot patterns, risks, or problem areas by scrolling through endless rows in a spreadsheet is not only exhausting but also a reliable way to miss things.

Visualization solves that by turning all those numbers into charts and interactive dashboards that people can actually read and act on quickly.

For this project specifically, having a visual layer means the team can keep an eye on the things that matter most, things like how stock is being used, where shortfalls might be coming, and what the financial exposure looks like at any given point. Rather than digging through raw files to find answers, anyone looking at the dashboard can spot problem inventory right away, whether materials have been sitting long enough to fall under the Recoverable Assets category or have gone completely dead and qualify as Magna Carta stock. Catching these things early means there is still time to do something about it before the losses pile up or production takes a hit.

On top of that, visualization is the natural finishing point for the Medallion Architecture the pipeline is built on. The earlier stages do the hard work of pulling in messy data and cleaning it up layer by layer. By the time it reaches the Gold layer, the data is structured and ready for reporting. Connecting it to a visualization tool at that point gives business users one reliable place to go for answers. All the complex processing that happens behind the scenes becomes an interface that is straightforward to navigate, making it easy for management to understand the real cost of dead stock and see exactly how material shortages could affect customer orders down the line.

2.6 Comparison of Previous Studies

The previous research related to this project can be roughly divided into two categories: traditional supply chain management (SCM) theory and modern data engineering practice.

2.6.1 Traditional SCM & Uncertainty

Early research on supply chain management mainly focusses on the conceptual framework and the physical flow of goods. For example, Thomas and Griffin (1996) emphasised the need for supply chain coordination to reduce overall logistics costs. Similarly, Davis (1993) pointed out that inventory is usually only an insurance

measure to deal with the uncertainty caused by suppliers, manufacturing processes and customer needs. Although these basic theories establish the vitality of inventory management and supply chain coordination, they mainly provide strategic business guidelines and lack the automated technical solutions required for real-time inventory tracking.

2.6.2 Process Integration & Risk

In order to address the vulnerability of the supply chain in a more practical way, subsequent literature began to emphasise proactive risk mitigation measures and strict requirements for data reliability. Later research turned to regard the supply chain as a comprehensive network. Anderson, Britt and Fafer (1997) and Lambert and Cooper (2000) advocate cross-organisational process integration rather than isolated functional management. Also, Christopher (2016) pointed out that active risk management is crucial to transform supply chain operations from passive emergency handling to active risk mitigation, which ultimately ensures that customer commitments will not be affected by unexpected inventory shortages. From a technical point of view, supporting this operational transformation is the view of Lawson (2010), who said that strict data quality management is a mandatory requirement. Implementing strict data quality protocols can prevent ambiguous or incorrect data from disrupting the core business logic used to identify these inventory risks. The limitation of this forward-looking risk mitigation approach is that organizations require a centralised, reliable data platform to effectively implement these proactive strategies

2.6.3 Data Engineering & Architecture

In recent years, many documents have explained how to deal with the large amount of data needed to support these forward-looking strategies from a technical perspective. Rees and Hosley (2022) define data engineering as the development and maintenance of a system that can receive raw data and continuously provide high-

quality information for analytical use cases. Based on this concept, Deberk (2023) officially proposed the “Medallion Architecture”, showing how to systematically improve the quality and structure of data with the flow of information in the special bronze, silver and gold layers in the data lake. Although these major technological advances are remarkable, there are still obvious gaps in the literature: traditional supply chain management research lacks automated technical implementation, while modern data engineering frameworks are usually highly universal and are not directly applied to specific manufacturing inventory problems.

Table 2.2 Comparison of Previous Literature

Focus	Sources	Contributions	Limitation
Traditional SCM & Uncertainty	(Thomas & Griffin, 1996), (Davis, 1993)	Measures to reduce costs and control inventory have been clearly defined as a buffer strategy to deal with uncertainty.	Lack of modern and automated technology solutions for real-time tracking.
Process Integration & Risk	(Lambert & Cooper, 2000), (Christopher, 2016)	Shift the focus to process integration and proactive risk management	A centralised data platform is needed to effectively implement these forward-looking strategies.
Data Engineering & Architecture	(Reis & Housley, 2022), (Databricks, 2023)	Defines the Medallion architecture of data life cycle management and data quality	It needs to be applied in specific business logic such as recoverable asset rules to realise business value.

Table 2.2 shows that this report fills the gaps in existing research by combining the traditional principles of supply chain management risk with modern scalable data architecture. By applying the layered Medallion architecture in cloud platforms such as Microsoft Azure, this study transforms theoretical active risk management into an automated end-to-end data pipeline. This practical implementation not only processes the original operational data, but also specifically applies targeted business logic, such as identifying recoverable assets and calculating the backlog inventory terms stipulated in the Magna Carta to provide actionable insights for analysis, thus supporting comprehensive inventory risk management.

2.7 Azure

Microsoft Azure has been one of the leading platforms for a public cloud computing platform as it offers multiple cloud services tailored for big data analytics and modern data engineering. In the context of supply chain and inventory risk management, Azure provides solutions that are highly scalable, secure and elastic. With its integrated ecosystem, it allows for seamless transition from fragmented, manual tracking to workflow automation, end-to-end data pipelines. For this project, four core Azure services will be in used which includes Azure Data Factory, Azure Databricks, Azure Synapse Analytics, and Azure Data Lake Gen2 (ADLS Gen2)

Azure Data Factory will act as the orchestrator of the Medallion Architecture data pipeline. It will enables data-driven workflows to move the data between on-premises and cloud data stores to support external compute resources like Azure Databricks for data transformation within silver and gold layer (Microsoft, 2023a). Specifically in this project, ADF is responsible for ingesting raw data from an on-premises SQL Server into the Bronze layer of the pipeline architecture, allowing the data to be loaded into data lake on a scheduled basis without manual intervention.

Azure Data Lake Storage Gen2 (ADLS Gen2) will further serves as centralized storage to store the data across different layers of the Medallion Architecture for both raw and structured data. In this project, the data lake will be in charged of persisting the Bronze, Silver and Gold layer as structured containers within ADLS Gen2 (Microsoft, 2024). Its hierarchical namespace capability allows the pipeline to organise data into clearly defined folder structures correspond to each Medallion layer, which in turn supports efficient access and transformation by downstream compute services.

Onto another service, Azure Databricks is used as the transformation engine and compute power for Silver and Gold layer stages of the pipeline. Within this project, Through the used of Databricks workspace notebook written in Python and compute cluster to apply the Recoverable Assets and Magna Carta business logic, perform data quality validations, calculate the 12-month consumptions of materials, and produce analytics-ready Gold layer datasets. Simplifying the flow, Databricks will performs

transformation steps by cleaning and standardizing the raw data into a format suitable for data-driven decisions.

Azure Synapse Analytics will further be used to query data on demand using serverless resources that are able to expand on scale. In this project, Synapse Analytics will contain the star schema for the inventory fact table and its surrounding dimension tables accessible for query. ASA also allows direct connect to the Power BI with the gold layer datasets which is the finalized structured and transformed data and allows query to be executed against the curated datasets without requiring a separately provisioned dedicated warehouse, resulting in a cost-effective and flexible analytical serving layer.

Taken together, the above five Azure services form a seamless solution that is highly secured and scalable. The Azure Data Factory facilitates the ingestion of data from on-premises locations. The ADLS Gen2 stores data at all levels of the pipeline. The Databricks service facilitates business transformation. The Synapse Analytics provides access to the Gold layer data for analysis. The Key Vault facilitates the governance of credentials.

2.8 Power BI

Microsoft Power BI is a self-service business intelligence tool that offers features such as data transformation, interactive visualization and collaborations. Advanced features like Power Query and DAX in Power BI can be used to make proper data preparation, data cleaning and data transformation (Frazer, 2024). The data transformation is important to produce high quality and no redundancy to ensure that the resulting visualizations are accurate and consistent. From these data transformations, the refined data will be used to make a dashboard. Power BI also comes with collaboration features, so reports generated from the power BI can be shared across all team members and viewed anywhere using the power BI mobile app. These capabilities enable organizations to transform raw data into meaningful insights and support data-driven decision-making (Chen et al., 2012).

2.9 Summary

This chapter studies the basic literature and theory of data engineering. By analysing the life cycle of data engineering, a clear structural framework is built for the whole process management from the original data source to the final consumption. The framework systematically covers all aspects of data generation, storage, reception, conversion and service. The literature shows that the industry has transformed from a complex environment of massive data to the current modular and abstract data stack and cloud data ecosystem (DataBricks, 2020).

The three-layer data architecture, the bronze layer, the silver layer and the gold layer, directly verifies its progress through this modern paradigm. This hierarchical method can import raw data, transform complex business logic such as identifying recyclable assets and calculating inventory shortage reserves under the Magna Carta, and separate and analysable services (Databricks, 2023). In addition, the evaluation of cloud-scale computing environments can verify whether public cloud platforms such as Microsoft Azure have the computing flexibility and on-demand scalability required to handle these transformations (Domo, 2026). When combined with Microsoft Power BI at the final service stage, these technologies provide the necessary functions for establishing an efficient and automated inventory risk management system.

CHAPTER 3

RESEARCH METHODOLOGY

3.1 Introduction

The methods and overall strategies used to implement the end-to-end data engineering pipeline for the PPG Recoverable Assets (RA) and Inventory Risk Management (IRM) projects are outlined in this chapter. The main purpose of this chapter is to describe the framework used for system development and to explain the methods, technologies, and processes used during the planning, analysis, design, and implementation stages.

In order to achieve the project goal, which is to automatically identify out-of-date inventory goods and expired materials, the system development is carried out following the Medallion Architecture approach. The data life cycle is divided by the framework into three stages: the bronze layer (raw data collection), the silver layer (data transformation and business rules), and the gold layer (star schema with analytical functions).

Furthermore, the reasons for choosing the Medallion approach are explained, and detailed descriptions of the requirements for hardware and software systems are provided. The use of pipeline orchestration with Microsoft Azure and final data visualisation with Microsoft Power BI is also emphasised.

3.2 Methodology Choice and Justification

For building the Recoverable Assets and Inventory Risk Management data pipeline, the team went with the Waterfall methodology, and honestly it was a straightforward

choice given how the project is structured. The scope was clear from day one: process six synthetic CSV files and apply a fixed set of business rules to identify inventory risks. There was no ambiguity around what needed to be built, which is exactly the kind of situation Waterfall handles well.

The step-by-step nature of the methodology also fits naturally with how a Medallion Architecture actually works. You cannot jump ahead. The raw data has to be properly ingested into the Bronze layer before anything gets transformed in Silver, and the Gold layer has to be solid before there is any point touching the Power BI dashboard. Each phase depends on the one before it, so a sequential approach is not just a preference here, it is practically a requirement.

Another reason the approach made sense is that Waterfall pushes you to think through the design and logic carefully before you start building anything. That meant the team could verify the data processing rules and business logic on paper before provisioning resources like Azure Data Factory and Databricks, which saves time and avoids rework down the line. With a fixed academic timeline to work within, having clearly defined phases and milestones also made it much easier to stay on track and make sure the analysis, design, implementation, and testing stages each got the attention they needed without things bleeding into each other.

3.3 Phases of the Chosen Methodology

This project adheres to a planned, phased implementation roadmap in order to effectively construct a robust, three-tier Medallion data architecture and methodically handle the inventory obsolescence issues at PPG Industries Malaysia. Every stage of the engineering workflow, from raw data intake to the delivery of final business intelligence, is methodically planned, tracked, and verified when it is divided into distinct development phases. This methodical methodology matches our data models with PPG's compliance criteria, lowers technical risk, and maintains data integrity throughout the pipeline phases.

The execution strategy used to construct the data platform is broken out in detail in this section.

3.3.1 Activities and Process Applied in Each Phases

The Waterfall model was chosen for this project because it is sequential and structured, which fits the strict specifications of creating a multi-tier golden data pipeline. Before moving on to high-level visualization, each stage serves as a prerequisite for the next, guaranteeing that the data integrity and business logic are verified.

1. Requirements Analysis and Specification

The Recoverable Assets (RA) and Inventory Risk Management (IRM) systems' business goals are converted into technical requirements at this phase, which forms the basis of the project.

- (a) Activities: Conducting a comprehensive analysis of the six synthetic source files to understand the schema and relationships between materials, consumption, and financial attributes.
- (b) Process: Creating the exact logical and mathematical definitions for Magna Carta (MC) classes, such as Expired materials based on lot dates and Dead Stock (9+ months inactivity).

2. System and Pipeline Design

In this stage, a technical blueprint for the MS Azure environment is created from the abstract requirements.

- (a) Activities: Designing the Medallion Architecture (Bronze, Silver, and Gold layers) to ensure a systematic flow of data from raw ingestion to analytics-ready structures.
- (b) Process: Building a Galaxy Schema data model with several fact tables that function as the sources of truth for several departments when reporting financial provisions and stockout concerns.

3. Implementation and Development

In this phase, the data engineering artifacts are constructed utilizing the designated technology stack.

- (a) Activities: Creating MS Azure ETL (Extract, Transform, Load) pipelines to manage raw intake into the Bronze layer and intricate transformations in the Silver tier
- (b) Process: Implementing the RA/MC business logic through SQL or Python scripts to automate the flagging of excess inventory and the quantification of financial write-downs.

4. Integration and Validation (Testing)

This stage is crucial in a waterfall method to ensure that the generated pipeline satisfies the needs of the industrial case study.

- (a) Activities: Performing data quality checks, including null detection and duplicate handling, and validating the output of the Gold layer against expected financial outcomes.
- (b) Process: Ensuring the Power BI dashboard accurately addresses the seven key business questions, such as identifying customer sales orders impacted by critical material shortages.

5. Operation and Maintenance (Final Delivery)

Formalizing project outcomes and submitting technical documentation are the main goals of the last stage.

- (a) Activities: Consolidating all architectural diagrams, data dictionaries, and transformation logic into the Technical Report in FYPI format.
- (b) Process: Preparing a professional slide deck to demonstrate the end-to-end data pipeline's efficacy in managing inventory obsolescence and financial risk.

3.3.2 Design Modelling

The design model of the project focusses on establishing data flow and storage mechanisms to effectively support inventory risk management. The main framework adopted is the Medallion Architecture which systematically simulates three different stages of the data life cycle, bronze layer, silver layer, and gold layer.

At the basic level, the bronze layer is responsible for extracting and storing raw data from six aggregate CSV source files. Its focus is on maintaining the authenticity of the data and conducting basic data quality checks. The silver layer acts as a transformation model. In this layer, the specific business logic of Recoverable Assets (RA) and Magna Carta (MC) slow-motion products is built and applied to filter, clean up and merge data. In the final analysis structure of the gold layer, the dimensional modelling method is adopted and the star schema is used.

This star schema will place a central fact sheet called the Inventory Status Fact Table in the middle of the structure as the only real source of reporting inventory shortage risks and financial preparation. This central fact table stores key quantitative indicators and is surrounded by various descriptive dimensional tables which provide descriptive background information. These surrounding dimensional tables come from the integrated source files of the operating system, including attributes related to materials, suppliers and outstanding sales orders. By separating quantifiable transaction data from external descriptive details, the model is very suitable for data warehouses and follow-up analysis functions. This simple and centralised structure ensures that when the data is sent to the visual dashboard, the system can effectively filter and organise information to answer specific business questions, such as calculating the financial impairment amount of non-order inventory or tracking the downstream impact of material shortages on customer sales orders.

3.3.3 Design Tools

The development and implementation of this end-to-end data pipeline depends on a specific combination of modern data engineering and business intelligence tools. These tools are chosen because they can handle large-scale data conversion and provide actionable insights for inventory risk management.

Microsoft Azure is the main cloud computing platform and infrastructure core of the project. As a leading public cloud service provider, Azure provides a large number of services designed for big data analysis and modern data engineering. In managing supply chain and inventory risks, Azure provides a highly scalable, secure and flexible environment. The platform is used to host the medal architecture and coordinate the ETL (Extract, Transform, Load) workflow to transmit data from the bronze layer to the gold layer. By using Azure's integrated ecosystem, the project has successfully achieved a seamless transition from a decentralised manual inventory tracking method to an automated end-to-end data pipeline. In this environment, the operations required to perform complex business logic, such as classifying recyclable assets and materials in the Magna Carta, can be carried out using programming tools such as SQL or Python.

For frontend visualisation and analysis components, Microsoft Power BI has been selected as a self-service business intelligence tool. Power BI can be directly connected to the final galaxy schema in the Azure to obtain detailed information. The software's built-in advanced functions, especially Power Query and DAX (Data Analysis Expression) are used to complete the final data preparation, cleaning and conversion tasks. This data conversion process is important to generating redundant and high-quality data, thus ensuring that the generated visual dashboard is both accurate and consistent.

In addition, Power BI has powerful collaboration functions. The inventory risk reports and dashboards developed can be safely shared with all management team members. Stakeholders, no matter where they are, can view and interact with these insights anytime and anywhere through the Power BI either in desktop or mobile

application. This enables organisations to convert raw data into meaningful data-driven business decisions without considering their actual location.

3.3.4 Gantt Chart

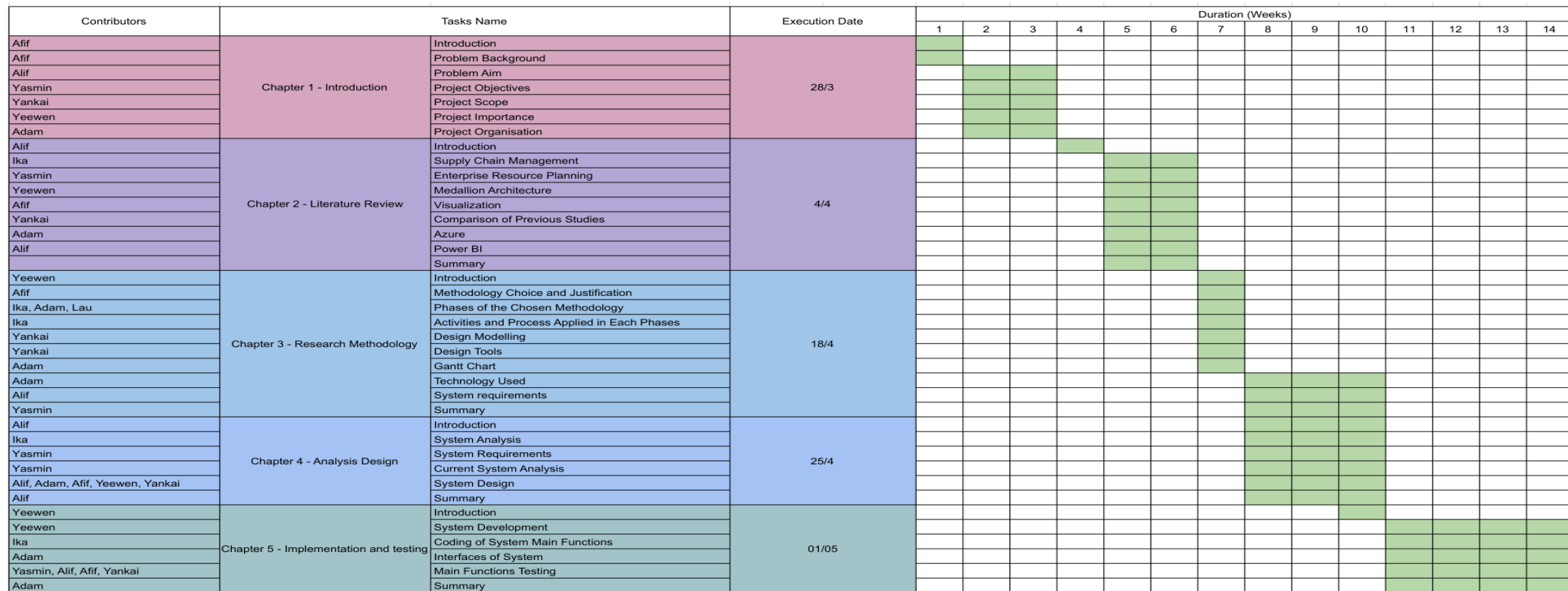


Figure 3.1 Gantt Chart for Project Progress

3.3.5 Technology Used

Microsoft Azure is the cloud computing platform for our project. It offers four services that we use throughout. Azure Data Factory helps move data from six CSV files into the Bronze layer on a schedule. This way we ingest the data directly into storage container without the involvement of Azure Data Factory, puts it into the Bronze layer. Azure Data Lake Storage Gen2 is where all of the data is stored. It keeps data for the three Medallion layers in folders that make sense for each stage. We use Azure Databricks to transform the data. In Azure Databricks it run Python notebooks on groups of compute clusters. These notebooks apply business rules for Assets and Magna Carta. They also check data quality. Create datasets ready for analysis. Azure Synapse Analytics is the data warehouse to model the schema. The data warehouse includes the fact_inventory_status table and related tables. It connects directly to Power BI. This direct connection lets people create reports easily without needing a warehouse.

Microsoft Power BI is where the project endpoint is. Here people can view data in a user-friendly way. Power BI gets data from Azure Synapse Analytics. It uses tools like Power Query and DAX to prepare data and ensure it is correct and not duplicated. With these tools it ensure our data is accurate and ready to use.

The final dashboard shows the risk of the inventory. Answers important questions like what materials are below the level where they should be reordered and how much money Magna Carta might lose. Microsoft Power BI also lets people share reports and dashboards, with the management team and they can see them anytime on their devices, which helps people make good decisions no matter where they are.

3.4 System Requirements

System requirements consist of two category which are software requirements and hardware requirements.

3.4.1 Software Requirements

Software Requirement is a detailed description of the intended purpose, features, functionality and other aspects of software needed in the system. Software Requirements that are needed in this project are:

- (a) Operating System: Windows 10/11. Required for full compatibility with Power BI Desktop and Microsoft SQL Server Management Studio.
- (b) Web Browser: Google Chrome. Required to access the Azure Portal and manage Azure Data Factory, Azure Databricks and Azure Synapse Analytics.
- (c) Power BI Desktop: The latest version for creating visualizations and DAX measures.
- (d) Microsoft SQL Server Management Studio: Used to manage local databases for connecting and importing the dataset to Azure Data Factory.
- (e) Azure Subscription: An active account to host the whole pipelines systems which are included Azure Data Factory, Azure Databricks, Azure Synapse Analytics, Azure Data Lake that contain three containers which are bronze, silver, and gold data layers and Key Vault.

3.4.2 Hardware Requirements

Hardware requirements refer to the minimum specifications for the system to run smoothly. Since the project was built on the Azure cloud, the local hardware requirements focus on the workstation needed to access cloud services and run Power BI.

- (a) **Processor (CPU):** At least an Intel Core i5 (8th generation) or an AMD Ryzen 5. The computer only needs enough power to run a web browser, Power BI Desktop and SSMS at the same time because all the heavy processes are used to use cloud.

- (b) **Memory (RAM):** At least 16 GB. Power BI Desktop minimum 16 GB RAM when building reports and display the report smoothly.
- (c) **Storage:** At least 100 GB SSD. Storage used for installing Windows, the software and store the CSV files before uploaded to the bronze container. SSD is chosen because it reads and writes faster than a hard disk.
- (d) **Network Connection:** A stable internet connection of at least 10 Mbps. A good connection is needed to use the Azure Portal in browser, upload the files and refresh the dashboard.

3.5 Summary

This chapter details the methodology and technical framework that are used in developing an end-to-end data engineering pipeline for PPG Recoverable Assets (RA) and Inventory Risk Management project. Waterfall model is used as a structured development process, which involves a sequence of steps, such as analysis, pipeline design, implementation, testing, and delivery. Furthermore, the system also built by following the Medallion Architecture, where it organize data lifecycle into three stages which are bronze layer for raw data ingestion, silver layer for applying RA/MC business logic and gold layer for hosting an analytics ready star schema, where the middle is an inventory status fact table. Then, the project would set up a workflow using Microsoft Azure for cloud orchestration and Microsoft Power BI for visualization to detect dead stock, act on the financial write-down and reduce customer sales order risk. Nevertheless, the system also needs to be complemented by an efficient software and hardware tools.

CHAPTER 4

ANALYSIS AND DESIGN

4.1 Introduction

This chapter explains the analysis and design of the Recoverable Assets (RA) and Inventory Risk Management (IRM) data pipeline. The chapter begins by introducing the overall system architecture, which illustrates the flow of data through the cloud, from the point of collection through to being displayed on the dashboard.

After that, the chapter looks at each part in more detail. It describes the data architecture and the Azure services that are used, with the bronze, silver, gold and curated that follow medallion architecture. It then introduces the need for a requirement analysis, the design of the project and the design of the database which influences the data presented in the curated layer for reporting. Last but not least, interface design demonstrates the presentation of results to the user as part of the Power BI dashboard. These combine to make the design that is constructed and tested in chapter 5.

4.2 System Analysis

To comprehend a system's present performance and constraints, system analysis, which a fundamental stage in the software and data engineering lifecycles involves examining operational workflows, business rules, and technological data generating points. The system analysis portion of this project creates a thorough understanding of how PPG Industries Malaysia's raw transactional files can be organized, cleaned, and integrated to reduce financial risks. This section offers the technical and operational foundation required to construct a three-tier cloud data architecture using the Medallion framework in Azure by breaking down the underlying

operational challenges, analyzing the departmental organizational layout, and describing the precise requirements-gathering techniques used.

4.2.1 Case Study

PPG Industries Malaysia, a well-known division of a worldwide leader in the production of high-performance paints, coatings, and specialty chemical products, is the subject of this project's industrial case study. Inventory obsolescence is a constant operational barrier that industrial production facilities operating at this size must deal with. This happens when raw supplies, like chemical solvents, pigments, resins, and packaging components, are bought in large quantities but kept in the facility's warehouse network for much longer than the firm can really use or sell. Keeping this excess inventory increases logistical expenses, holds up vital working capital, takes up physical storage space, and exposes the business to significant balance sheet write-downs when supplies deteriorate or expire.

PPG uses two different internal compliance and financial risk categories to categorize their stock holding profiles in order to methodically manage, monitor, and reduce this exposure:

- (a) **Recoverable Assets (RA):** This classification applies to any raw material stock on hand whose current physical inventory level in the warehouse exceeds its total actual or projected manufacturing consumption over a rolling 12-month period. An immediate "at-risk" buffer is represented by raw materials designated as Recoverable Assets. While they are still legally usable, they signal a supply-and-demand mismatch and are subjected to strict quarterly business reviews to determine if procurement actions must be frozen.
- (b) **Magna Carta (MC):** This is a critical sub-category of Recoverable Assets that represents highly stagnant or compromised inventory that is no longer deemed operationally viable. Magna Carta stock is divided into two distinct logical states:

- (c) **Magna Carta – Dead Stock:** Materials that are a subset of Recoverable Assets but have recorded zero actual consumption on the manufacturing floor for a consecutive period of 9 or more months, and do not possess a documented lot expiration date.
- (d) **Magna Carta – Expired:** Materials that are a subset of Recoverable Assets where the physical lot expiration date has passed relative to the pipeline evaluation anchor date.

From a corporate accounting standpoint, any inventory carrying value caught in either of the two Magna Carta states requires a 100% financial provision write-down. The financial exposure is calculated deterministically using the following mathematical model:

$$\text{Magna Carta Provision Amount} = \text{Quantity on Hand} \times \text{Unit Cost}$$

Currently, the PPG Malaysia plant operates without an automated, unified system to handle these computations. Because transactional records are still separated into separate text files, manual rolling computations are extremely time-consuming and prone to human mistake. This operational gap prevents plant operators from seeing how supply shortages in these categories affect pending customer sales orders, conceals financial exposure, and delays risk discovery. By automating data integration and risk assessment, the data engineering pipeline created for this project closes this precise systemic gap.

4.2.2 Company Organization Structure

A thorough grasp of the organizational departments inside the company is essential to a cross-functional data pipeline. Parts of the enterprise dataset are generated and used by several divisions within PPG Industries. The following describes the five main organizational entities in this ecosystem and how they relate to the data model operationally:

- (a) **Supply Chain & Warehouse Management Department:** This department is responsible for physical material storage and inventory control across the plant's various warehouse zones. They monitor important metrics including manufacturer lead times, reorder thresholds, and item uniqueness. In terms of operations, they create the materials.csv master records and log the cyclical snapshot intervals recorded in inventory_snapshot.csv, which captures lot expiration variables and precise physical stock levels.
- (b) **Production & Operations Planning Department:** The actual factory floor operations are managed by this section, which transforms raw chemical groups into final paints and commercial coatings. They compute material requirements and record daily usage volumes. The production_orders are filled in by their activity. The source of truth for rolling consumption analytics is a CSV table that records precise timestamps and the quantities consumed (quantity_consumed) per batch.
- (c) **Sales & Customer Relations Department:** This division records commercial requests from corporate distributors, handles market demand, and oversees client interaction. They keep track of distinct order identities, customer demographics, particular item variations, volume needs, and fulfillment state attributes like "Open" or "Delivered" in the sales_orders.csv ledger. In order to proactively determine which client orders are at risk because of raw material stockouts, this department mostly depends on the data engineering system.
- (d) **Finance & Corporate Accounting Department:** This section is in charge of the plant's operating expenses, quarterly margins, and overall financial situation. They compute financial provision write-downs using the pipeline's output and keep cost books (unit_cost). They create compliance risk reserve summaries for executive reporting, making them the main internal users of the Gold layer data warehouse.
- (e) **IT & Data Engineering Project Team:** The analytics team makes up the system architecture's technical core. This unit manages data orchestration using technologies like Azure Data Factory, cleans data types within Azure Databricks notebooks, enforces data restrictions, and serves optimized star

schemas to business intelligence interfaces rather than creating core business transactions.

4.2.3 Requirements Gathering Technique

The team employed two main methods to collect requirements from these departments in order to obtain explicit processing instructions and data formats:

- (a) **Document Analysis (Quantitative Schema Mapping):** The group conducted a thorough structural examination of the PPG-provided raw files. Six transactional files, materials.csv, inventory_snapshot.csv, production_orders.csv, purchase_orders.csv, supplier.csv, and sales_order.csv have to be examined and expanded. This method helps design the data-cleaning procedures for the Silver processing layer by identifying important associations, such as the use of material_id to link files, and exposing data problems, such as dates being saved as text strings and the missing expired date column.
- (b) **Joint Application Design (JAD) and Collaborative Briefing:** The technical developers and departmental business leaders held interactive briefing to align everyone on the project's final deliverables. These collaborative sessions helped define the **7 Core Business Questions**. This ensured that the final product has the exact metrics needed to answer these questions and feed the downstream Power BI dashboards.

To conclude the analysis, **Table 4.1** maps each requirement gathering technique to its primary discovery outcomes:

Table 4.1 Requirement Gathering Methodologies and Engineering Outputs

Requirement Gathering Technique	Primary Target Audience / Source Material	Direct Analytical & Engineering Output
---------------------------------	---	--

Document Analysis	Six operational CSV data files from transactional databases.	Identified schema formats, relational primary/foreign key attributes, row count, missing column and data type errors to be corrected in the Silver tier.
JAD & Briefing	Department Heads, Analysts, and Cloud Engineers.	Defined the 7 core business questions (Q1–Q7) and finalized data delivery and Power BI dashboard requirements.

4.3 System Requirement Analysis

System requirement analysis specifies the requirements for a data engineering system that automates PPG Malaysia’s classification of inventory obsolescence and stockout risk. It translates the business problem into functional requirements and non-functional requirements that the delivered system must fulfill.

4.3.1 Functional Requirements

Functional requirements describe what the system must do. At the ingestion stage, the system must load all six source files; *materials.csv*, *inventory_snapshot.csv*, *production_orders.csv*, *purchase_orders.csv*, *suppliers.csv*, and *sales_orders.csv* into the bronze layer by maintaining the original structure. In the silver layer, the system must produce a cleaned version of these tables that maintaining same structure while removing duplicate rows, resolving null values, correcting data types, and standardizing format such as converting text-stored dates into true date values and resolving the missing expiry information identified during document analysis.

The gold layer carries the business logic that the current manual process. The system must calculate each material’s expected twelve months of consumption from

the production records and flag material as a Recoverable Assets where its quantity on hand exceeds that consumption. Then, it must derive the two Magna Carta states from this Recoverable Asset population as dead stock, where material has recorded no lot expiry date and expired, where the lot expiry date has passed relative to the pipeline evaluation anchor date. The system must additionally detect materials at stockout risk by evaluating stock levels against consumption and supplier lead time and trace the consequences of the shortages through to the affected production orders and open customer sales orders.

Next, the curated layer must present these results in an analysis-ready star schema centered on fact_table, and supported by dimension table for materials, suppliers and time, and carrying all Recoverable Asset and Magna Carta fags together with their associated financial values. This curated model must be able to answer the seven business questions that are defined during JAD session. Finally, the system must show these answers through an interactive Power BI dashboard that presents the Recoverable Asset and Magna Carta overview to users. Table 4.2 summarizes these functional requirements.

Table 4.2 Functional requirements

ID	Functional Requirements
FR 1	Ingest all six source CSV files into bronze layer as-is, maintaining original structure and recording basic quality checks
FR 2	Produce a cleaned silver layer of identical structure where remove duplicate, resolve nulls, correct data type, and standardize date and unit formats.
FR 3	Calculate 12 months consumption per material from production records
FR 4	Flag material as Recoverable Asset
FR 5	Flag Magna Carta Dead Stock
FR 6	Flag Magna Carta Expired
FR 7	Detect materials at stockout risk using stock level, consumption and supplier lead time
FR 8	Trace at risk materials through to affected production orders and open sales orders.

ID	Functional Requirements
FR 9	Build curated star schema with supporting dimension table
FR 10	Answer the seven core business questions from the curated model
FR 11	Present the Recoverable Asset and Magna Carta overview through an interactive Power BI dashboard

4.3.2 Non-functional requirement

Non-functional requirements explained how the system should perform and how the system must satisfy a set of quality attributes. The bronze layer must guarantee data integrity by preserving raw inputs without modification, providing an immutable point of resolving every downstream value. Next, the rules used for the gold layer should be repeatable and should correspond exactly with the asset and magna carta logic defined, and all figures should point back to the source of the records. The pipeline should be flexible and easily modifiable, so that business thresholds (e.g. twelve months window, nine months window) can be changed without changing the design of the system. The dashboard must be usable by non-technical finance and inventory staff, and the overall architecture must scale to accommodate additional materials, suppliers and snapshot months without structural change. Table 4.3 lists these non-functional requirements

Table 4.3 Non-functional requirements

ID	Non-functional Requirement
NFR 1	Data integrity: The bronze layer preserves raw data exactly as an immutable backup
NFR 2	Data quality: The silver delivers deduplicated, correctly typed and standardized data before any business logic is applied
NFR 3	Correctness: classification and provision of results are reproducible and conform exactly to the defined RA/MC business rules.

ID	Non-functional Requirement
NFR 4	Traceability: every classification and financial value can be traced back to source records.
NFR 5	Maintainability: the pipeline is modular by layer so thresholds and rules can be changed without redesign
NFR 6	Repeatability: result can be generated on demand to support quarterly review and period-over-period comparison
NFR 7	Usability: the Power BI dashboard is clear and accessible
NFR 8	Scalability: the architecture accommodates additional materials, suppliers, and snapshots without structural change.

4.4 Current System Analysis

Before designing the proposed data pipeline, it's important to identify how inventory risk classification is performed at the PPG currently. The existing system does not consist of one application but more to a manual, file-driven process spread throughout the departments. All department generate their own transactional information and keep their own copies as separate text and CSV spreadsheets, which means that cross departmental perspective must be combine together manually.

Physical quantities on hand and lot expiration information was now being captured by the supply chain and warehouse management department, while material master data was now recorded into a special file, as well as a periodic snapshot of material stock in the warehouse. Different departments keep a ledger of customer orders and their order status, and then, record logs of orders by material consumption per batch in its own. Next, finance department wants to calculate the inventory risk for their quarterly review. The analyst must manually gather those files, match on common data points and then manually calculate the inventory consumption over last 12 months. Then, inventory counts performed by comparing the stock available with the consumption forecast, and the Magna Carta states are determined by analyzing the gaps in consumption and the expiration of lots, one by one. These result are written into spreadsheets and sent up for reporting.

This manual approach suffers from several weaknesses such as no single view of inventory risk exists and reconciling them is labour intensive and slow. Next, each cycle of the rolling calculations for the twelve month and nine-month threshold must be recomputed from scratch. The data quality problems compound the difficulty such as dates are frequently stored as plain text, expiry information is missing for some lots, and units and formats vary between files.

4.5 System Design

This chapter covers the system architecture, explanation of the system architecture components, systems UML diagrams, database design and interface design.

4.5.1 System Architecture

The System Architecture consists of five layers, which are data sources, data ingestion, storage layer, processing layer and serving layer as shown in Figure 4.1.

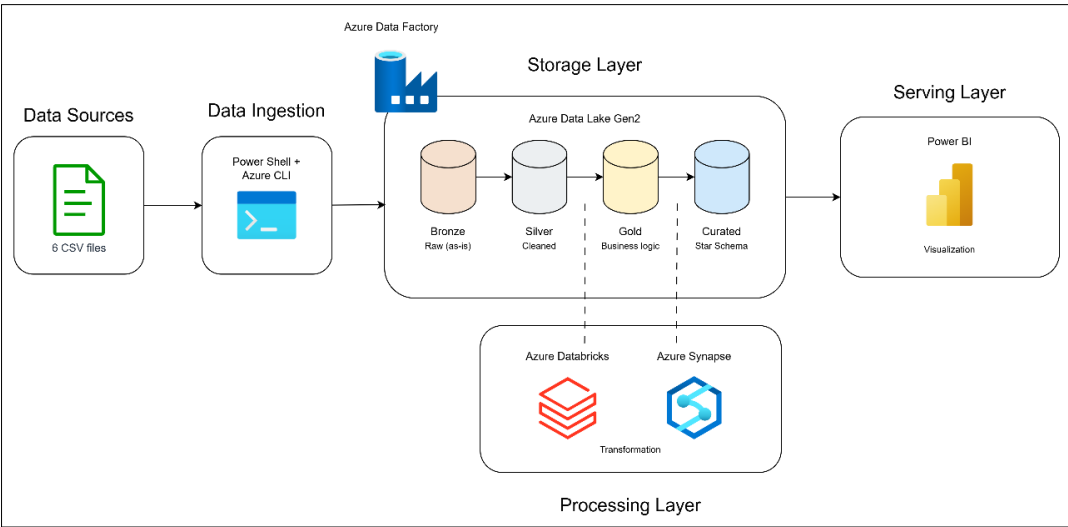


Figure 4.1 Data Architecture in PPG Recoverable Assets & Inventory Risk Management System

4.5.2 Explain each component in system architecture

The system architecture is structured into five parts that work together to move data from raw sources of csv files into the final representable dashboard. Within Medallion Architecture, each component performs specific role including ingestion, transformation, visualization

Data Sources and Ingestion. The pipeline begins with the creation of six synthetic datasets that simulates PPG's operational system's workflow, namely `inventory_snapshot.csv`, `material.csv`, `purchase_order.csv`, `production_order.csv`, `supplier.csv` and `sales_order.csv`. These files represent master data generated across PPG's procurement, production, and sales activities, and act as the single-entry point for all data entering data pipeline. After that, Azure CLI through PowerShell is utilize to ingest all the raw csv files into the storage container inside Azure Data Lake Gen 2 via Azure CLI command.

Storage Layer. Azure Data Lake Gen2 (ADLS Gen2) serves as the centralised data lake storage for the entire pipeline. The Bronze container inside the file storage system holds the raw data exactly as it received to preserved the source, the Silver container holds the cleaned data, Gold container holds transformed data based on the business logic applied to fulfil the schema modelling. The final curated data is then organised into the galaxy schema with multiple fact and dimension tables.

Processing Layer. The transformation process is carried out inside Azure Databricks, which act as compute and transformation engine of the pipeline. Using python notebook, databricks performs several processes including calculation of the 12-month material consumption and applies the Recoverable Assets and Magna Carta business logic to flag risky inventory. Azure synapse later host the resulting star schema creation in the table form via SQL script command serving on Serverless SQL Pool. All of the resulting schema generation are stored inside curated container are then queried by Power BI without provisioning a separate dedicated warehouse, which keeps the serving layer cost-effective and scalable.

Serving Layer. This layer connects Power BI directly to Curated container data exposed through Azure Synapse Analytics. It is utilized to deliver an interactive dashboard presenting indicators such as stock consumption, stockout risk, financial provision, at risk customer and production, enabling for management to monitor inventory risk and answer the 7 business questions defined in the project scope.

4.5.3 System Design (Data Engineering Process)

This subchapter consists of an activity diagram and a use case diagram for the system.

4.5.3.1 Use Case Diagram

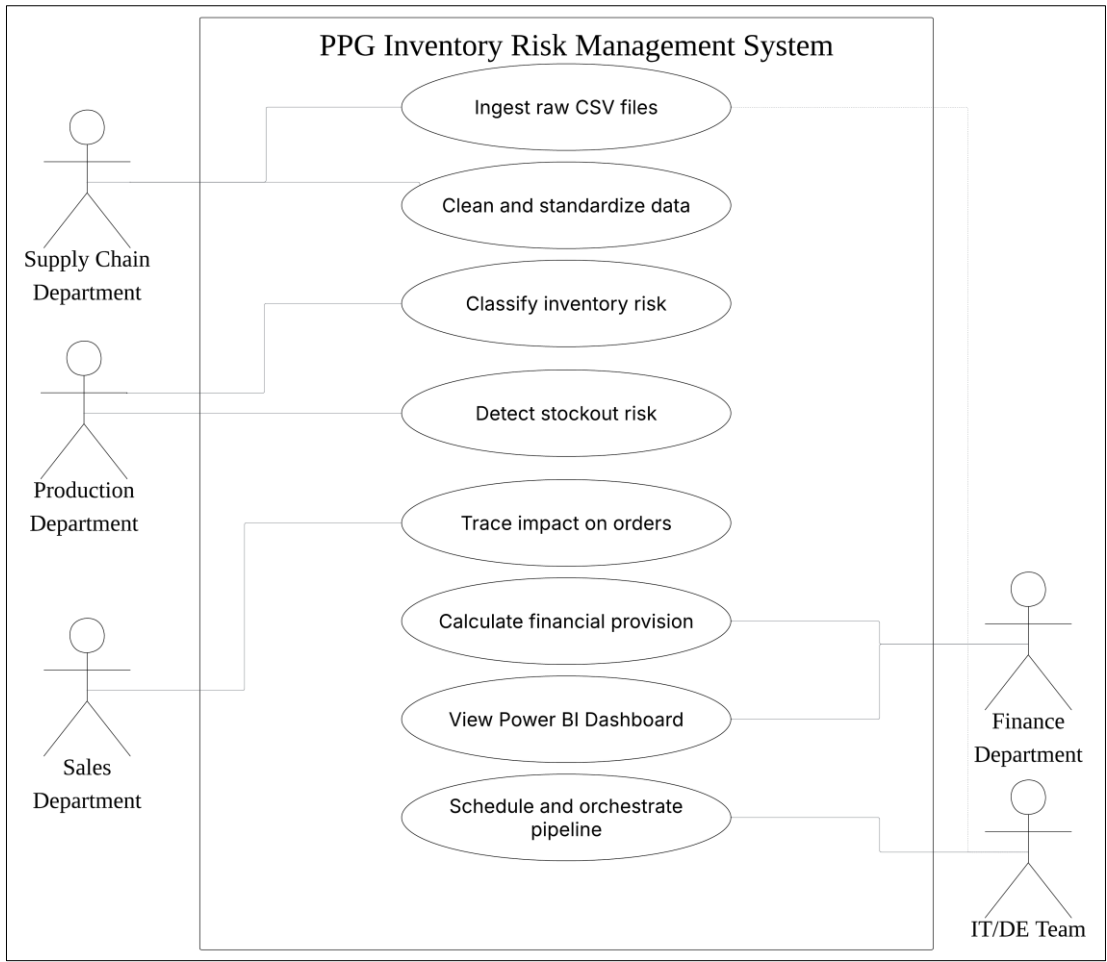


Figure 4.2 Use Case Diagram for PPG Inventory Risk Management System

Figure 4.2 shows the Use Case Diagram for the PPG Inventory Risk Management System. It maps out how the different users and departments interact with the system and what the system does.

The system boundary covers everything the platform handles, starting from when raw data first comes in, all the way to when the final dashboard is ready to view. Five actors are involved. On the left side are the Supply Chain Department, Production Department, and Sales Department. These are the teams whose day-to-day transaction records get fed into the pipeline. On the right side are the Finance Department and the IT and Data Engineering Team, who are the main users of the outputs and the ones responsible for keeping the system running.

Inside the boundary, eight use cases are defined. These walk through the entire pipeline: pulling in raw CSV files into the Bronze layer, cleaning and standardizing that data into the Silver layer, classifying inventory risk using Recoverable Asset and Magna Carta logic, flagging stockout risks, tracing how those risks flow downstream into production and sales orders, calculating financial provision write-downs, viewing the Power BI dashboard, and scheduling and managing the pipeline runs. Taken together, these eight use cases show that every functional requirement gathered during the requirements phase has been accounted for in the system design.

4.5.3.2 Activity Diagram

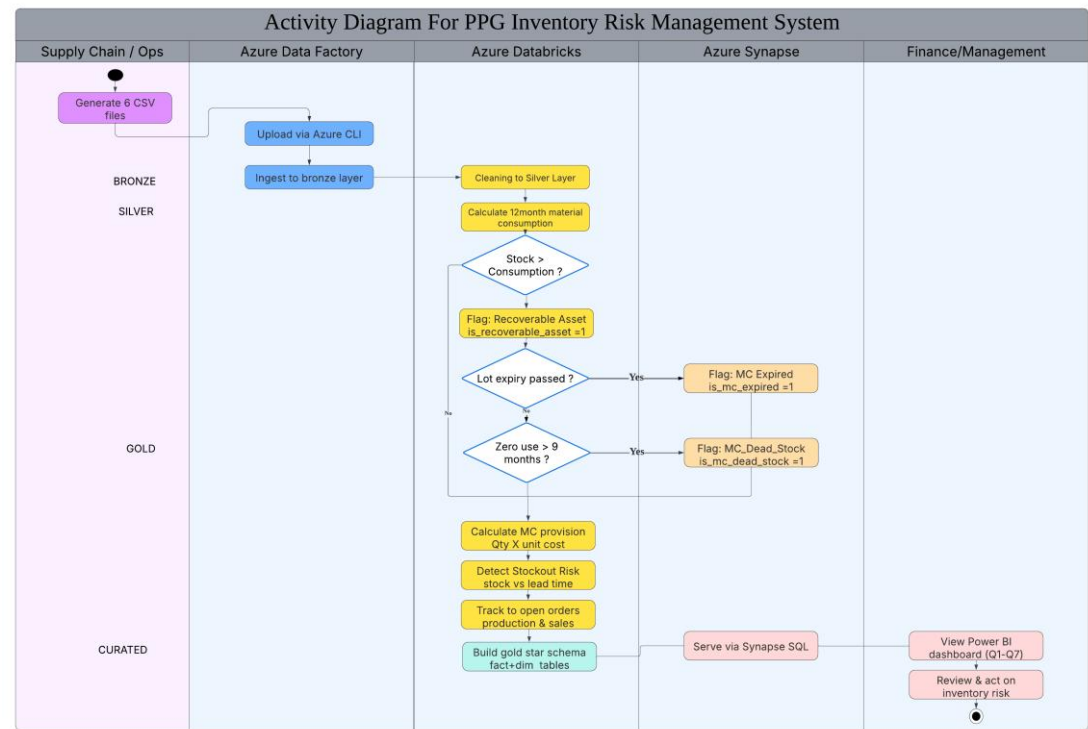


Figure 4.3 Activity Diagram for PPG Inventory Risk Management System

Figure 4.3 presents the Activity Diagram for the PPG Inventory Risk Management System. It shows the full pipeline workflow from start to finish, laid out across five swimlanes. The workflow starts in the Supply Chain and Operations swimlane, where six source CSV files are generated and made available as the single entry point into the pipeline. From there, the Azure Data Factory swimlane takes over. The files are uploaded to the Bronze container in ADLS Gen2 using an Azure CLI script and ingested as they are, keeping the raw data untouched in its original form.

The heaviest processing happens in the Azure Databricks swimlane, which covers both the Silver and Gold layers. The raw data is first cleaned, removing duplicates, resolving nulls, correcting data types, and standardizing date fields. The pipeline then calculates each material's rolling twelve-month consumption, which acts as the foundation for all the risk classification steps that follow. From there, a series of sequential checks are applied to flag Recoverable Assets, identify Magna Carta Dead Stock and Expired materials, compute financial provision write-downs, detect stockout risk, and trace material shortages through to the affected production and

sales orders. All of the enriched records are then brought together into a curated Gold layer star schema.

In the Azure Synapse swimlane, the star schema is made accessible through a Serverless SQL Pool, which provides cost-effective and high-performance query access for the Power BI semantic model. Finally, in the Finance and Management swimlane, the dashboard is rendered to answer all seven core business questions, giving management the information they need to act on inventory risks.

Overall, the activity diagram shows how the system takes six disconnected transaction files and turns them into a fully automated, audit-ready risk intelligence pipeline.

4.5.4 Database Design

Through the design of the database, the proposed inventory analysis system will provide an effective process for storing, retrieving and processing data for analysis, allowing data to be organised into a structured format. Based on the business requirements outlined above, the database is designed to enable inventory monitoring, identification of recoverable assets, dead stock analysis, tracking expired inventory, production order performance analysis and customer sales order impact assessment. To achieve these objectives, the multi-dimensional data warehouse approach is used in the Gold Layer, which will enable the system to effectively provide support for the generation of reports, business intelligence and business decisions. Figure 4.4 shows the relational diagram and figure 4.5 shows the galaxy schema in this system.

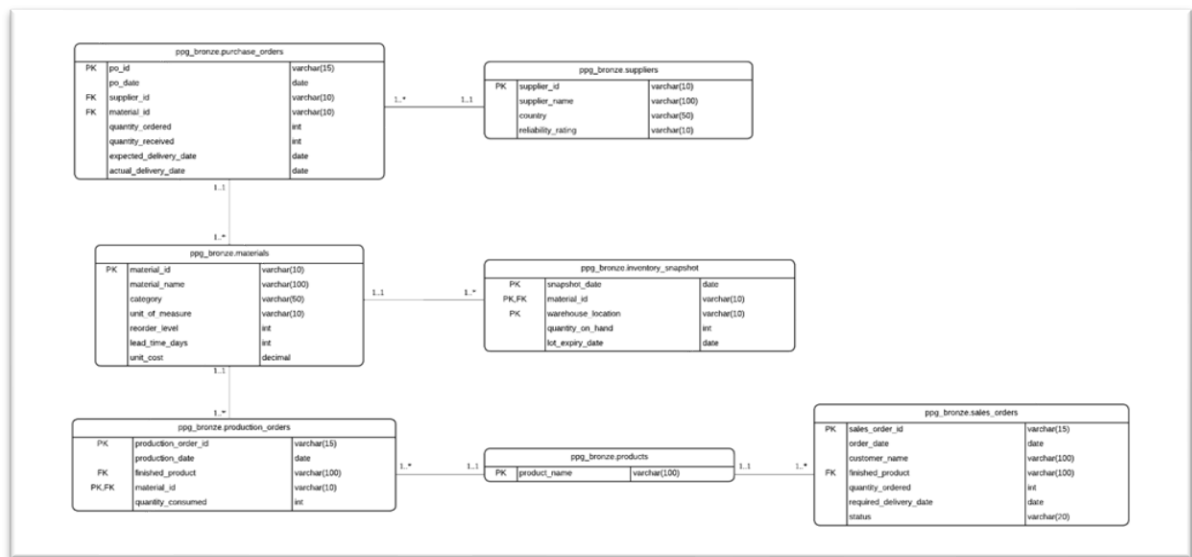


Figure 4.4 Relational Design for Recoverable Assets and Inventory Risk Management Data Warehouse

An Entity-Relationship Diagram (ERD) in figure 4.4 is used to see the relationships between the tables in the system. It visually shows how each table connects to the others like how tables link through primary keys and foreign keys to understand at a glance how data flows across suppliers, materials, products, purchase orders, sales orders, production orders and inventory.

Table 4.4 shows the relationship for each table. A Parent table contains a single list of unique items such as suppliers or materials and a child table records all activities involving such items such as orders or inventory logs that may occur on a daily basis. The child table contains a special column for tracking unique IDs (foreign key) that reference the unique ID on the parent table.

Table 4.4 Relationship for each table

Child (many side)	Parent (one side)	Meaning
purchase_orders.supplier_id	suppliers.supplier_id	Each PO is placed with one supplier.

Child (many side)	Parent (one side)	Meaning
purchase_orders.material_id	materials.material_id	Each PO buys one material.
inventory_snapshot.material_id	materials.material_id	Each stock record is for one material.
production_orders.material_id	materials.material_id	Each production line consumes one material.
production_orders.finished_product	products.product_name	Each production run makes one product.
sales_orders.finished_product	products.product_name	Each sales order is for one product.

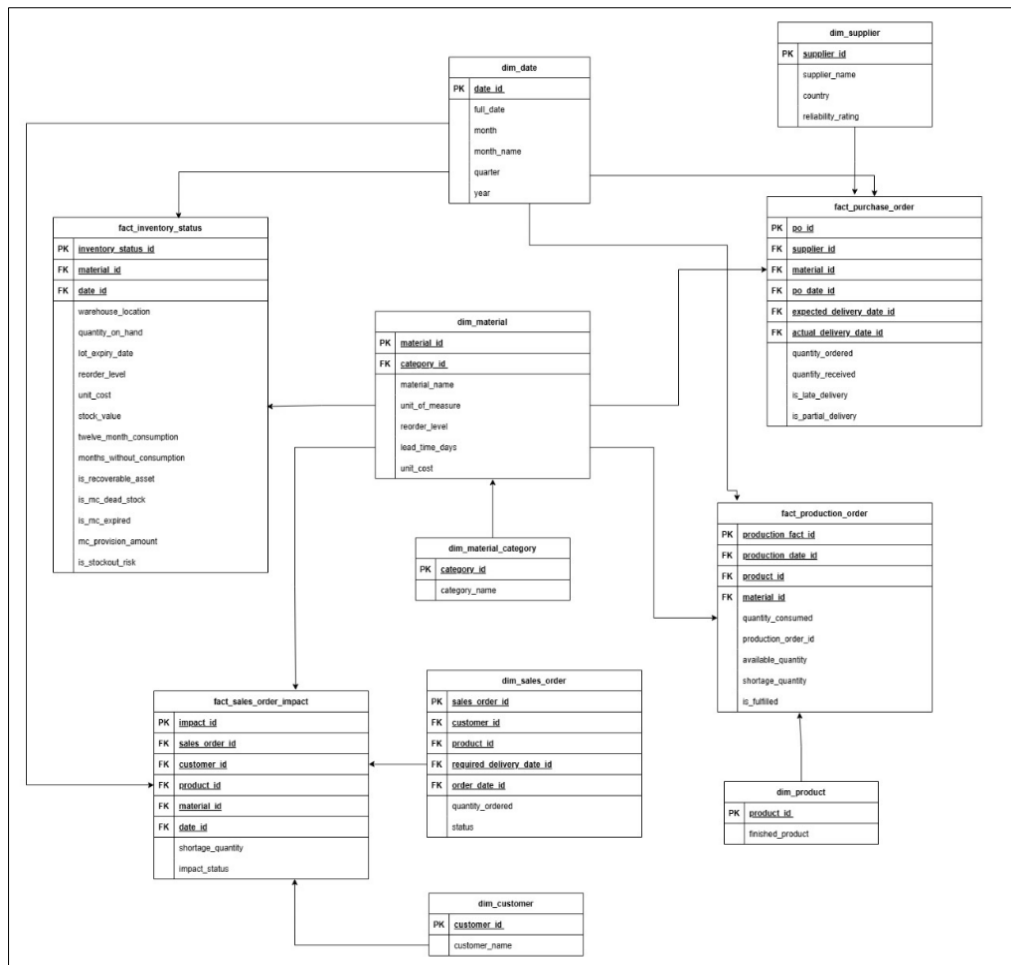
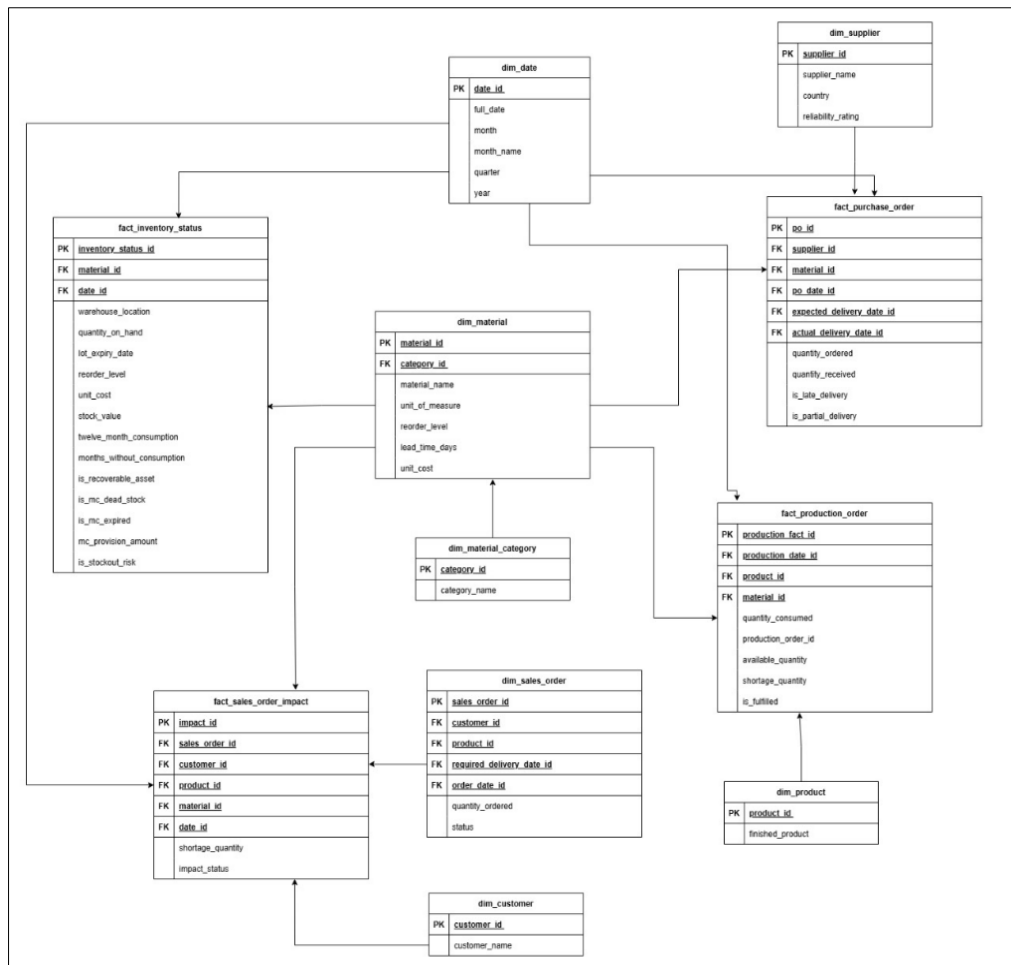
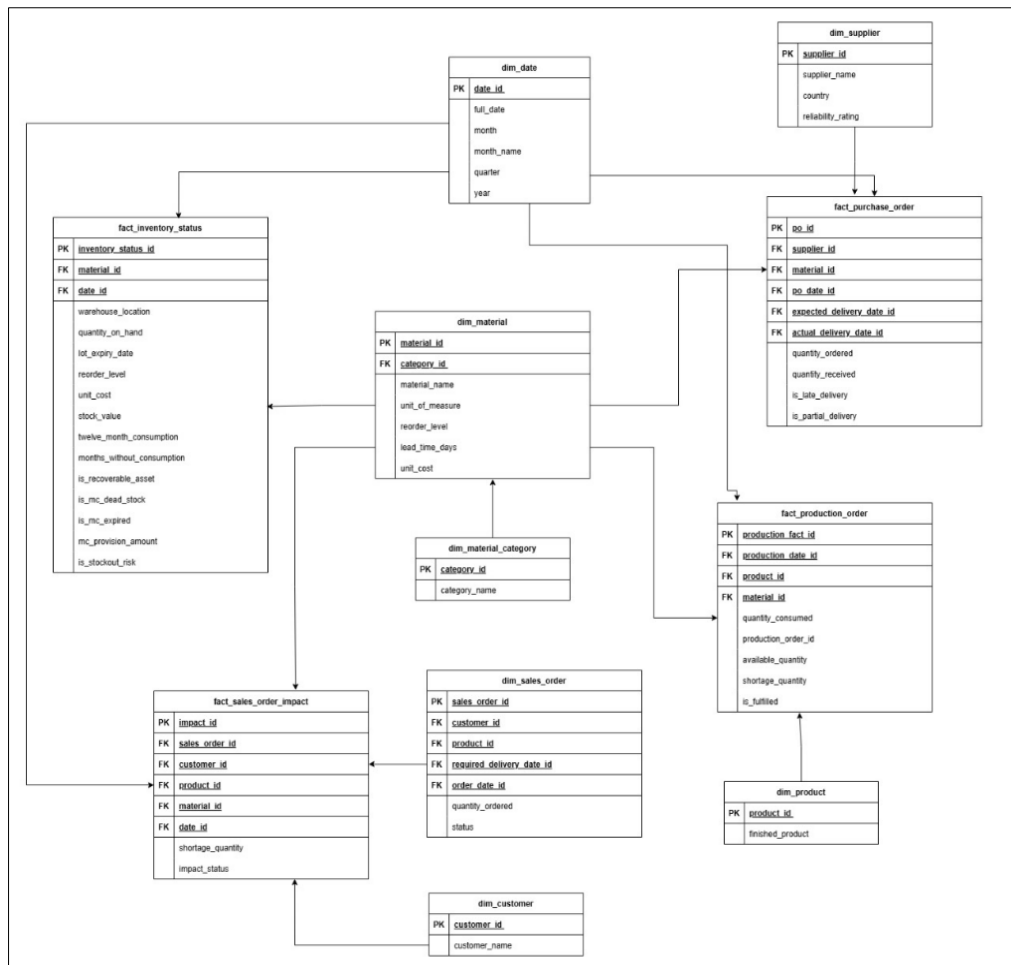
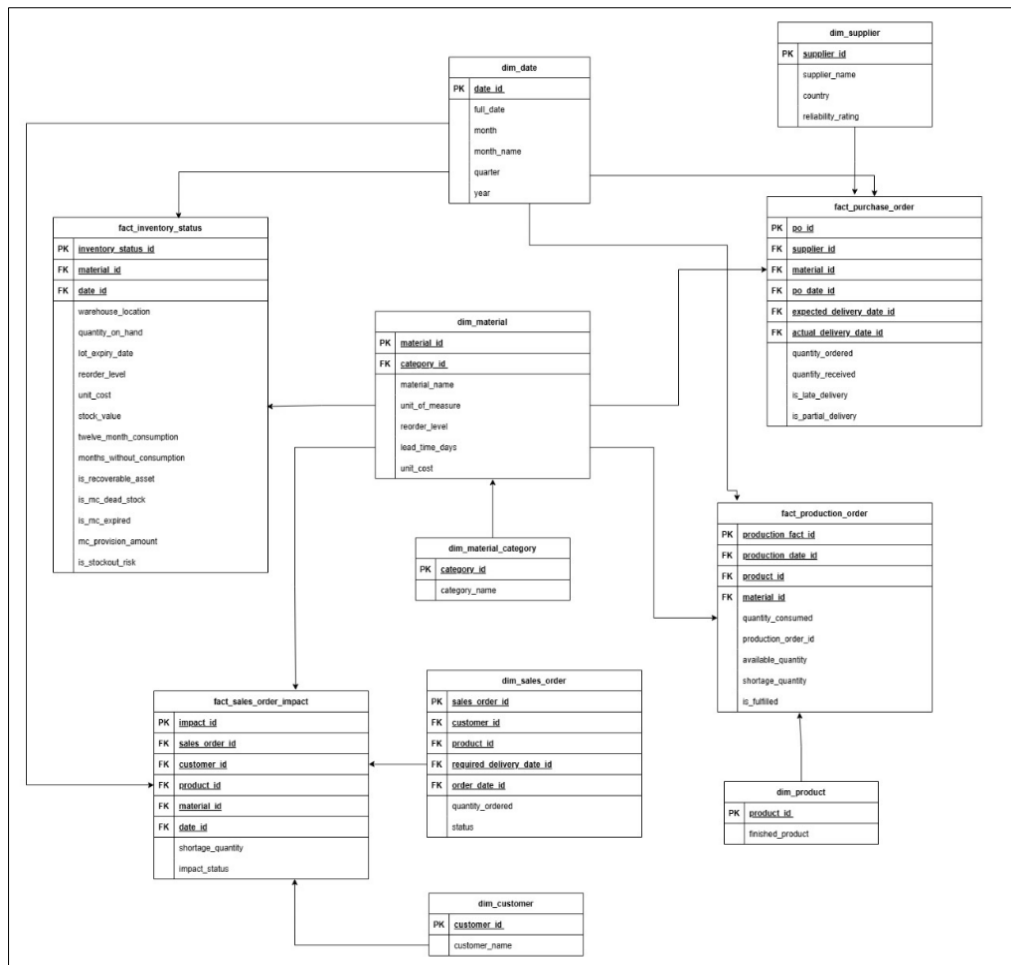
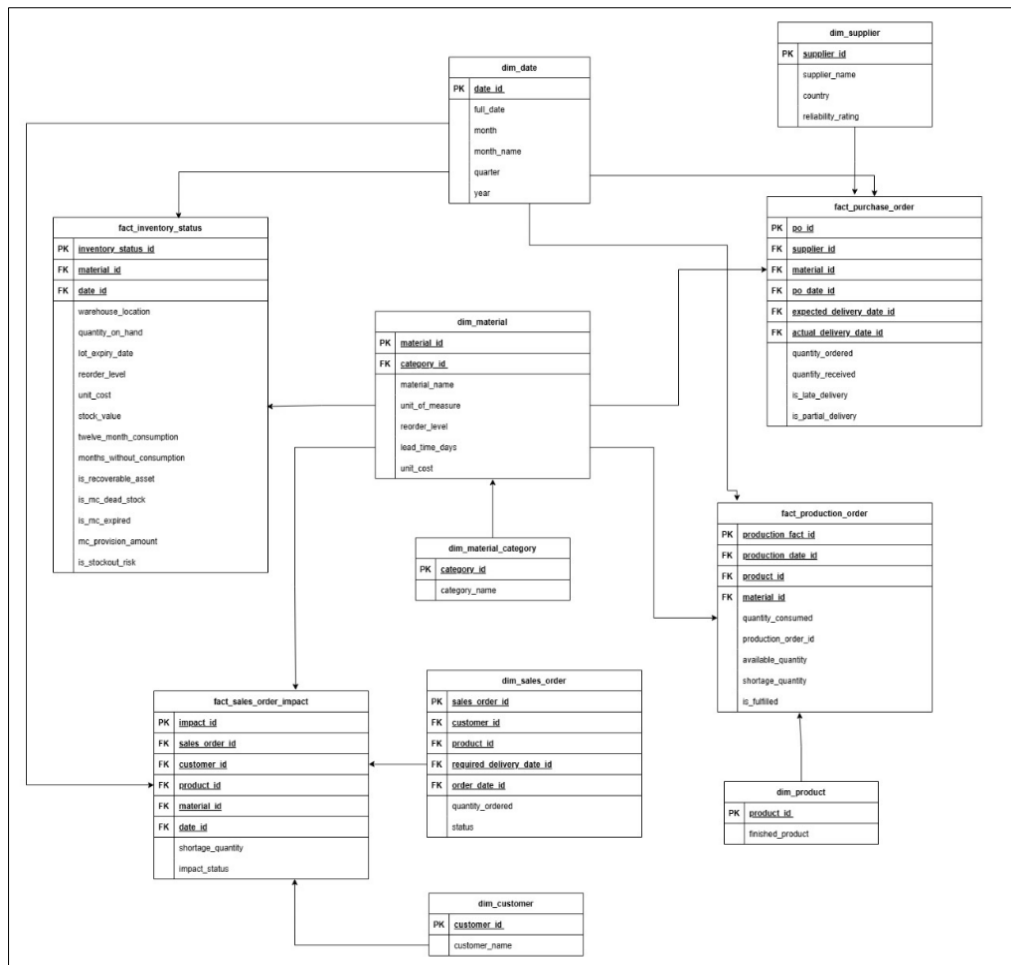
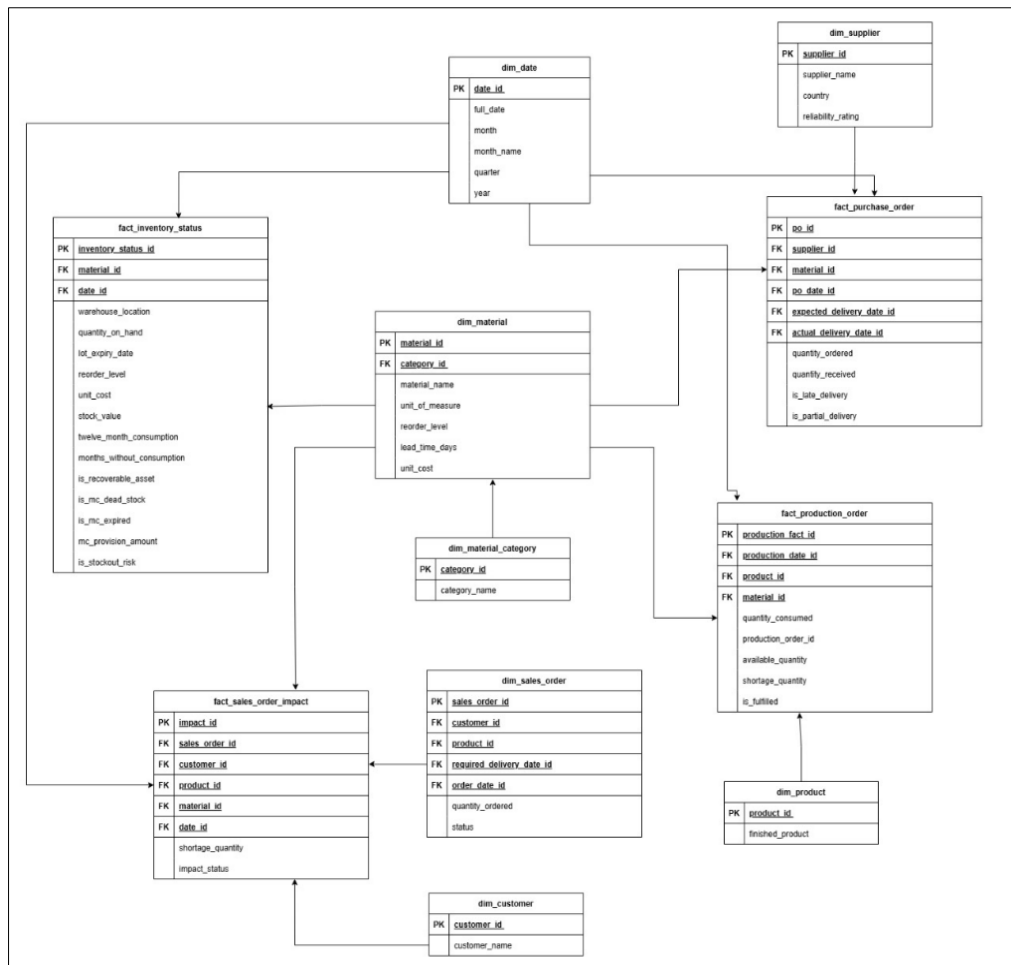
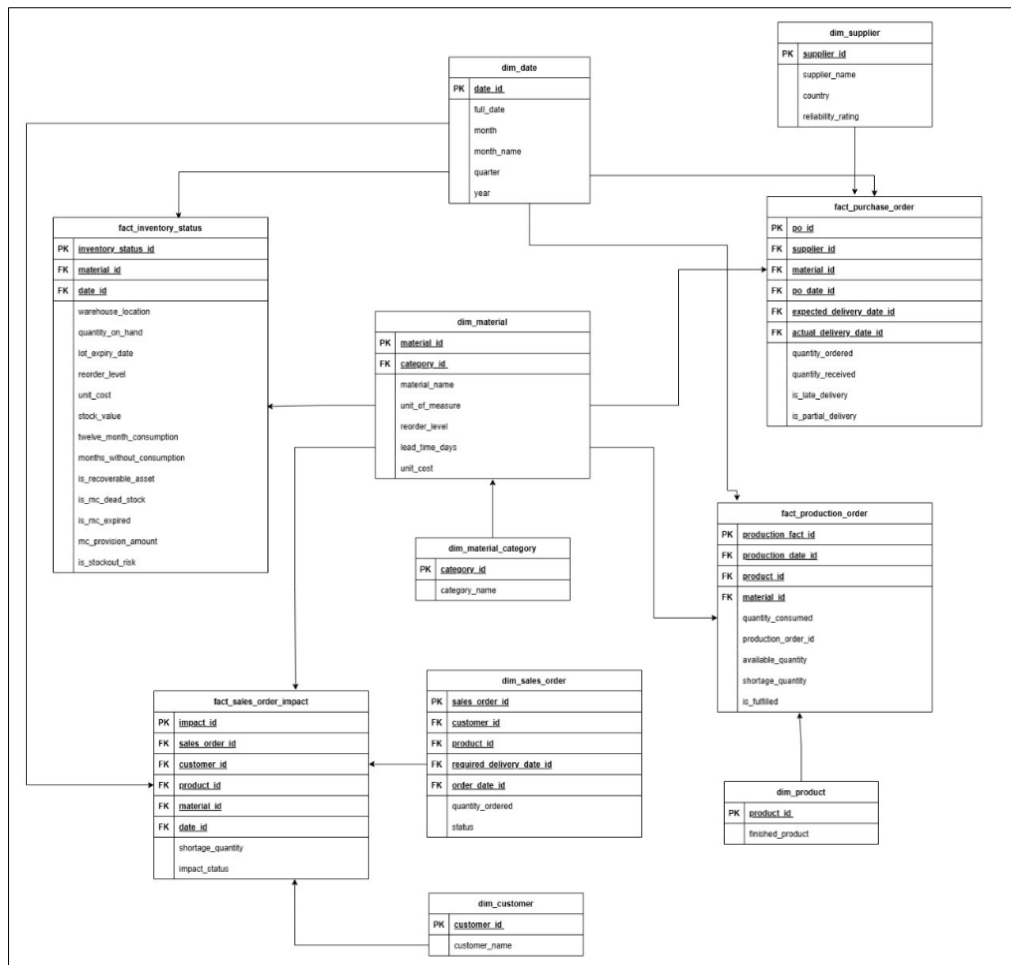
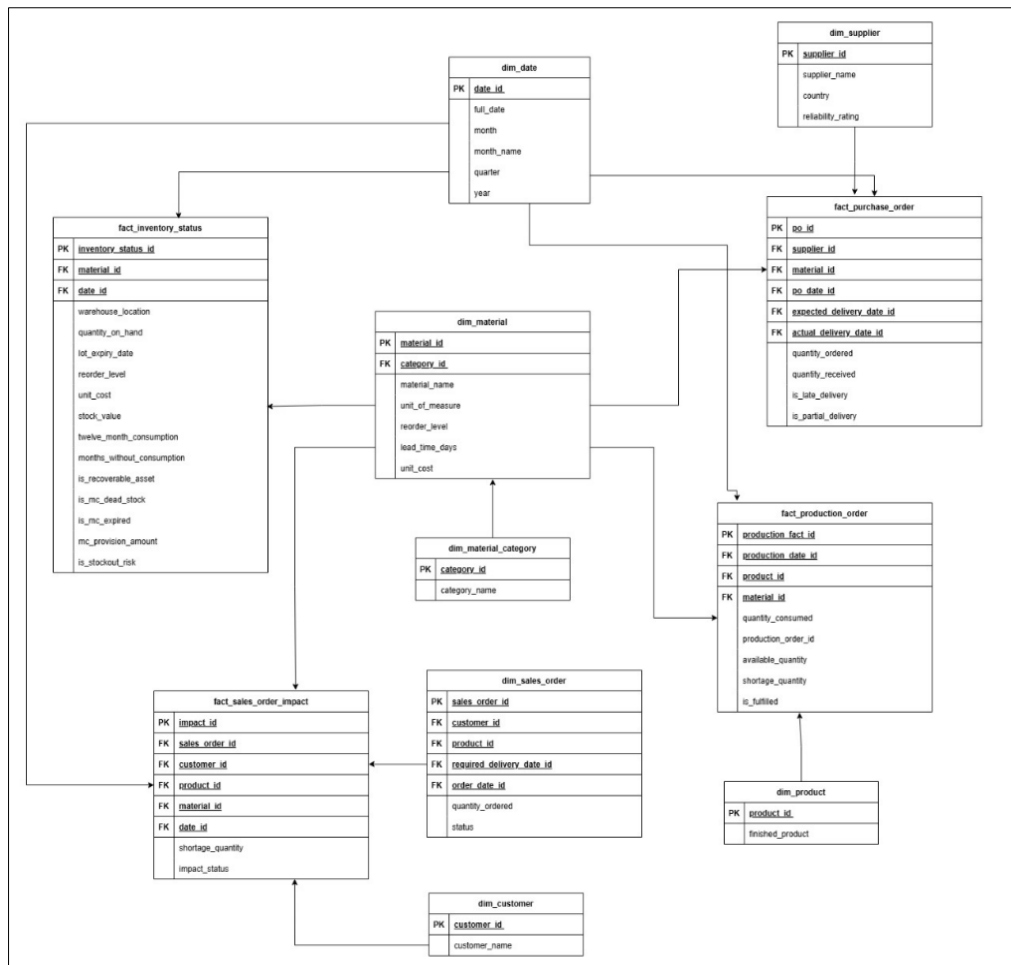
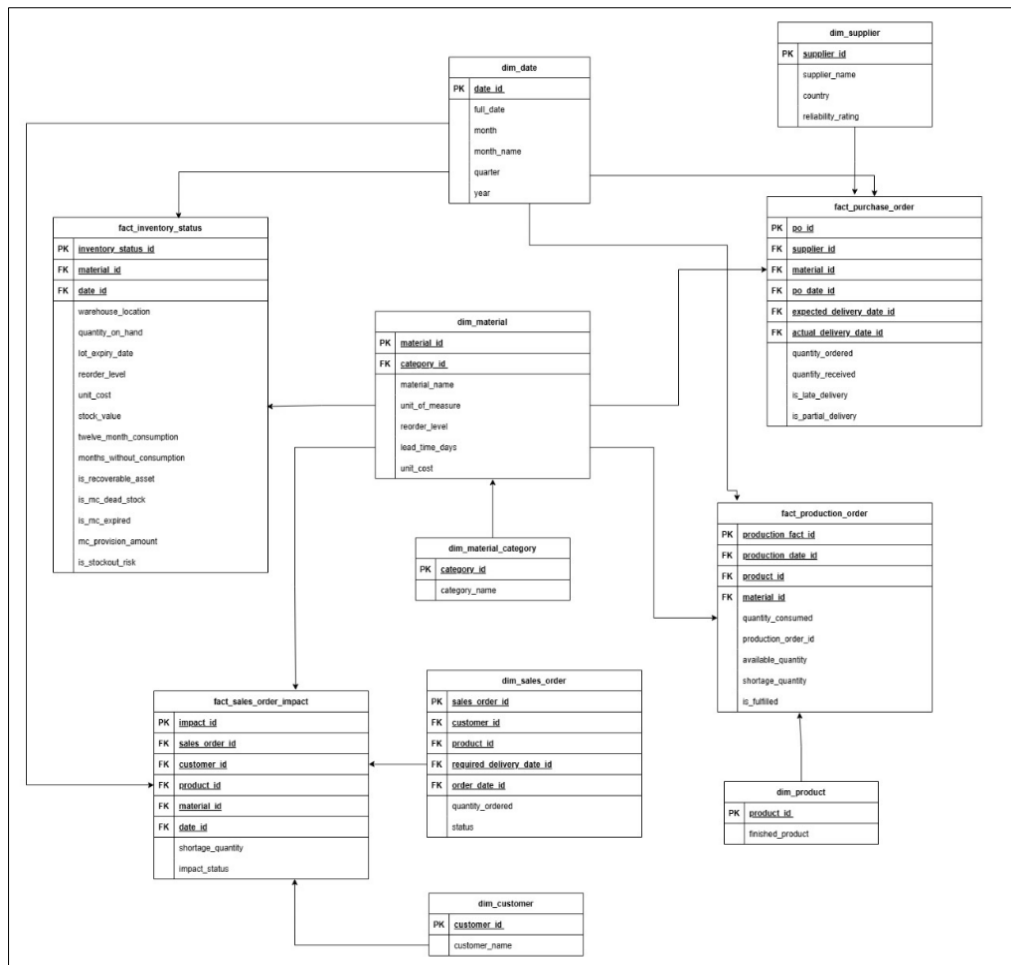
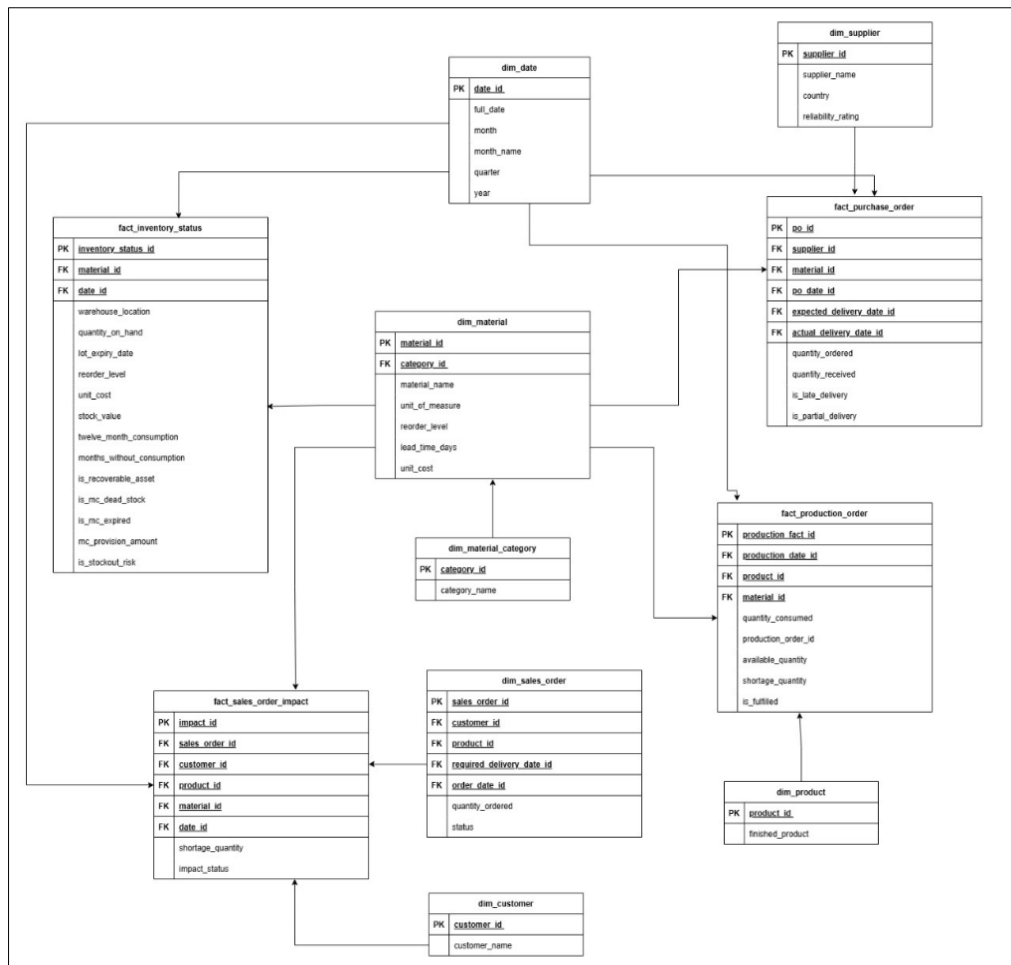
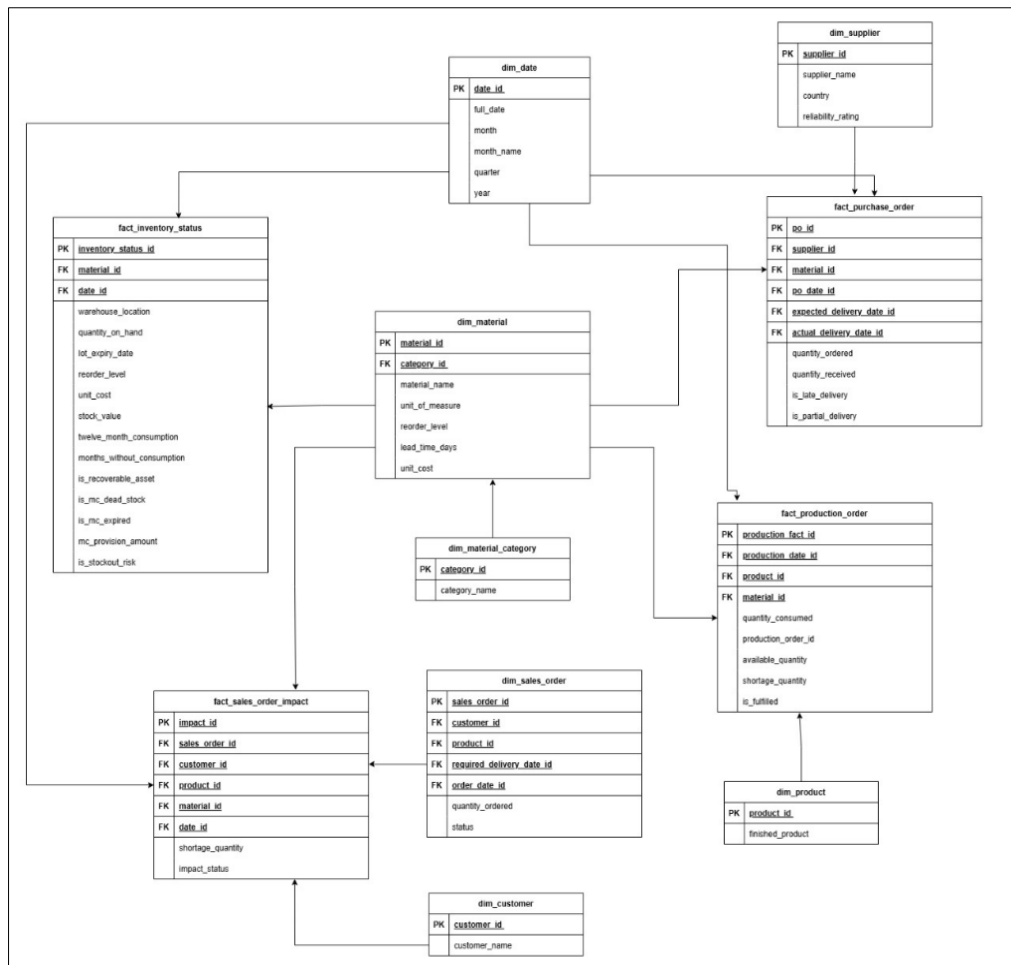


Figure 4.5 Galaxy Schema Design for Recoverable Assets and Inventory Risk Management Data Warehouse

The golden layer of the data warehouse is the Galaxy Star model (fact constellation model), which is used to support multiple business processes of inventory risk management, production planning, supplier performance monitoring and customer order fulfilment as illustrated in Figure 4.5. The galaxy model has multiple fact tables, instead of one central fact table, and all of the fact tables share the same dimension table. This helps to optimize the analysis and processing power of the system while minimizing data duplication and increasing the system's scalability. The model proposed consists of four fact tables: fact_inventory_status, fact_purchase_order, fact_production_order and fact_sales_order_impact. These fact tables have some dimensional tables in common, such as dim_material, dim_material_category, dim_supplier, dim_product, dim_customer, and dim_date. This model can meet all seven business questions that required by PPG.

(a) Fact Tables

1. *fact_inventory_status*

As shown in Table 4.5, this is the primary fact table of the warehouse. It stores inventory measurements and business rule calculations used to identify Recoverable Assets (RA), Magna Carta Dead Stock (MC Dead Stock), Magna Carta Expired materials, and stockout risks.

Table 4.5 *fact_inventory_status* fact table

Attribute	Description
inventory_status_id	Unique identifier for inventory record
material_id	Reference to material dimension
date_id	Reference to date dimension
warehouse_location	Inventory storage location
quantity_on_hand	Current stock quantity available
lot_expiry_date	Expiry date of inventory lot
reorder_level	Minimum stock level before replenishment
unit_cost	Cost per unit
stock_value	Total inventory value
twelve_month_consumption	Total consumption over the previous 12 months
months_without_consumption	Number of months without material usage
is_recoverable_asset	Recoverable Asset flag
is_mc_dead_stock	Magna Carta Dead Stock flag
is_mc_expired	Magna Carta Expired flag
mc_provision_amount	Financial provision amount
is_stockout_risk	Stockout risk indicator

2. *fact_purchase_order*

Refer to Table 4.6, this fact table records supplier deliveries and procurement activities. It is used to analyse supplier reliability, late deliveries, and partial deliveries.

Table 4.6 *fact_purchase_order* fact table

Attribute	Description
po_id	Unique identifier for inventory record
supplier_id	Reference to supplier dimension
material_id	Reference to material dimension
po_date_id	Purchase order date
expected_delivery_date_id	Expected delivery date
actual_delivery_date_id	Actual delivery date
quantity_ordered	Quantity ordered
quantity_received	Quantity received
is_late_delivery	Late delivery indicator
is_partial_delivery	Partial delivery indicator

3. *fact_production_order*

Refer to Table 4.7, this fact table stores production consumption records and supports shortage analysis.

Table 4.7 *fact_production_order* fact table

Attribute	Description
production fact id	Production fact identifier
production date id	Production date
product_id	Reference to product dimension
material_id	Reference to material dimension
quantity_consumed	Quantity consumed in production
production order id	Production order identifier
available_quantity	Available inventory quantity
shortage_quantity	Material shortage quantity
is_fulfilled	Production fulfilment status

4. *fact_sales_order_impact*

Refer to Table 4.8, this fact table records customer orders affected by inventory shortages and stockout risks.

Table 4.8 *fact_sales_order_impact* fact table

Attribute	Description
impact_id	Impact record identifier
sales_order_id	Sales order reference
customer_id	Customer reference
product_id	Product reference
material_id	Material reference
date_id	Date reference
shortage_quantity	Quantity shortage
impact_status	Impact status indicator

(b) Dimension Tables

1. *dim_material*

As shown in Table 4.9, this table stores descriptive information about raw materials used in manufacturing and inventory management.

Table 4.9 *dim_material* dimension table

Attribute	Description
impact_id	Impact record identifier
sales_order_id	Sales order reference
customer_id	Customer reference
product_id	Product reference
material_id	Material reference
date_id	Date reference
shortage_quantity	Quantity shortage
impact_status	Impact status indicator

2. *dim_material_category*

As shown in Table 4.10, this table stores material classification information.

Table 4.10 *dim_material_category* dimension table

Attribute	Description
category_id	Unique identifier for material category
category_name	Category name such as Pigment, Resin, Solvent, or Packaging

3. *dim_supplier*

As shown in Table 4.11, this table stores supplier reference information used for procurement analysis.

Table 4.11 *dim_supplier* dimension table

Attribute	Description
supplier_id	Unique identifier for supplier
supplier_name	Name of supplier company
country	Country where the supplier is located
reliability_rating	Supplier reliability score used for performance evaluation

4. *dim_product*

As shown in Table 4.12, this table stores information about finished paint products produced by PPG.

Table 4.12 *dim_product* dimension table

Attribute	Description
product_id	Unique identifier for finished product
finished_product	Name of the finished paint product

5. *dim_customer*

As shown in Table 4.13, this table stores customer information for sales order analysis.

Table 4.13 *dim_customer* dimension table

Attribute	Description
customer_id	Unique identifier for customer
customer_name	Name of customer or organisation

6. *dim_date*

As shown in Table 4.14, this table stores calendar information to support time-based analysis and reporting.

Table 4.14 *dim_date* dimension table

Attribute	Description
date_id	Unique identifier for date record
full_date	Complete calendar date
month	Numeric month value
month_name	Name of month
quarter	Quarter of the year
year	Calendar year

7. *dim_sales_order*

As shown in Table 4.15, this table stores customer order information and serves as a supporting dimension for sales impact analysis.

Table 4.15 *dim_sales_order* dimension table

Attribute	Description
sales_order_id	Unique identifier for sales order
customer_id	Reference to customer dimension
product_id	Reference to product dimension
required_delivery_date_id	Requested delivery date by customer

Attribute	Description
order_date_id	Date when order was placed
quantity_ordered	Quantity requested by customer
status	Current order status (e.g., Open, Closed, Pending)

The Galaxy Schema was selected because the project involves multiple analytical business processes. Inventory analysis, supplier performance analysis, production fulfilment analysis, and customer order impact analysis require different measurements while sharing common business dimensions. By using multiple fact tables connected to shared dimensions, the schema supports efficient reporting and avoids data duplication.

Furthermore, the schema enables the organisation to answer all seven business questions defined in the project scope, including identifying materials below reorder level, Recoverable Assets, Magna Carta classifications, provision amounts, production shortages, and customer sales order impacts. Therefore, the Galaxy Schema provides a scalable and flexible data warehouse design suitable for PPG's inventory risk management requirements.

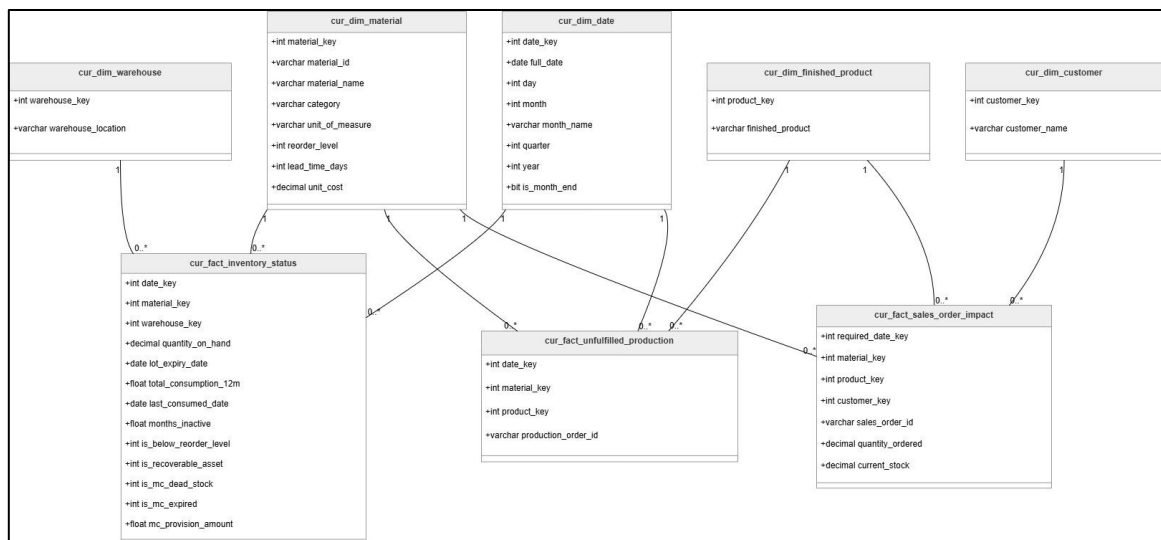


Figure 4.6 Galaxy Schema Implementation on Actual Pipeline

The finalized Galaxy Schema for the PPG Inventory Risk Management system is presented in figure 4.6. The overall Galaxy Schema design is similar to that shown in Figure 4.5, but some modifications were made to served the business questions differently. The end result is a schema of three fact tables (cur_fact_inventory_status, cur_fact_unfulfilled_production, cur_fact_sales_order_impact) and five shared dimension tables (cur_dim_material, cur_dim_date, cur_dim_warehouse, cur_dim_finished_product, cur_dim_customer). These tables were adjusted to the need of the serving layer that will answer all the given business questions that requires bridging some tables through composite keys.

The fact_purchase_order, dim_supplier, and dim_material_category tables were not included in the final schema because supplier performance analysis was not part of the final project and the source data for the analysis was not available. Several descriptive attributes were also added directly to the dimension tables to make the schema more simple, and to enhance analytical performance. All the 7 business questions outlined by PPG have been covered in the finalized Galaxy Schema, which include inventory health monitoring, recoverable asset identification, dead stock analysis, expired material tracking, production order fulfilment analysis and customer sales order impact assessment.

4.5.5 Interface Design

The user interface of this project is developed using Microsoft Power BI to visualise the refined data generated by the Gold Layer in the data pipeline. The dashboard is divided into two separate pages, each of which is customised to solve specific business problems related to inventory outage and operational risks.

4.5.5.1 Inventory Health & Financial Risk

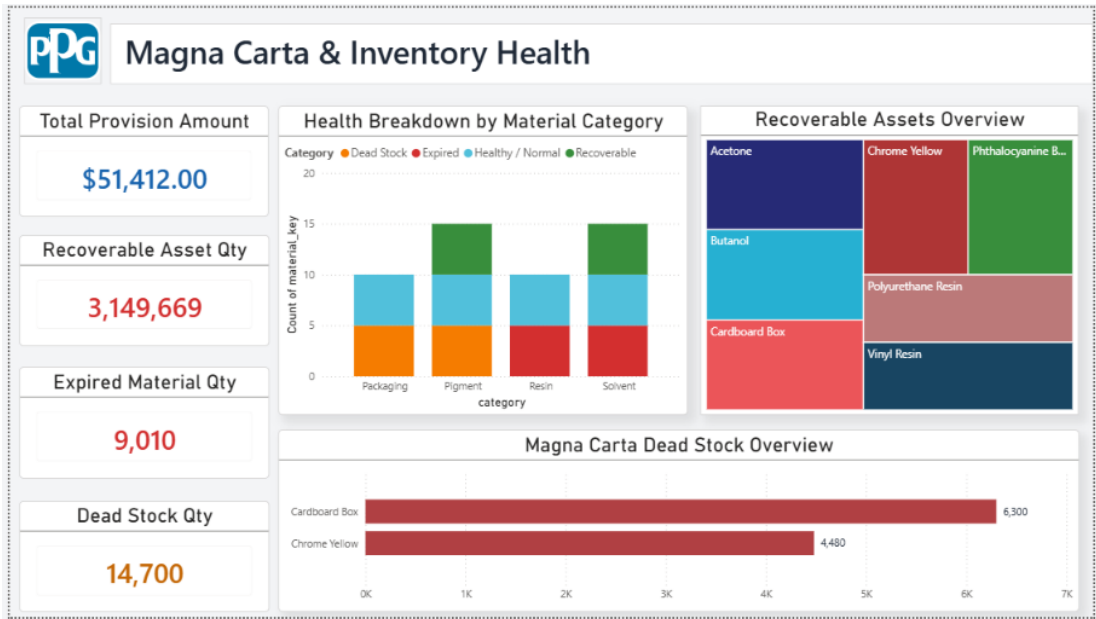


Figure 4.7 Dashboard for PPG Magna Carta and Inventory Health

Figure 4.7 above shows the financial and quantitative evaluation dashboard that focusses on the health of the inventory. The four key performance indicators (KPI) cards on the left show important main indicator, the total inventory is \$51,412.00, the number of recoverable assets is 3,149,669, the number of expired materials is 9,010, and the dead stock inventory is 14,700. Next to these indicators, stacked bar charts provide “Health Status Subdivision by Material Category”, visually showing the number of key materials such as packaging, pigments, resins and solvents, and further dividing each category into dead stock, expired, healthy/normal and recoverable status. In addition, the tree map titled “Overview of Recoverable Assets” shows the distribution of the proportion of recoverable assets, highlighting items such as ethyl acetate, chromium, butanol, polyurethane resin, carton and vinyl resin. Finally, the horizontal bar chart at the bottom shows the “Overview of Magna Carta Dead Stock”, which lists the specific number of dead stock, especially 6,300 cardboard boxes and 4,480 chrome yellow.

4.5.5.2 Fulfilment Risk and Customer Impact

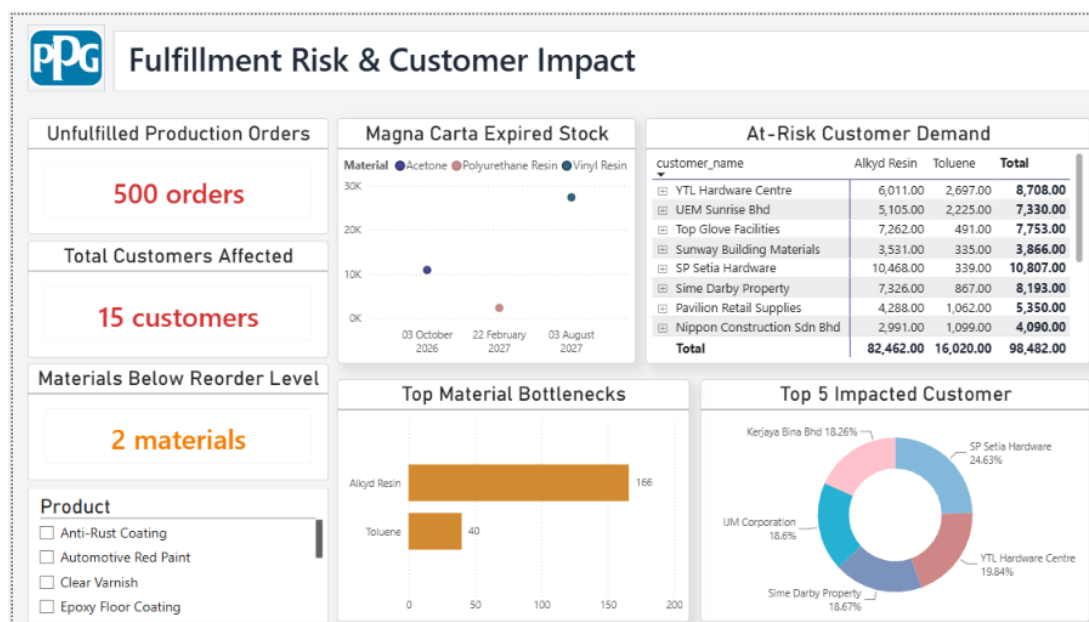


Figure 4.8 Dashboard for PPG Fulfilment Risk and Customer Impact

Figure 4.8 above shows the dashboard that focusses on operational risks, material bottlenecks and their impact on customers. The left column highlights key operational issues through three key performance indicators (KPI) cards, showing 500 unfinished production orders, 15 affected customers, and 2 materials below the order level. The main canvas shows a scatter plot to draw the number of “Magna Carta Expired Stock” of materials from the end of 2026 to mid-2027, including acetone, polyurethane resin and vinyl resin. The following is a horizontal bar chart to identify the “Top Material Bottleneck” showing that alkyd resin blocked 166 orders, while toluene blocked 40 orders. The dashboard also explains the specific losses of individual customers through the “At-risk Customer Demand” matrix, and accurately analyses which customers such as YTL Hardware Centre and SP Setia Hardware face demand risks due to waiting for alkyd resin and toluene, a total of 98,482 units. Finally, the pie chart shows the distribution of the “Top 5 Impacted Customer” indicating that SP Setia Hardware has the greatest impact, accounting for 24.63%, followed by YTL Hardware Centre, Sime Darby Property, IJM Company and Kerjaya Bina Bhd and other major customers.

4.6 Summary

Chapter 4 explained about the analysis and design of the new system for inventory and risk management of PPG Malaysia to solve current problem faced by the company in the operation such as scattered files, slow manual check and repetitive operation on spreadsheet. The team conducted six original CSV files studies and facilitated workshops with stakeholders to derive seven key business questions, eleven functional data-moving requirements and eight quality standards. The new system is capable of tackling these problems by leveraging a multi-stage data architecture that is entirely powered by Microsoft Azure, providing centralized storage, automated data movement, quick processing and reporting through Power BI. The database also follows the galaxy schema that allows for a minimum of data duplication and rapid reporting based on four main activity tables and seven common description tables. Finally, this design can be used to populate a two-page Power BI dashboard with inventory health, financial risks and impacts of stockouts, which will provide a step-by-step inventory design plan for the building phase in Chapter 5.

CHAPTER 5

IMPLEMENTATION AND TESTING

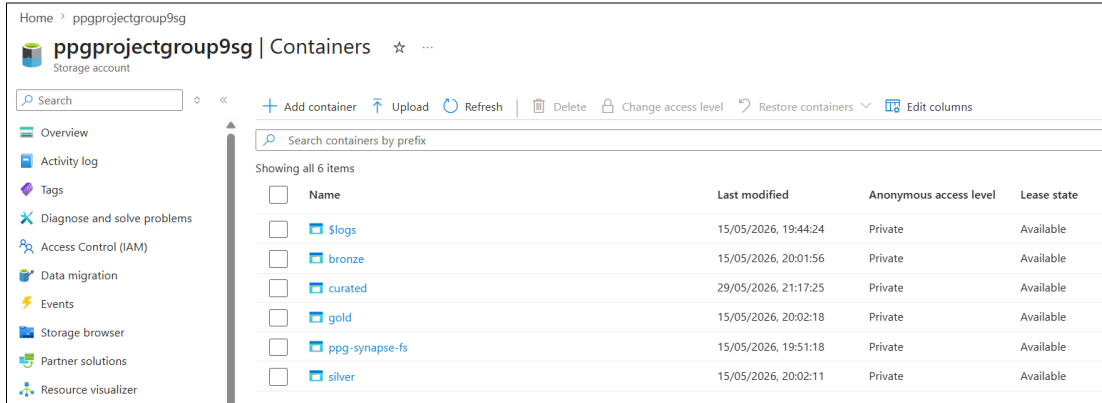
5.1 Introduction

This chapter presents the design and development of the inventory analysis system designed for PPG. Microsoft Azure cloud services, such as Azure Data Factory, Azure Data Lake Storage Gen2, Azure Databricks, Azure Synapse Analytics and Power BI, are used for implementing the system. The implementation process includes data import, data conversion, data storage, analytical database development and dashboard visualisation. This chapter also presents the functions of the system, the user interface and the tests performed to validate the system running in accordance to the requirements.

5.2 Pipeline Development

The PPG Industries' inventory risk management data platform is built on an enterprise cloud architecture and is deployed on Microsoft Azure and Power BI. The Medallion framework (bronze, silver, gold and curated layer) ensures that identified and decoupled computing resources can manage the data ingestion, cleaning, transformation and serving without support from others. This method guarantees a highly high-performance data lake system, which can be expanded with the increase of PPG's operational data. This section outlines the development of technology at each step along the data pipeline.

5.2.1 Data Storage and Transformation Process



Name	Last modified	Anonymous access level	Lease state
\$logs	15/05/2026, 19:44:24	Private	Available
bronze	15/05/2026, 20:01:56	Private	Available
curated	29/05/2026, 21:17:25	Private	Available
gold	15/05/2026, 20:02:18	Private	Available
ppg-synapse-fs	15/05/2026, 19:51:18	Private	Available
silver	15/05/2026, 20:02:11	Private	Available

Figure 5.1 Azure Data Lake Storage Gen2 Containers

Azure Data Lake Storage Gen2 as the main storage platform for inventory analysis system. The decision was made to break out the data into various containers to facilitate the organisation of the data according to the Medallion architecture: the bronze layer, the silver layer, the gold layer and the curated layer as illustrated in Figure 5.1.

The raw data extracted from the original CSV file is in the bronze layer. The cleaned and standardised data is created and saved in the silver stage in the first phase of data conversion. The gold layer is for storing business ready data which can be directly used for business analysis. Datasets that have been processed and can be used for Azure Synapse Analytics and Power BI are stored in the curated layer.

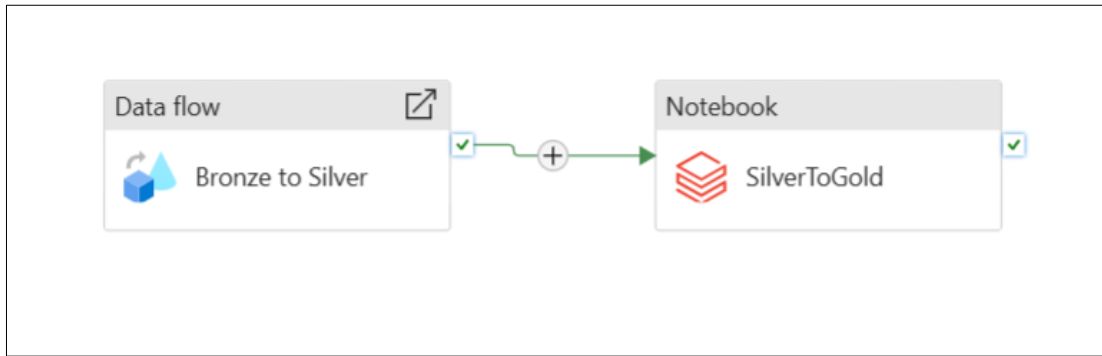


Figure 5.2 Bronze-to-Silver and Silver-to-Gold Transformation Pipeline

Azure Data Factory and Azure Databricks were used to implement the data transformation process. As shown in Figure 5.2, the conversion pipeline mainly includes two stages. During the first stage, the Bronze to Silver data flow activity cleans, verifies, and standardises the raw data of the Bronze layer, ensuring the consistency of the data and ensuring data quality for data analysis applications.

In the second stage, the SilverToGold Databricks notebook converts the cleaned Silver layer data into a complete dataset that can be directly used for business. The final Gold layer dataset is calculated, aggregated and analytically converted for inventory monitoring, inventory analysis and business intelligence reporting. This multi-layer storage and conversion can significantly enhance the data organisation, data quality, scalability and system maintainability in the whole analysis pipeline. Azure Data Lake Storage Gen2 as the main storage platform for inventory analysis system.

5.2.2 Analytical Database Development using Azure Synapse Analytics

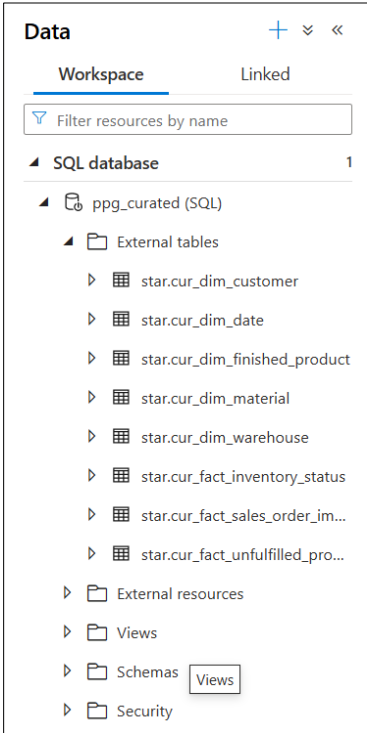


Figure 5.3 Azure Synapse Analytics External Tables in the *ppg_curated* Database

Azure Synapse Analytics was used to develop the analytical database for the Inventory Analytics System. Azure Synapse was then used to access the data in the Curated layer of Azure Data Lake Storage Gen2, using external tables, which allowed the data to be queried without replicating storage. A database called *ppg_curated* was set up to control the analytical datasets as seen in Figure 5.3.

Azure Data Lake Storage Gen2		Name
▲	ppgproject-group-9-synapse (Prim...	dim_customer
	ppg-synapse-fs (Primary)	dim_date
	bronze	dim_finished_product
	curated	dim_material
	gold	dim_warehouse
	silver	fact_inventory_status
▶	(Attached Containers)	fact_sales_order_impact
		fact_unfulfilled_production

Figure 5.4 Implemented Dimension and Fact Tables in Azure Synapse Analytics

The database was designed with several dimension and fact tables that enabled the analysis and reporting of inventory. The tables implemented are shown in Figure 5.4 and contain dimension tables (dim_customer, dim_date, dim_finished_product, dim_material and dim_warehouse) and fact tables (fact_inventory_status, fact_sales_order_impact, and fact_unfulfilled_production).

The dimension tables hold descriptive data that contextualize and give meaning to business analysis, and the fact tables hold transactional and operational data that can be used to measure inventory performance. This architecture allows efficient analytical queries and facilitates business intelligence reporting and dashboards.

The database schema designed and implemented was targeted to support some critical business analysis such as inventory status monitoring, stockout risk assessment, sales order impact evaluation, production order fulfilment analysis and so on. The system uses Azure Synapse Analytics for scalable and high-performance data processing for inventory analytics.

The analytical database in Azure Synapse Analytics is the physical representation of the finalized Galaxy Schema from Chapter 4. The dimension and fact tables that are necessary to support the identified business requirements and Power BI dashboards were materialized as external tables in the ppg_curated database. This approach of implementation makes the database less redundant, enhances the

performance of queries and keeps the analytical database in sync with the project's business intelligence goals.

5.3 Coding of System Main Functions

The implementation phase of the PPG Industries Inventory Risk Management platform involves translating the conceptual data architecture into production-grade programmatic code and orchestration components across the Azure cloud ecosystem. The development follows the Medallion framework, transitioning data through a series of progressive refinement states: Ingestion (Bronze), Cleansing and Schema Conformity (Silver), Business Logic Calculations (Gold), Analytics Serving (Curated), and Visual Reporting (Semantic Power BI Engine). This section documents the actual code structures, query scripts, and orchestration logic developed to address the seven core business requirements.

5.3.1 Phase 1: Bulk Ingestion and Landing (Source to Bronze)

The initial phase of the data pipeline establishes a robust, production-grade baseline dataset within the bronze landing tier of the Azure Data Lake Storage Gen2 (ADLS Gen2) account. During initial consultations, feedback from the PPG Industries Specialist indicated that the baseline transactional records were insufficient in scale and volume to simulate realistic enterprise-level analytics or stress-test downstream data warehouse configurations.

To resolve this limitation and ensure the pipeline could be thoroughly tested, the data engineering team implemented a two-part ingestion strategy: Local Programmatic Dataset Expansion followed by Scripted Azure PowerShell Deployment.

5.3.1.1 Programmatic Data Enrichment and Anomaly Seeding

Before cloud transmission, a specialized Python script (`generate_expanded_dataset.py`) was executed locally to programmatically extend the scope of the six core transactional datasets (`materials.csv`, `suppliers.csv`, `inventory_snapshot.csv`, `production_orders.csv`, `purchase_orders.csv`, and `sales_orders.csv`). The script anchored the system's chronological timeline to a reference evaluation date of December 31, 2027.

Crucially, the script mathematically seeded realistic business anomalies and missing data fields into the tables to ensure the downstream Azure Databricks notebook could be validated against real-world failures. The programmatic enhancements included:

- (c) Lot Expiry Vectoring: Automatically computed and injected missing temporal boundaries (`lot_expiry_date`) across inventory logs based on specific raw material chemical shelf-lives.
- (d) Dead Stock Seeding (Q3/Q5 Validation): Isolated specific raw materials (such as MAT005 - Chrome Yellow and MAT010 - Vinyl Resin) and blocked all production floor consumption records for them after April 1, 2026. This explicitly forces them into a stagnant state of 9+ months of inactivity.
- (e) Overstock/Slow-Mover Injection (Q2 Validation): Programmatically scaled purchase order receipts for MAT004 (Phthalocyanine Blue) and MAT015 to deliver 5x the annual historical demand, trapping excess capital in the warehouse.
- (f) Fulfillment Failure Simulation (Q6 Validation): Altered roughly 2% of active manufacturing runs, zeroing out the `quantity_consumed` metric to explicitly simulate production orders that failed due to inventory stockouts.
- (g) Backlog Generation (Q7 Validation): Structured the sales transaction ledger to leave approximately 25% of late-2027 client orders in an 'Open' fulfillment status, creating a test backlog to link to low-stock raw components.

To verify that the programmatic generation successfully executed and seeded the parameters requested by the PPG specialist, the python script was run in a local terminal environment. The resulting execution log is captured in Figure 5.5.

```
Wrote expanded dataset to: c:\Users\Owner\Dev\python\test_ppg\dataset_expanded

Dead-stock materials (no consumption after 2026-04-01): ['MAT005', 'MAT010']
Slow-mover materials (5.0x overstocked) : ['MAT004', 'MAT015']
Discontinued products (BOM contained dead material) : []
Failed production orders (qty_consumed=0) : 499
Expired lots in stock @ 2027-12-31 : ['MAT009', 'MAT010', 'MAT014']

materials : 20
suppliers : 10
purchase_orders : 402 (+362)
sales_orders : 14328 (+14280)
production_orders : 100176 (+99840)
inventory_snapshot : 720 (+480)
```

Figure 5.5 Python Execution Output for Dataset Expansion and Target Business Anomaly Seeding.

As demonstrated by the terminal logs in Figure 5.5, the script successfully generated the expanded datasets within the target directory. The output validates that MAT005 and MAT010 were restricted from production tracks after the deprecation date to simulate Dead-stock anomalies, while a baseline of failed production runs and expired lots were successfully planted to benchmark downstream pipeline testing.

5.3.1.2 Local PowerShell Bulk Ingestion Engine

Once the local Python script successfully executed and generated the expanded, anomaly-seeded datasets, a specialized Windows PowerShell automation script was deployed to orchestrate the programmatic ingestion of the files into the cloud. Manually uploading files through the graphical Azure Portal web interface introduces risks of network timeouts, lacks automated validation, and is inefficient for enterprise data workflows. To establish a more resilient, production-grade ingestion channel, the team utilized native Azure command-line automation.

The script logs directly into the Azure subscription, binds to the project resource group, and targets the specific storage account (ppgprojectgroup9sg). The

execution and terminal feedback from this process are captured in Figure 5.6, serving as empirical proof of a successful cloud transfer.

```
$StorageAccount = "ppgprojectgroup9sg" # your ADLS Gen2 account name
$Container       = "bronze"           # filesystem (container) name
$SourceDir       = "C:\Users\Owner\Dev\python\test_ppg\dataset_expanded"

$Files = @(
    "materials.csv",
    "suppliers.csv",
    "purchase_orders.csv",
    "sales_orders.csv",
    "production_orders.csv",
    "inventory_snapshot.csv"
)

foreach ($f in $Files) {
    $table = [IO.Path]::GetFileNameWithoutExtension($f)
    $localPath = Join-Path $SourceDir $f
    $blobPath = "$table/$f"

    Write-Host "Uploading $f -> $Container/$blobPath"

    az storage blob upload `
        --account-name $StorageAccount `
        --container-name $Container `
        --name $blobPath `
        --file $localPath `
        --auth-mode login `
        --overwrite true `
        --only-show-errors
}

Write-Host "`nDone. Bronze landing path: $Container/"
```

Figure 5.6 Windows PowerShell Terminal Verification of Bulk Blobs Data Ingestion into the Bronze Storage Container

To verify the successful completion of the edge-to-cloud data ingestion phase and guarantee data persistence, the cloud storage architecture was audited using the Azure Portal Storage Browser interface. The resulting cloud state verification is displayed in Figure 5.7.

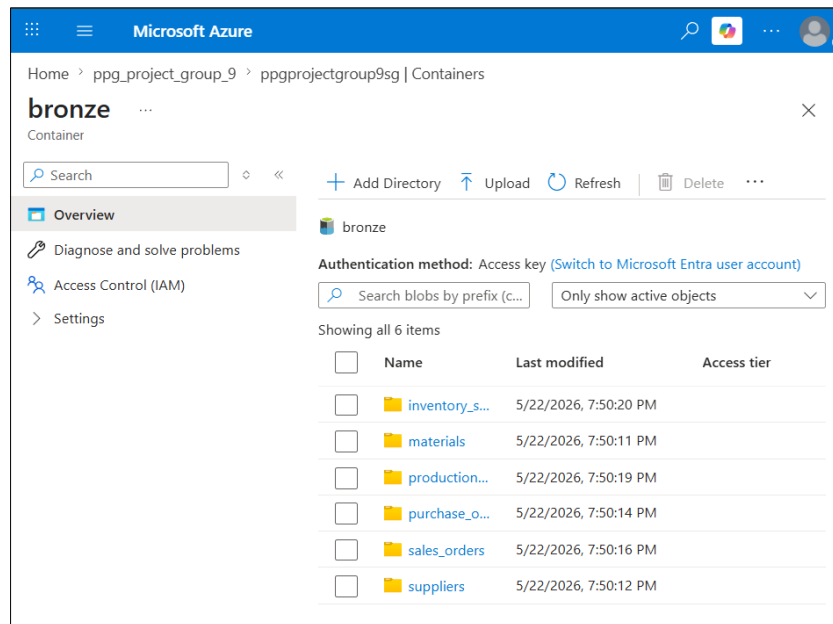


Figure 5.7 Azure Portal Storage Browser Interface Confirming Successful Data Landing in the Bronze Container

With all six expanded files successfully staged in the bronze container, the data platform has a secure, validated data source ready to feed the Data Factory mapping workflows in the Silver layer.

5.3.2 Phase 2: Structural Integrity and Parquet Conversion (Bronze to Silver)

Once the raw, expanded CSV files are securely landed within the data lake's bronze tier, they remain in their native, unvalidated states. Raw CSV files lack strict schema definitions, store all data types as plain text strings, and consume unnecessary storage space due to their uncompressed nature. To transition these operational assets into high-performance analytical datasets, the data engineering team built a centralized orchestration pipeline within Azure Data Factory (ADF) to govern the Bronze-to-Silver refinement process.

The primary objective of the Silver tier is to enforce schema validation, clean up structural anomalies, standardize data types, and convert raw rows into optimized columnar formats.

5.3.2.1 Data Factory Pipeline Orchestration & Transformation Logic

The transformation process is driven by an Azure Data Factory master pipeline that processes the files through isolated execution tracks. Figure 5.8 display the functional layout of the pipeline design and execution canvas inside the ADF workspace.



Figure 5.8 Azure Data Factory Pipeline Workspace Configuring the Bronze-to-Silver ETL Processing Workflows

To guarantee structural integrity before loading datasets into downstream compute environments, the ADF pipelines execute three major data processing steps:

- (h) **Strict Schema Mapping and Type Casting:** The engine overrides generic string definitions from the raw text files. It explicitly maps numeric keys to integers, converts financial values (unit_cost) and volume fields (quantity_on_hand,

quantity_consumed) to double-precision floats, and reformats date strings into a standardized ISO YYYY-MM-DD date index format.

- (i) **Text Cleansing and Trimming:** The pipeline sanitizes character strings across the text fields such as material_name, warehouse_location, and customer_name by stripping hidden trailing whitespaces, removing broken characters, and normalizing casing across row batches.
- (j) **Columnar Parquet Conversion:** Rather than saving the outputs back as raw text, the ADF sink parameters translate the tabular structures into binary Apache Parquet files. Parquet stores data in an efficient columnar format, embedding rich metadata schemas directly inside the file headers and optimizing data compression.

5.3.2.2 Storage Tier Verification (Silver Container Population)

Upon successful execution of the ADF data pipelines, the processed records are pushed as optimized directories into the secure silver container. To ensure that data persistence rules were maintained and file divisions occurred cleanly, a post-execution check was conducted using the Azure Portal Storage Browser, as shown in Figure 5.9..

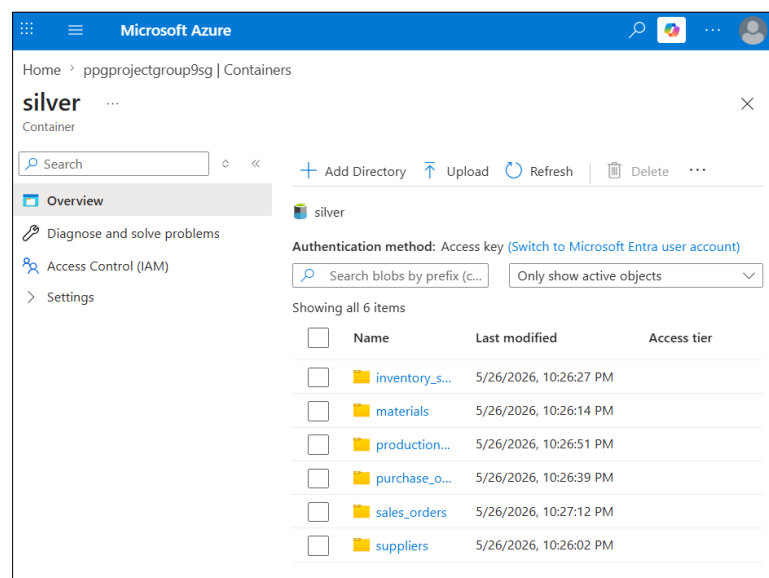


Figure 5.9 Azure Portal Storage Browser Verifying Clean Structure and Population within the Silver Tier Container

As verified by the folder layout in Figure 5.9, the pipeline successfully isolated each independent data stream into clear, individual directories (such as `inventory_snapshot`, `materials`, `production_orders`, etc.). This organized structure allows downstream Databricks clusters to crawl and read specific tables efficiently without needing to scan the entire data lake. Converting the files into compressed Parquet sets within the Silver layer reduces total storage size while significantly cutting down the input/output bottleneck (I/O) for subsequent Spark transformations, providing a clean foundation for the complex business logic scoring in the Gold layer.

5.3.3 Phase 3: Algorithmic Scoring and Business Rules Enforcement (Silver to Gold)

Once the standardized, conformed data assets are persisted as Parquet tables in the Silver tier, they are ready for the application of complex, domain-specific corporate supply chain rules. The transition from Silver to Gold shifts the data from a clean structural representation to an enriched, query-optimized analytics tier.

This transformation is managed by orchestrating an Azure Databricks Jupyter Notebook Compute Node directly from an Azure Data Factory pipeline activity runner. The PySpark transformation engine handles intensive calculations, handles data-type mismatches, aggregates historical data, and flags inventory assets based on PPG Industries' risk categories.

5.3.3.1 PySpark Programmatic Transformation Logic

(k) Phase 1: Consumption & Inactivity Logic (Answering Q2 & Q3)

This phase performs complex temporal (time-based) aggregations across the manufacturing ledger to establish a historical baseline for material usage.

8. 12 Month Consumption Vendor

```
df_consumption_12m = df_production_orders \
    .filter((F.col("production_date") >= "2026-12-01") & (F.col(
        "production_date") <= evaluation_date)) \
    .groupBy("material_id") \
    .agg(F.sum("quantity_consumed").alias("total_consumption_12m"))
```

Figure 5.10 Python Code Snippet of 12 Month Consumption Vendor

As shown in figure 5.10, it isolates a strict 12-month trailing window relative to the evaluation date (2027-11-30). It filters manufacturing records back to 2026-12-01, groups by each material ID, and aggregates the sum of all consumed quantities. This calculation creates the core benchmark required to answer Q2 (Recoverable Assets). It allows the system to check if current stock balances exceed what the business actually uses in a year.

9. Stagnation Tracking Engine (df_last_consumed)

```
# Find the last date a material was consumed to evaluate Q3 (Dead Stock inactivity)
df_last_consumed = df_production_orders \
    .filter(F.col("production_date") <= evaluation_date) \
    .groupBy("material_id") \
    .agg(F.max("production_date").alias("last_consumed_date")) \
    .withColumn("months_inactive", F.months_between(F.lit(
        evaluation_date), F.col("last_consumed_date")))
```

Figure 5.11 Python Code Snippet of Stagnation Tracking Engine

As shown in 5.11, it searches the entire history up to the evaluation date to find the absolute most recent date a material was used on the production floor via `F.max("production_date")`. It then calculates the time difference (`F.months_between`) between that last transaction and the evaluation date. This determines the exact age of operational inactivity, directly mapping to the rules needed to identify Q3 (Magna Carta Dead Stock).

(I) Phase 2: Unfulfilled Production Orders (Answering Q6)

```
# Identifying production orders where quantity consumed is 0 or missing
due to stockouts
df_unfulfilled_production = df_production_orders \
    .filter((F.col("quantity_consumed") == 0) | (F.col
("quantity_consumed").isNull())) \
    .select("production_order_id", "production_date", "finished_product",
"material_id")
```

Figure 5.12 Python Code Snippet of Unfulfilled Production Orders

As shown in figure 5.12, this segment scans the manufacturing logs and uses a logical OR condition to extract rows where the quantity_consumed is either explicitly recorded as 0 or is missing entirely (Null). It directly isolates Q6 (Fulfillment Stockouts). In an enterprise system, a production run that is created but features zero component usage is a clear indicator of a stockout event that halted the assembly or mixing floor.

(m) Phase 3: Master Gold Inventory & Scoring (Answering Q1, Q2, Q4, & Q5)

This phase acts as the centralized business-rules engine, joining the reference domains together, hardening the schemas, and applying the classification rules.

10. Data Integration and Schema Hardening (df_gold_processing)

```

df_gold_processing = (
    df_current_snapshot
    .join(df_materials, "material_id", "left")
    .join(df_consumption_12m, "material_id", "left")
    .join(df_last_consumed, "material_id", "left")
    # \ FIX #1: cast strings to numerics so comparisons are numeric, not
    lexicographic
    .withColumn("quantity_on_hand", F.col("quantity_on_hand").cast(
        ("double")))
    .withColumn("reorder_level",      F.col("reorder_level").cast("int"))
    .withColumn("lead_time_days",      F.col("lead_time_days").cast("int"))
    .withColumn("unit_cost",           F.col("unit_cost").cast("double"))
    .na.fill(0, subset=["total_consumption_12m"])
    .withColumn("months_inactive",
        F.when(F.col("months_inactive").isNull(), F.lit(999.0))
        .otherwise(F.col("months_inactive")))
)

```

Figure 5.13 Python Code Snippet of Data Integration & Schema Hardening

As shown in figure 5.13, it filters the inventory down to the specific evaluation date snapshot, then uses Left Outer Joins to merge material metadata, the 12-month consumption totals, and the inactivity variables. Crucially, Fix #1 converts string text representations into strict numeric formats (double and int). This prevents critical calculation failures where computer strings are evaluated lexicographically (e.g., text "9" being evaluated as greater than text "100"). If a material had zero historical consumption, the left join results in a Null value; the `.na.fill(0)` function cleans this up by converting those gaps into true numerical zeros. If a material has never been consumed, it defaults its `months_inactive` to a high dummy value (999.0) to ensure it gets flagged as stagnant.

11. Algorithm Classification Engine

```

.withColumn(
    "is_below_reorder_level",
    F.when(F.col("quantity_on_hand") < F.col("reorder_level"), 1).
    otherwise(0)
)
.withColumn(
    "is_recoverable_asset",
    F.when(F.col("quantity_on_hand") > F.col(
        "total_consumption_12m"), 1).otherwise(0)
)
.withColumn(
    "is_mc_expired",
    F.when((F.col("lot_expiry_date").isNotNull()) &
        (F.col("lot_expiry_date") < F.lit(evaluation_date)), 1).
        otherwise(0)
)
.withColumn(
    "is_mc_dead_stock",
    F.when(
        (F.col("quantity_on_hand") > F.col("total_consumption_12m")) &
        (F.col("months_inactive") >= 9) &
        (F.col("is_mc_expired") == 0),
        1
    ).otherwise(0)
)
.withColumn(
    "mc_provision_amount",
    F.when(
        (F.col("is_mc_expired") == 1) | (F.col("is_mc_dead_stock") ==
        1),
        F.col("quantity_on_hand") * F.col("unit_cost")
    ).otherwise(0.0)
)

```

Figure 5.14 Python Code Snippet of Algorithm Classification Engine

- (n) Q1 Verification: If physical stock on hand is less than the cataloged reorder point, it flags a 1.
- (o) Q2 Verification: If stock on hand exceeds the trailing 12-month consumption history, it tags it as a Recoverable Asset.
- (p) Q4 Verification: If a material batch contains a lot expiration date stamp that has already passed the current evaluation calendar date, it marks it as Magna Carta Expired.

- (q) Q3 Verification: Implements a logical hierarchy rule. An item is only flagged as Dead Stock if it satisfies the structural overstock check, has been completely idle for 9 months or longer, and is not already flagged as expired. This step prevents counting the same financial write-down twice.
- (r) Q5 Verification: Calculates financial provision liabilities. If an item triggers either of the two Magna Carta risk conditions, its full volume value must be written down. The code multiplies the stock on hand by its unit cost (quantity_on_hand * unit_cost), providing an exact loss figure for financial audit reports.
- (s) Phase 4: Supply Chain Tracing (Answering Q7)

This final segment provides an end-to-end trace, linking low-level warehouse material shortages directly to affected high-level customer orders.

1. Stockout Risk Identification

```
df_at_risk_materials = df_inventory_analysis.filter(  
    (F.col("total_consumption_12m") > 0) &  
    ((F.col("quantity_on_hand") / (F.col("total_consumption_12m") / F.lit(  
        365.0)))  
    < F.col("lead_time_days"))  
).select("material_id", "material_name", "quantity_on_hand")
```

Figure 5.15 Python Code Snippet of Stockout Risk Identification

As shown in figure 5.15, this isolates materials that are on the verge of running out. It divides the 12-month usage total by 365 to calculate the Average Daily Consumption Rate. It then divides current stock by this daily rate to find the exact number of Days of Remaining Stock. If the remaining days are less than the material's supplier replenishment lead_time_days, it means the factory will hit a shortage before a new order can arrive.

2. Bill of Materials (BOM) Bridge

```
df_bom = df_production_orders.select("material_id", "finished_product").distinct()
```

Figure 5.16 Python Code Snippet of BOM Bridge

As shown in figure 5.16, because the raw sales ledger only lists the final paint products sold, the code scans historical manufacturing lines to map which raw component materials (`material_id`) are needed to mix each end-product (`finished_product`), building a dynamic Bill of Materials dictionary.

3. Downstream Customer Impact Trace (`df_sales_impact`)

```
df_sales_impact = (  
    df_at_risk_materials  
    .join(df_bom, "material_id", "inner")  
    .join(df_sales_orders, "finished_product", "inner")  
    .filter(F.lower(F.col("status")) == "open")  
    .select(  
        F.col("sales_order_id"),  
        F.col("customer_name"),  
        F.col("finished_product"),  
        F.col("material_id").alias("at_risk_material_id"),  
        F.col("quantity_on_hand").alias("current_stock"),  
        F.col("quantity_ordered"),  
        F.col("required_delivery_date")  
    )  
)
```

Figure 5.17 Python Code Snippet of Downstream Customer Impact Trace

Figure 5.17 shows the code that chains the vulnerable raw components to the BOM bridge, then performs an inner join against the active sales ledger. It applies a case-insensitive check (`F.lower`) to isolate orders that are still currently 'open'. It provides the answer to Q7 (Client Backlog Risks). It identifies exactly which open sales orders, delivery timelines, and industrial clients are threatened by current upstream warehouse raw-material shortages.

5.3.3.2 Cloud Storage Verification (Gold Container Population)

To confirm that the Spark job successfully ran and output its files, the team audited the storage layer using the Azure Portal Storage Browser. The active directory configuration of the Gold landing zone is captured in Figure 5.18.

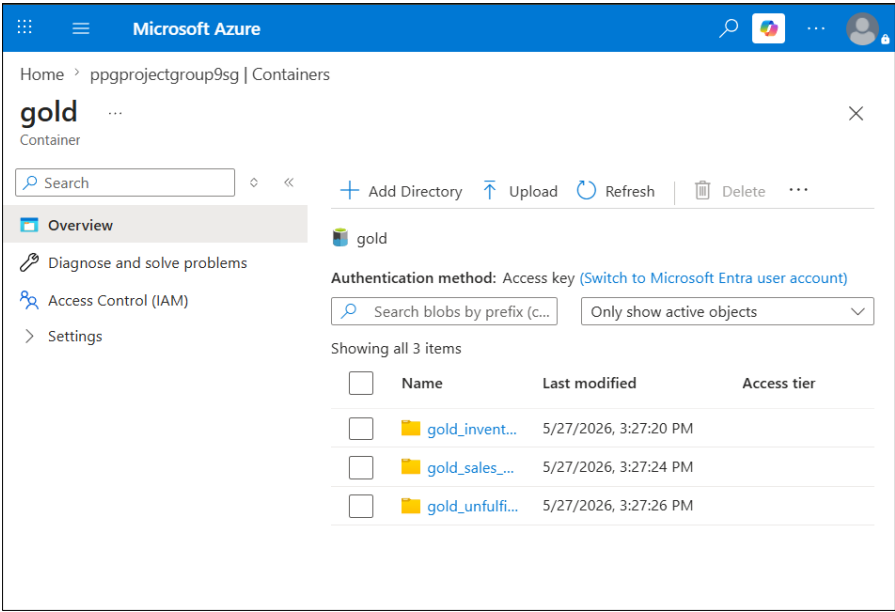


Figure 5.18 Azure Portal Storage Browser Verifying the Generation of the Gold Delta Table Directory

This verified Gold layer completes the primary data processing stage. The platform has successfully turned raw transactional logs into verified, algorithmically scored compliance data records, providing a high-performance foundation for the Synapse serving views and Power BI dashboards.

5.3.4 Phase 4: Serving Layer Aggregation and Optimization (Gold to Curated)

Once data lands in the Gold tier, it undergoes a final transformation step using Azure Synapse Analytics serverless SQL pools. This step optimizes data retrieval performance, creates structured business entities, and prepares the datasets for direct consumption by the BI layer.

The Synapse SQL script constructs views that structure the underlying Delta files into optimized, high-performance virtual tables, ensuring rapid data load speeds into the visualization semantic models.

1. Phase A — Cloud Infrastructure & Security Prerequisites

Figure 5.19 shows the step 1 documents the mandatory cloud setup and Access Management (IAM) configurations applied in the Azure Portal before running any SQL queries. It establishes five foundation tasks:

- (t) A1. Storage RBAC for Synapse: Grants the Storage Blob Data Reader role to the Synapse Workspace Managed Identity (ppgproject-group-9-synapse). This allows the serverless SQL engine to read data securely from the gold container without needing hardcoded keys.
- (u) A2. Developer Access Validation: Verifies that the developer account (muhammadadam.r@live.utm.my) holds Storage Blob Data Contributor rights to manage files across the data lake containers.
- (v) A3. Provisioning the Curated Container: Creates a new private container named curated within ADLS Gen2. This serves as the landing zone for the final aggregated tables generated by CETAS.
- (w) A4. Synapse Studio Permissions: Assigns the Synapse Administrator role to the developer account to enable query execution and script management in Synapse Studio.
- (x) A5. Serverless SQL Pool Login: Creates a master-level SQL login for the external Azure AD provider. This elevates the developer to the dbmanager role, granting permissions to build databases and metadata schemas.

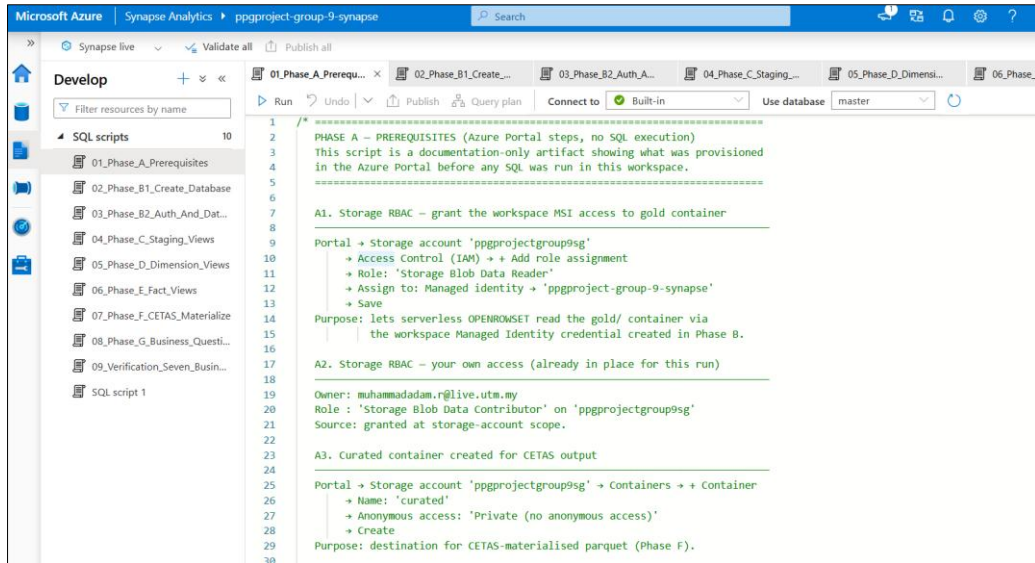


Figure 5.19 Phase A Content Screenshot

2. Phase B1 — Provisioning the Serving Database

Figure 5.20 shows the step 2 executes the initial SQL deployment command inside the Built-in serverless SQL pool, anchoring the metadata storage layer.

Target Database Initialization: The script switches context to the system master database to check for existing dependencies. If the target identity does not exist, it runs `CREATE DATABASE ppg_curated`.

Analytical Character Collation: The database configuration enforces a specialized collation flag: `Latin1_General_100_BIN2_UTF8`. This ensures strict case-sensitivity and binary comparison accuracy (BIN2) while natively supporting compressed UTF-8 character structures, optimizing processing speeds when reading raw strings directly from the data lake.

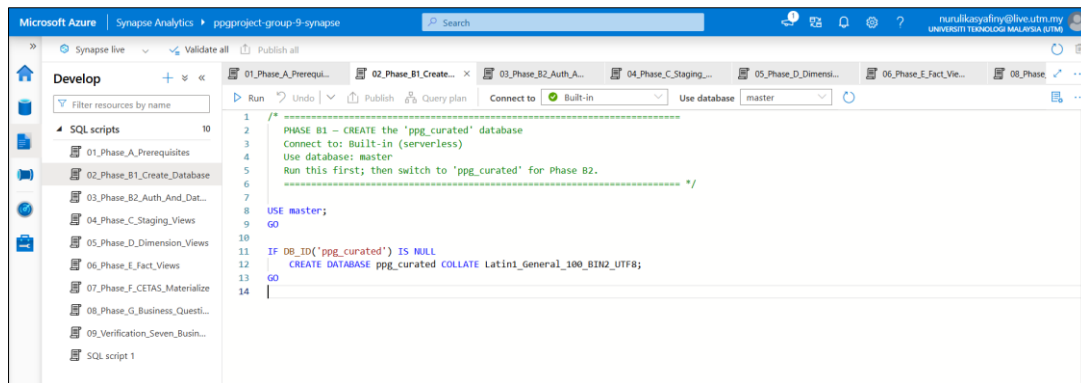


Figure 5.20 Phase B1 Content Screenshot

3. Phase B2 — Authentication, Data Sources, and Format Initialization

Figure 5.21 shows the step 3 sets up the security, storage connections, file compression rules, and logical organization inside the newly created ppg_curated database. It handles five structural configurations:

- (y) B2. Master Key Creation: Establishes a database master key encrypted with a strong password. This step is a security prerequisite required by Azure SQL to encrypt and protect credential secrets stored inside the database.
- (z) B3. Workspace Scoped Credential: Provisions a database-scoped credential named WorkspaceIdentity linked natively to Azure's Managed Identity. This configures Synapse to authenticate requests using its own cloud identity instead of using risky, hardcoded storage keys.
- (aa) B4a & B4b. External Data Sources (GoldLake & CuratedLake): Creates two pointer definitions pointing directly to the ADLS Gen2 file system paths. GoldLake is established as the read-only input path for the Databricks Delta tables, while CuratedLake is initialized as the dedicated write directory for the final serving layer files.
- (bb) B5. File Format Definition (ParquetSnappy): Configures a standard external file format. It instructs Synapse to compress and structure the upcoming physical table exports using Apache Parquet with Snappy compression, which drastically lowers cloud storage footprints and speeds up Power BI dashboard load times.

- (cc) B6. Schema Namespace Isolation (star): Spins up a custom metadata schema layer named star. This cleanly isolates the analytical Star Schema data assets (such as Fact and Dimension tables) from standard system objects.

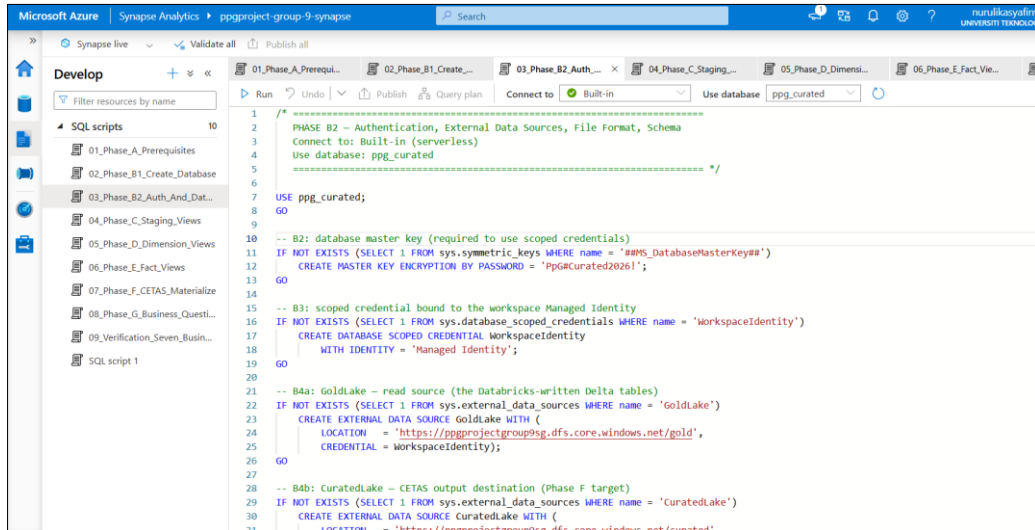


Figure 5.21 Phase B2 Content Screenshot

4. Phase C — Provisioning Data Staging Views

Figure 5.22 shows the step 4 builds a virtual staging layer using Synapse Serverless views (star.stg_...) to read directly from the Databricks Delta tables via the GoldLake external data source. This script executes three major tasks:

- (dd) Direct Cloud Reading via OPENROWSET: Rather than physically importing data, the views use the OPENROWSET query mechanism to parse files straight from the underlying Delta Lake architecture (FORMAT='DELTA').
- (ee) Data-Type Conversion and Schema Hardening: It acts as a defensive schema boundary. Since Delta tables can sometimes store values under loosely typed formats or generic VARCHAR text strings, the script uses safe conversion functions (TRY_CONVERT) to transform text into explicit, production-grade SQL types like DATE, INT, and exact DECIMAL scales.
- (ff) Creation of Key Downstream Pipelines: It provisions three virtual datasets mapping to your specific project goals:

- a. `star.stg_inventory`: Captures the core material log records along with calculated risk and write-down flags (Q1–Q5).
- b. `star.stg_sales_impact`: Exposes vulnerable open customer balances linked to component shortages (Q7).
- c. `star.stg_unfulfilled`: Exposes the manufacturing runs with zero floor consumption (Q6).

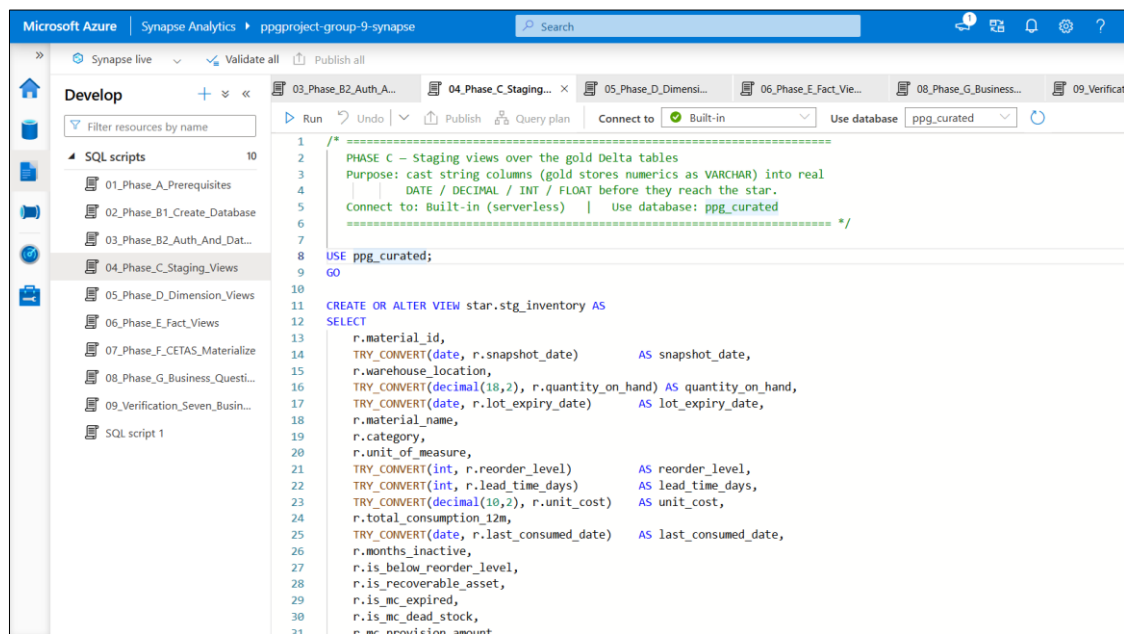


Figure 5.22 Phase C Content Screenshot

5. Phase D — Building the Dimension Views (Star Schema Foundation)

Figure 5.23 shows the step 5 defines the descriptive master attributes of the data model by provisioning 5 specialized Dimension Views (`dim_material`, `dim_warehouse`, `dim_customer`, `dim_finished_product`, and `dim_date`). This script establishes three critical data modeling structures:

Surrogate Key Generation (ROW_NUMBER()): To decouple the analytical schema from physical transactional keys, the script uses `ROW_NUMBER() OVER (ORDER BY ...)` over `DISTINCT` staging queries. This automatically generates clean, numerical, sequential primary surrogate keys (e.g., `material_key`, `warehouse_key`) required for robust star-schema modeling.

Consolidation and De-duplication: It reads from the staging views to extract unique descriptive entities. For example, `dim_finished_product` uses a UNION clause to combine final mixed paint products across both the sales ledger and unfulfilled production order tracking tables, eliminating duplicates.

Serverless-Optimized Time Dimension (`dim_date`): To avoid mixing serverless system catalogs with external file sources (which causes engine errors in Synapse), the script generates a highly creative temporal calendar table on the fly. It utilizes a VALUES-based cross-join digit generator to create sequential numbers, computes the minimum and maximum date boundaries found across all project transactions, and expands them into a full calendar. This view outputs descriptive reporting keys like `date_key` (in integer format YYYYMMDD), quarter, year, and an `is_month_end` compliance flag.

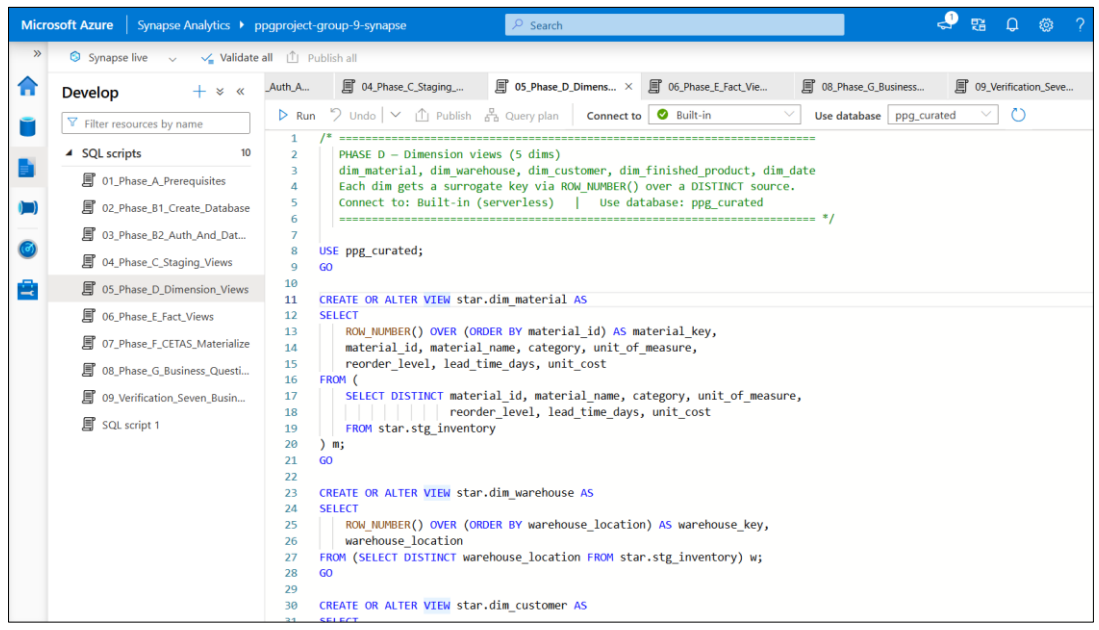


Figure 5.23 Phase D Content Screenshot

6. Phase E — Constructing the Fact Views (Star Schema Core)

Figure 5.24 shows the step 6 defines the relational core of the Star Schema by provisioning 3 specialized Fact Views (`fact_inventory_status`, `fact_unfulfilled_production`, and `fact_sales_order_impact`). This script acts as the structural bridge that wires metrics to dimensions:

Surrogate-Key Mapping (Foreign Key Stitching): Each view takes a flat staging view and runs standard SQL JOIN statements against the dimension views created in Step 5. It matches raw identifiers (like text dates or business IDs) to substitute them with the clean integer surrogate keys (such as `date_key`, `material_key`, `warehouse_key`), which are highly optimized for fast dimensional querying.

Separation of Measures and Context: The script isolates the numeric business metrics and flags (measures) while anchoring them to the dimension tables that provide analytical context (who, what, where, and when).

Creation of Three Core Analytical Focus Areas:

- (gg) `fact_inventory_status`: Implements the central business matrix. It captures inventory volumes along with all calculated risk parameters (such as safety stock violations, dead-stock statuses, and cash write-down amounts) on a specific date at a particular warehouse.
- (hh) `fact_unfulfilled_production`: Sets up the metrics for production-floor issues. It maps specific manufacturing failures and materials to the planned finished paint product, answering product fulfillment risks.
- (ii) `fact_sales_order_impact`: Sets up the metrics for delivery backlog risks. It links client demand parameters (`quantity_ordered`) directly to the specific raw material shortage tracking keys.

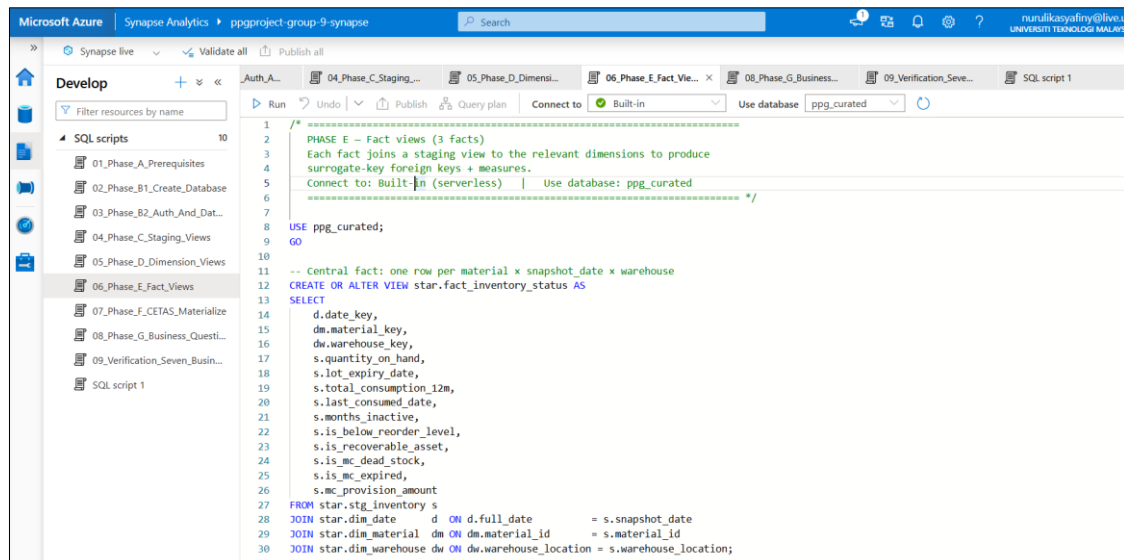


Figure 5.24 Phase E Content Screenshot

7. Phase F — Materializing the Star Schema via CETAS

Figure 5.25 shows the step 7 optimizes and solidifies the virtual views into permanent physical storage by running CETAS (Create External Table As Select) operations across 5 dimension tables and 3 fact tables. It executes three core tasks:

- (jj) **Physical Data Materialization:** Virtual views are convenient, but scanning raw data lakes repeatedly causes performance bottlenecks. This script runs a compilation loop that physically extracts data from the views and writes it directly into the curated storage container via the CuratedLake data source.
- (kk) **Storage and Speed Optimization:** By binding the tables to the ParquetSnappy format defined in Step 3, the engine compresses all output streams into highly performant, columnar Snappy-compressed Apache Parquet files. This reduces file system sizes and ensures near-instantaneous load times for downstream Power BI direct-import engines.
- (ll) **Idempotent Cleanup Routines:** Because Synapse's CETAS engine strictly guards existing directory files and will throw an error instead of overwriting data, each block includes a protective drop statement (DROP EXTERNAL TABLE...). This drops the database metadata pointers, allowing the new optimized tables (star.cur_...) to be generated cleanly.

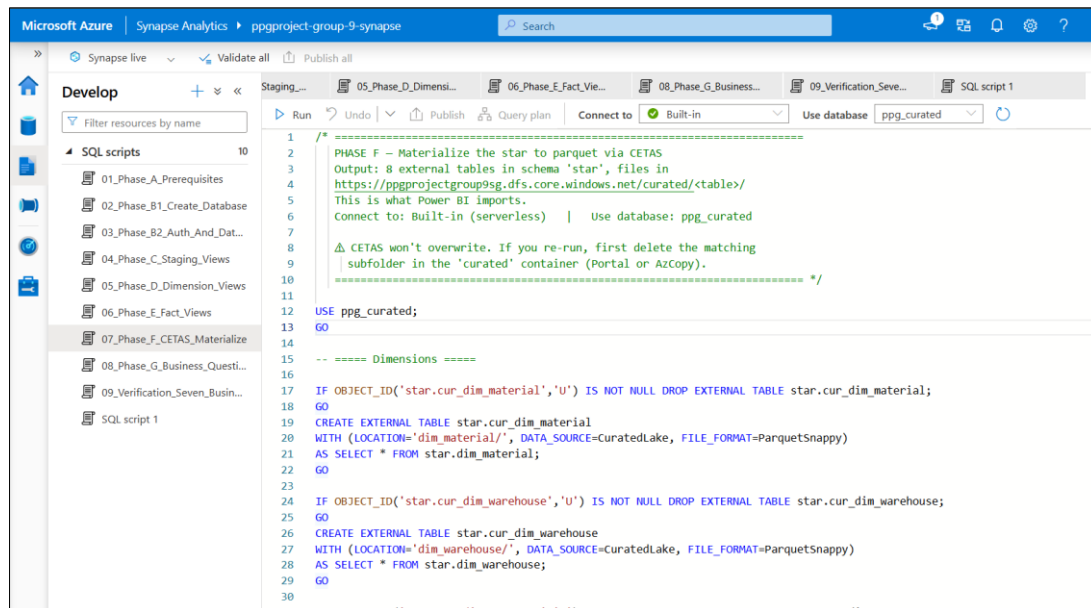


Figure 5.25 Phase F Content Screenshot

8. Phase G — Serving Layer Business Queries Execution

Figure 5.26 shows the step 8 contains the diagnostic SQL reporting layer. It runs specialized analytical queries against the finalized, CETAS-materialized external tables (star.cur_...) to immediately provide actionable answers to PPG's core business questions. It is broken into seven clear reporting components:

- (mm) Q1. Safety Stock Breach Tracker: Extracts every inventory asset whose physical stock level has fallen strictly below its cataloged safety threshold (is_below_reorder_level = 1). This allows procurement managers to identify immediate raw material deficit trends.
- (nn) Q2. Recoverable Assets Audit: Filters the warehouse stock records to highlight any item with quantities exceeding its entire 12-month historical consumption track (is_recoverable_asset = 1). This pinpoints overstock targets where capital can be reclaimed.
- (oo) Q3 & Q4. Magna Carta Risk Segmentation: Splitting the operational risk layers, Q3 extracts assets marked as completely stagnant for 9 months, along with their associated financial write-down values. Q4 isolates components that have bypassed their expiration threshold (is_mc_expired = 1), sorting them by execution date.

- (pp) Q5. Financial Provision Write-Down Summary: Aggregates and reports the overarching, headline project KPI (total_mc_provision_rm) using high-precision casting (decimal(18,2)). It compiles a separate summary block grouping data into 'Expired', 'Dead Stock', or 'No provision' buckets to show auditors the exact source of material losses.
- (qq) Q6. Production Plant Outage Diagnostic: Returns a list of the 10 most recent factory assembly errors driven by a component stockout (quantity_consumed = 0). It follows this up with an aggregation query counting total failures by finished paint product, showing which formulas suffer from the worst supply shortages.
- (rr) Q7. Downstream Customer Backlog Impact Trace: Pins down supply chain risks by matching component stockout vulnerabilities directly to open customer sales files. The final aggregation sub-query tallies the exact number of client accounts impacted by each specific raw material bottleneck, letting account managers prioritize incoming replacement shipments.

```

1  /*
2  3  PHASE G - The 7 business questions, paste-ready
4  5  Each query reads from the CETAS-materialised cur_* external tables.
6  7  Connect to: Built-in (serverless) | Use database: ppg_curated
7  8  ===== */
9  10
10 11 USE ppg_curated;
12 13 GO
14 14
15 15 -- Q1 - materials below reorder level
16 16 SELECT dm.material_id, dm.material_name, dm.category,
17 17 f.quantity_on_hand, dm.reorder_level
18 18 FROM star.cur_fact_inventory_status f
19 19 JOIN star.cur_dim_material dm ON dm.material_key = f.material_key
20 20 WHERE f.is_below_reorder_level = 1
21 21 ORDER BY dm.material_id;
22 22
23 23 -- Q2 - Recoverable Assets
24 24 SELECT dm.material_id, dm.material_name, dm.category,
25 25 f.quantity_on_hand, f.total_consumption_12m
26 26 FROM star.cur_fact_inventory_status f
27 27 JOIN star.cur_dim_material dm ON dm.material_key = f.material_key
28 28 WHERE f.is_recoverable_asset = 1
29 29 ORDER BY dm.material_id;
30 30
31 31 -- Q3 - Magna Carta Dead Stock
32 32 SELECT dm.material_id, dm.material_name,
33 33 f.months_inactive, f.quantity_on_hand, f.mc_provision_amount
34 34 FROM star.cur_fact_inventory_status f
35 35 JOIN star.cur_dim_material dm ON dm.material_key = f.material_key
36 36 WHERE f.is_mc_dead_stock = 1
37 37 ORDER BY dm.material_id;
38 38
39 39
40 40
41 41
42 42
43 43
44 44
45 45
46 46
47 47
48 48
49 49
50 50
51 51
52 52
53 53
54 54
55 55
56 56
57 57
58 58
59 59
60 60
61 61
62 62
63 63
64 64
65 65
66 66
67 67
68 68
69 69
70 70
71 71
72 72
73 73
74 74
75 75
76 76
77 77
78 78
79 79
80 80
81 81
82 82
83 83
84 84
85 85
86 86
87 87
88 88
89 89
90 90
91 91
92 92
93 93
94 94
95 95
96 96
97 97
98 98
99 99
100 100

```

Figure 5.26 Phase G Content Screenshot

9. Verification Phase — End-to-End Star Schema Performance Verification

Figure 5.27 shows the step 9 serves as the final system verification and testing layer within the Azure Synapse serverless pool. This script serves two primary

purposes: generating a centralized master scorecard and auditing execution performance benchmarks.

Centralized Master Scorecard Integration: The script initiates a comprehensive SELECT statement that combines scalar sub-queries to capture a summary of all 7 core business parameters at a single glance. It simultaneously pulls metrics on inventory safety stock levels, overstock vectors, expired batches, dead stock reserves, financial write-down allocations, manufacturing plant shortages, and open customer backlog positions into a single row.

Granular Reporting Loop Redundancy: Following the summary scorecard, the script runs through individual verification blocks for business metrics Q1 through Q7. This matches exactly with Step 8 to ensure the analytical counts remain consistent across both individual drill-down views and aggregate reporting states.

Programmatic Performance and Latency Tracking: This block serves as empirical proof of architectural efficiency for project examiners. By setting an initial timestamp marker (`DECLARE @t0 datetime2(3) = SYSDATETIME();`) and calculating the millisecond difference after every individual block (`DATEDIFF(millisecond, @t, SYSDATETIME())`), the script prints live latency metrics to the messages console. This provides mathematical verification that the underlying Snappy-compressed Parquet tables optimize query runtimes, delivering responses for all seven business questions in under 5 seconds.

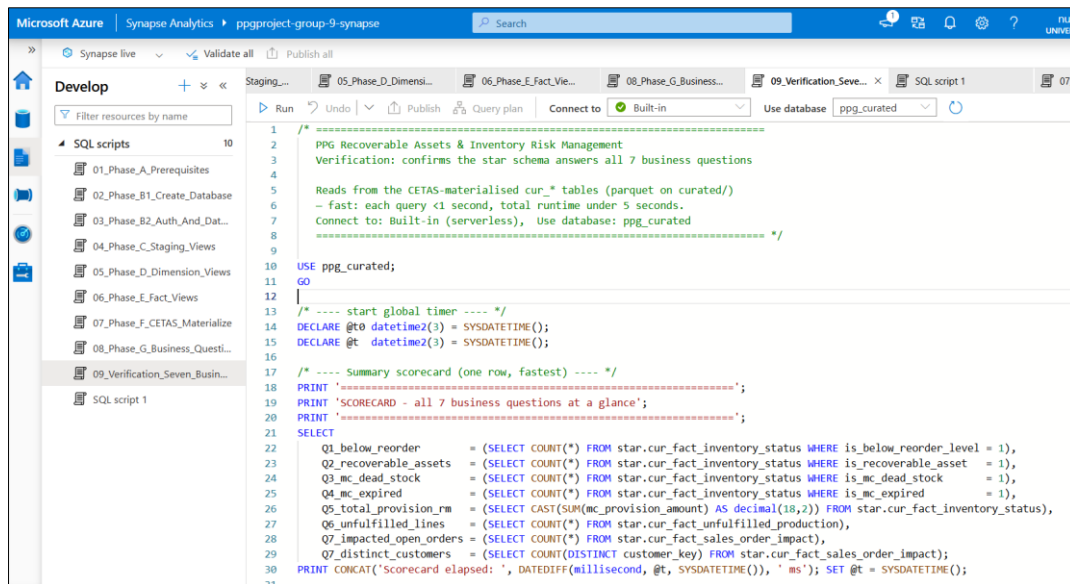


Figure 5.27 Verification Phase Content Screenshot

To conclude Phase 4 and confirm that the serverless SQL pool successfully executed the CETAS operations, a final cloud state audit was performed using the Azure Portal Storage Browser. The resulting directory structure within the analytical serving layer is captured in Figure 5.28.

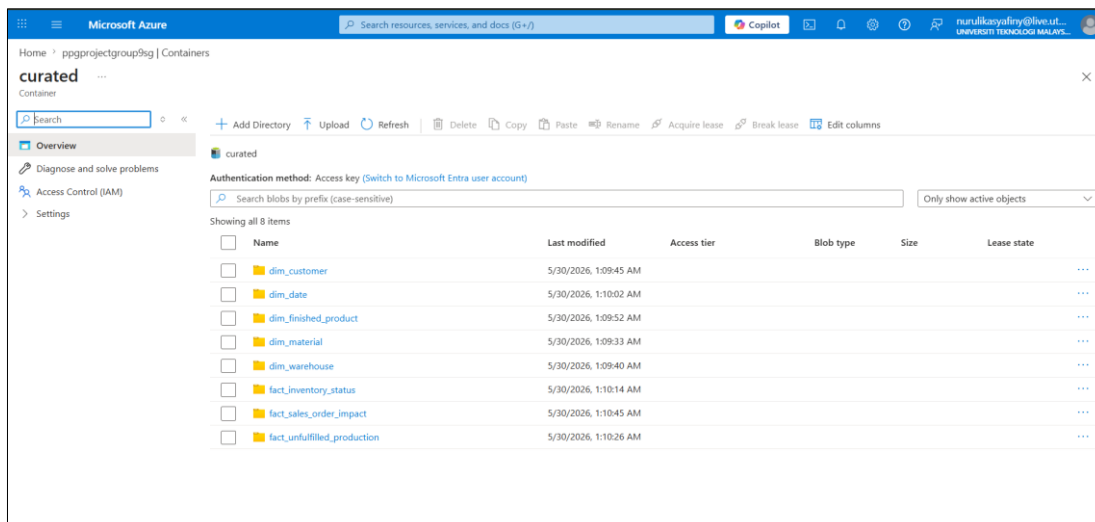


Figure 5.28 Azure Portal Storage Browser Verifying Clean Structure and Population within the Curated Tier Container

5.3.5 Phase 5: Semantic Modeling and DAX Formula Engineering (Curated to Power BI)

The final phase of the Medallion Data Platform architecture transitions the data assets from the physical storage layers of the data lake into an analytical semantic model within Power BI Desktop. Shifting data from a raw lake to a structured semantic layer allows business users to interact with complex supply chain information effortlessly, running calculations over millions of rows with sub-second response times.

5.3.5.1 Semantic Model Galaxy Schema Architecture

Rather than implementing a basic Star Schema with a single isolated fact table, our team designed and implemented an enterprise-grade Galaxy Schema or Fact Constellation Schema. This approach is necessary because complex supply chain networks involve multiple business processes like inventory records, manufacturing operations, and customer order backlogs that exist at different levels of detail but must share core master dimensions. The implementation state of this semantic model is captured in Figure 5.29.

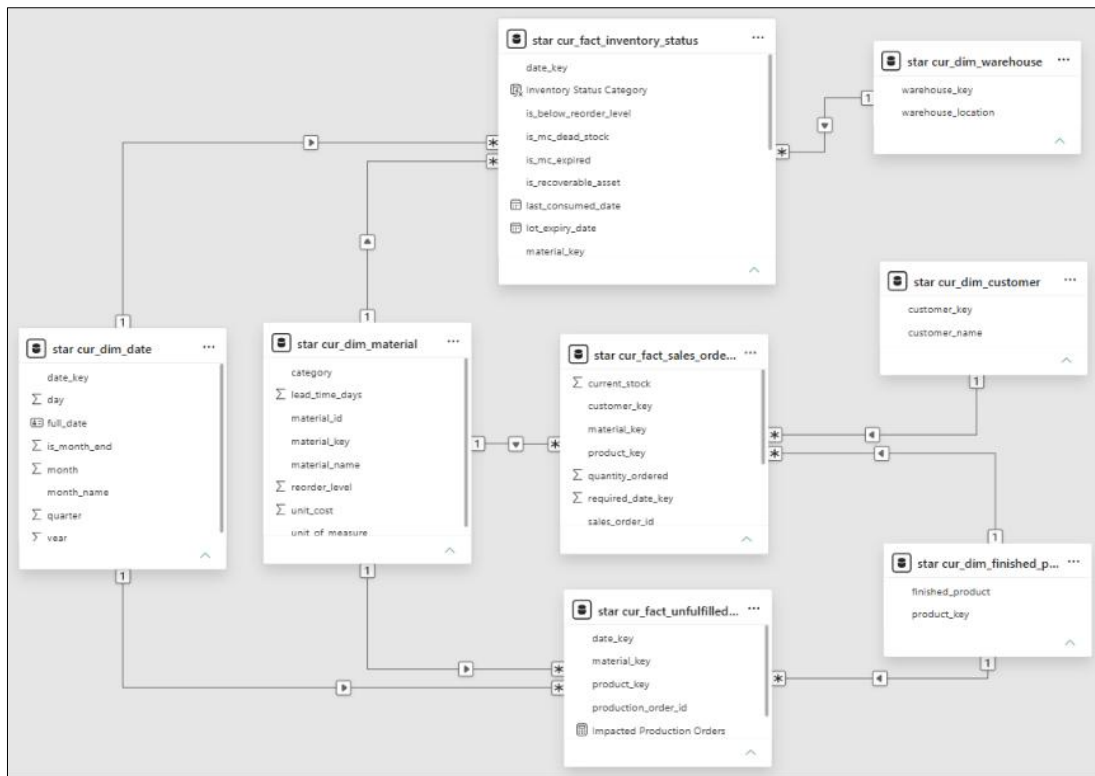


Figure 5.29 Power BI Semantic Model Galaxy Schema Blueprint

Fact Constellation Routing: The architecture features three distinct fact tables processing data simultaneously: **cur_fact_inventory_status**, **cur_fact_unfulfilled_production**, and **cur_fact_sales_order_impact**. Each fact table handles a unique transactional process.

Conformed Dimensions (Shared Context): The model relies on conformed dimensions—specifically **cur_dim_material** and **cur_dim_date**. These dimensions bridge the gaps between your tables, allowing cross-fact analysis. For instance, a single filter applied to a specific paint material code will filter warehouse stock levels, list corresponding plant stockouts, and flag affected customer orders at the same time.

One-to-Many (1:*) Single Direction Filters: All active relationships use a strict 1:* cardinality rule. The filter propagation paths are set to Single Direction, flowing exclusively downstream from the master lookup tables (**cur_dim_...**) to the fact tables. This structure completely prevents circular relationship loops, ambiguous pathways, and cross-filtering errors.

5.3.5.2 Technical Breakdown of Core DAX Measures

To transform the raw factual data within the Galaxy Schema into interactive dashboard metrics, specialized Data Analysis Expressions (DAX) were engineered. Figure 5.30 is the functional breakdown of each measure deployed:

1. Safety Stock Alert Metric (COUNT Below Reorder Level)

```
1 COUNT Below Reorder Level =  
2 CALCULATE(  
3 |   COUNTROWS('star_cur_fact_inventory_status'),  
4 |   'star_cur_fact_inventory_status'[is_below_reorder_level] = 1  
5 )
```

Figure 5.30 Code for Counting Below Reorder Level

As shown in figure 5.30, this measure utilizes CALCULATE to alter the filter context and applies a strict boolean filter where `is_below_reorder_level = 1`. It executes a COUNTROWS scan over the materialized `star_cur_fact_inventory_status` table to evaluate the exact number of unique inventory records that have breached their minimum safety stock threshold. This acts as an automated procurement flag for plant managers.

2. Stagnant Stock Quantification (Qty Dead Stock)

```
1 Qty Dead Stock =  
2 CALCULATE(  
3 |   SUM('star_cur_fact_inventory_status'[quantity_on_hand]),  
4 |   'star_cur_fact_inventory_status'[is_mc_dead_stock] = 1  
5 )
```

Figure 5.31 Code for Counting Dead Stock Quantity

As shown in figure 5.31, designed to quantify the physical volume of non-moving inventory, this measure aggregates the `quantity_on_hand` column. By restricting the evaluation context to rows matching `is_mc_dead_stock = 1`, it filters out

active materials to calculate the total physical volume of paint components that have remained dormant for less or equal than 9 months.

3. Spoiled Product Quantification (Qty Expired Stock)

```
1 Qty Expired Stock =  
2 CALCULATE(  
3     SUM('star_cur_fact_inventory_status'[quantity_on_hand]),  
4     'star_cur_fact_inventory_status'[is_mc_expired] = 1  
5 )
```

Figure 5.32 Code for Counting Expired Stock Quantity

As shown in figure 5.32, this is metric sums the total physical volume of inventory (quantity_on_hand) that has passed its expiration window. By applying a filter context where is_mc_expired = 1, the calculation isolates unuseable chemical or material batches, allowing logistics teams to trace active shelf-life failures across warehouses.

4. Excess Capital Allocation (Qty Recoverable Assets)

```
1 Qty Recoverable Assets =  
2 CALCULATE(  
3     SUM('star_cur_fact_inventory_status'[quantity_on_hand]),  
4     'star_cur_fact_inventory_status'[is_recoverable_asset] = 1  
5 )
```

Figure 5.33 Code for Counting Recoverable Assets Quantity

As shown in figure 5.33, this metric evaluates working capital inefficiencies by summing up the physical quantity_on_hand for materials where is_recoverable_asset = 1. This targets items whose current stock levels exceed their trailing 12-month total consumption, highlighting exactly where the organization has over-purchased materials.

5. Financial Risk Valuation (Total Provision Amount)

```
1 Total Provision Amount = SUM('star cur_fact_inventory_status'[mc_provision_amount])
```

Figure 5.34 Code for Counting Total Provision Amount

Figure 5.34 serving as a headline financial KPI for executive auditing, this expression computes the overall fiscal loss exposure. It executes a direct SUM over the calculated mc_provision_amount column within the central inventory fact table. This provides a clear monetary value of the written-down capital tied up in expired and dead stock.

6. Supply Chain Client Risk Tracker (Impacted Customers)

```
1 Impacted Customers = DISTINCTCOUNT('star cur_fact_sales_order_impact'[customer_key])
```

Figure 5.35 Code for Counting Impacted Customer

Figure 5.35 shows the measure shifts analytical context from inventory metrics over to customer fulfillment tracking. By performing a DISTINCTCOUNT on the customer_key field inside the star cur_fact_sales_order_impact table, it calculates the net number of individual clients whose open commercial orders are actively delayed by upstream raw material stockouts.

The culmination of the Medallion pipeline, the Galaxy Schema, and the engineered DAX measures is the interactive executive dashboard. Figure 5.36 displays the production-ready Power BI reporting canvas, which translates these complex supply chain metrics into actionable business insights.

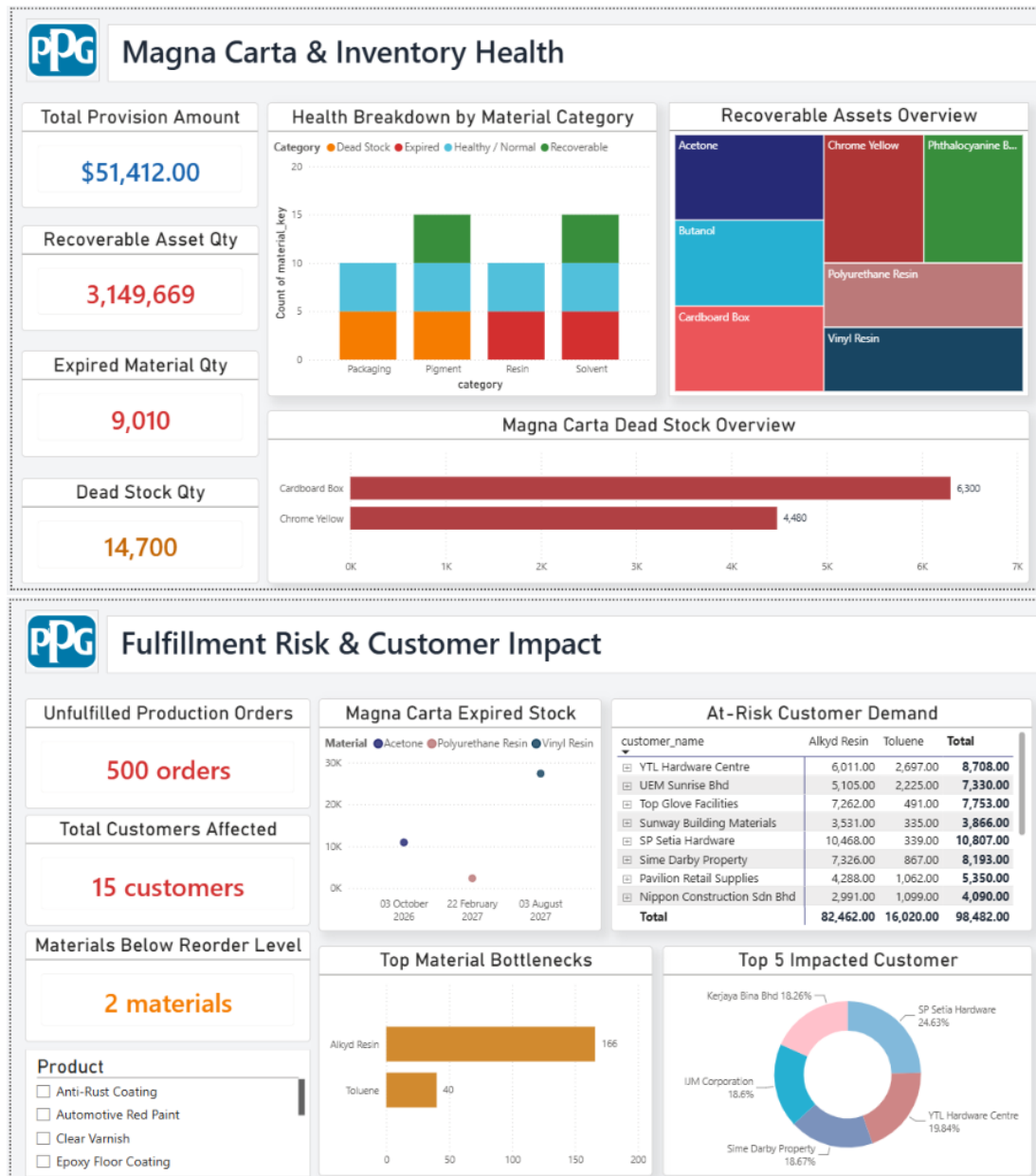


Figure 5.36 Final Power BI Dashboard

5.4 Interfaces of System

The PPG Inventory Risk Management contain four layers of processing including ingestion, cleaning and transformation, and analytical serving. This section explains the the user-facing tools used to operate, monitor, or consume each layer. Each layer uses different tools, and designed to be used by different user role.

5.4.1 Ingestion Interface – Azure Data Lake Storage Gen 2

Based on figure 5.37, the Bronze layer is managed using the storage container inside Azure Data Lake Storage Gen2. After the ingestion script finishes uploading files, the user opens the storage account through ppggroupprojectsg and clicks Storage Browser in the left menu. The Storage Browser will show the storage account in a tree list format that include containers like Bronze, Silver, Gold, and Curated. The user will be able to click any container to view the file inside it. There is a right-hand panel showing detailed information about the file including the size, last modified date, lease status, storage connector and content hash.

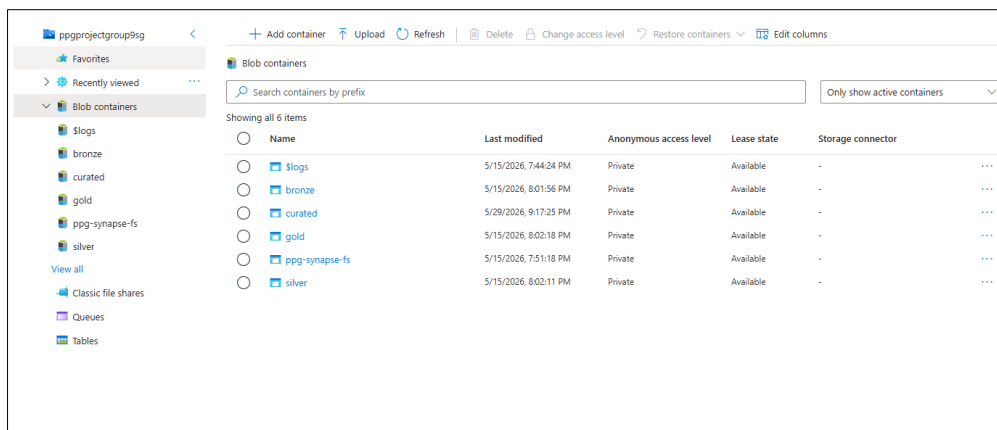


Figure 5.37 Azure Storage Browser in ADLS Gen2

The Storage Browser is used mainly to check the six CSV files that were uploaded and created correctly by looking at their file sizes and timestamps. The second is to monitor any mistaken file upload, therefore acting like an inspection tool for all phases of container. This means operators only need to learn one interface for all four medallion layers.

5.4.2 Transformation Interface (Silver Layer) - Azure Data Factory Studio

The silver layer managed through Azure Data Factory Studio, a web-based tool lets the user to design, schedule and observed the Bronze to Silver transformations. The Studio contains four hubs in its arsenal including Home, Author, Monitor, and

Manage. Figure 5.38 shows the canvas of the pipeline1 which is the main pipeline interface that connect two activities together including a Mapping Data Flow activity followed by Databricks Notebook activity. Each of the activity is a draggable block.

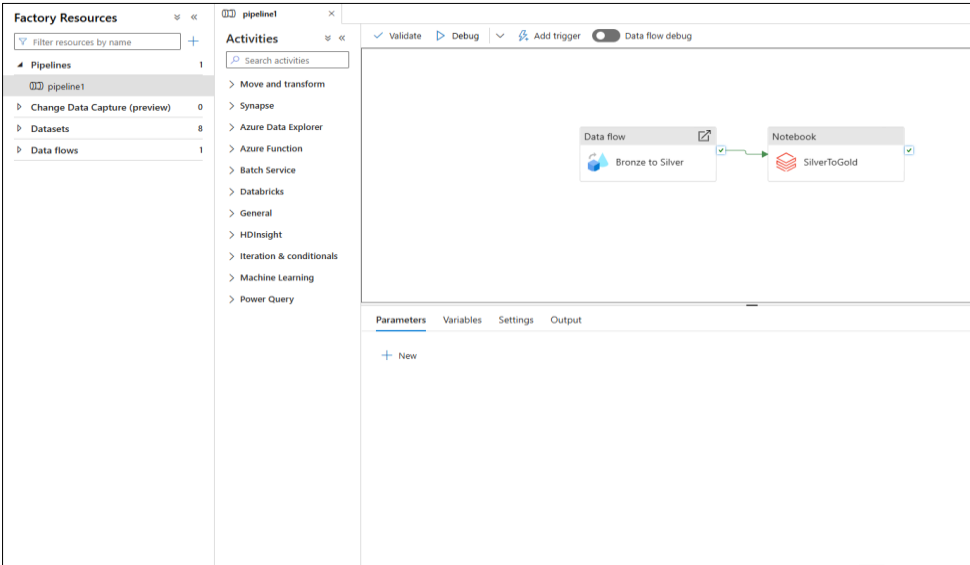


Figure 5.38 Azure Data Factory Studio: Pipeline1

The Monitor hub, shown in Figure 5.39, displays a time-ordered list of every pipeline execution. For each run, the Studio shows the trigger source (manual or scheduled), the duration of every activity, and a colour-coded status pill (Succeeded, Failed, or In Progress). Clicking a specific run opens a detail page with input and output panels, debug logs, and data previews. The engineering team uses this hub mainly to confirm that the pipeline finishes correctly after every demonstration cycle.

Pipeline name	Run start	Run end	Duration	Status	Triggered by	Run ID	Parameters
pipeline1	6/8/2026, 6:29:33 PM	--	1m 22s	In progress	Manual trigger	ab4832cb-f9b7-4c06-b2d...	

Figure 5.39 Azure Data Factory Studio: Monitor Hub

5.4.3 Transformation Interface (Gold Layer) - Azure Databricks Workspace

The Gold Layer as represented in the architecture is built using the Azure Databricks Workspace, which provides a python notebook style editing for Pyspark code. Figure 5.40 shows the SilverToGold notebook open in the editor. The workspace itself combine a folder and cluster navigator on the left, a scrollable list of code cells in the centre, a top toolbar with Run, Run-all, Schedule, and Clear-state buttons, and a result panel that shows DataFrame previews directly below each cell after it runs.

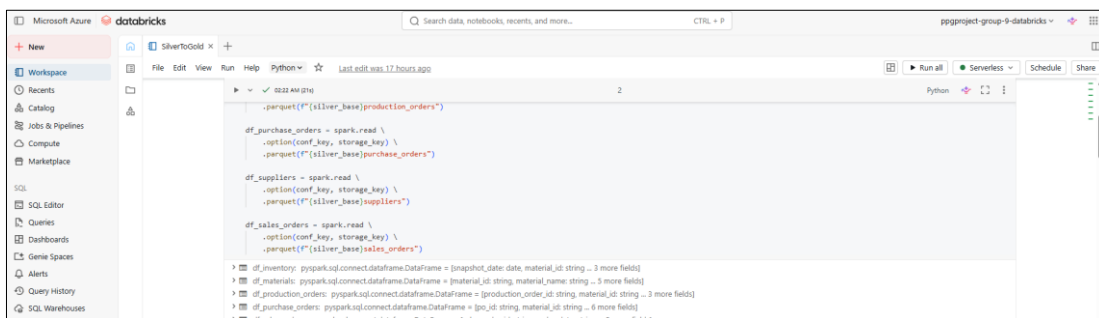


Figure 5.40 Azure Databricks Workspace: SilverToGold Notebook Editor

5.4.4 Transformation Interface (Curated Layer) – Azure Synapse Studio

The curated layer in this pipeline is served through Azure Synapse Studio that combines SQL editing, lake-storage navigation, access control, and pipeline monitoring in a single workspace. The menu in the left side of the screen is quite similar with ADF with extra two hub including Data and Develop hub.

Figure 5.41 shows the Develop Hub containing the total of 10 SQL scripts generated in order to build the star schema table inside the curated container with an inclusion of verification phase and E2E test case. Any user with access control can read the scripts and adjust it in order, from phase 01 until phase 09 and recreate the same database state.

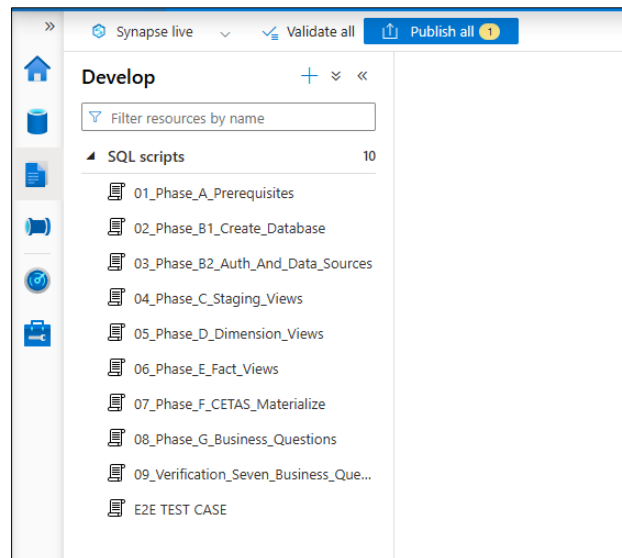


Figure 5.41 Azure Synapse Studio Develop Hub: Curated SQL Scripts

Figure 5.42 below shows the result state of a runned sql script. Before running, the sql scripts should be connected to the built-in Serverless pool and the ppg_curated database. After the script runs, the bottom panel splits into a Results grid and a Messages console.

Results		
Messages		
Select	Query 9	View
		Table Chart
Export results		
Search		
material_id	material_name	impacted_orders
MAT006	Alkyd Resin	166
MAT012	Toluene	40

00:00:15 Query executed successfully.

Figure 5.42 Azure Synapse Studio SQL Editor: Results Grid and Messages

5.4.5 Serving Layer – Microsoft Power BI Desktop

The presentation layer is accessible to business users via Microsoft Power BI Desktop. The galaxy schema (Figure 5.9) alongside the page layouts in Section 4.5.5 specifies what the dashboard must display. This part outlines the runtime interface

specifically, what the user interacts with after the data has been loaded. Power BI Desktop provides three views in the left sidebar: Report (the canvas for the dashboard), Data (a grid representation of any imported table), and Model (a diagram illustrating table relationships). Two report pages were created, to represent different operational issues.

5.5 Main Functions Testing

In this chapter, the whole pipeline will be tested to ensure that the data quality and the business logic in this project. The testing method included data quality and schema testing, business logic testing and end-to-end pipeline integration testing.

5.5.1 Data Quality and Schema Testing

Data quality and pattern testing can verify whether the raw data entering the bronze layer has undergone correct cleaning, standardisation and structural verification before entering the silver layer.

5.5.1.1 Test Approach

Refer to figure 5.43, the test is crucial because it can prevent ambiguous or improperly formatted data corruption from the complex business logic that will be applied in the later stages of the Medallion architecture.

Schema

Input

Output

Number of columns

Total

7

Order	↑↓	Column	↑↓	Type	↑↓	Used by	↑↓
1		material_id		abc	string	material_id	
2		material_name		abc	string	material_na	
3		category		abc	string	category	
4		unit_of_measure		abc	string	unit_of_me	
5		reorder_level		abc	string	reorder_lev	
6		lead_time_days		abc	string	lead_time_c	
7		unit_cost		abc	string	unit_cost	

Schema

Input

Output

Number of columns

New

6

Dropped

6

Unchanged

1

Total

7

Order	↑↓	Column	↑↓	Type	↑↓	Aggregated as	Based on	↑↓
1		material_id		abc	string	Group by	material_id	
2		material_name		abc	string	Aggregate	material_na	
3		category		abc	string	Aggregate	category	
4		unit_of_measure		abc	string	Aggregate	unit_of_me	
5		reorder_level		abc	string	Aggregate	reorder_lev	
6		lead_time_days		abc	string	Aggregate	lead_time_d	
7		unit_cost		abc	string	Aggregate	unit_cost	

Figure 5.43 Inspecting input and output schemas within the ADF data flow to validate correct data type casting for material dataset

5.5.1.2 Test Cases

As shown in table 5.1, multiple test cases have been built to verify the data cleaning process during the transition from bronze to silver layer.

Table 5.1 Test Case Description

Test Case	Description
Schema Mapping and Type Casting	The general string definition has been verified and successfully converted into a clear digital format. For example, unit_cost and quantity_on_hand have been tested to ensure that they are accurately converted to double-precision floating point, while reorder_level has been successfully converted to integers.
Date Standardisation	Check the date string in the file such as lot_expiry_date to confirm the successful reformatting and convert the plain text to the standardised ISO YYYY-MM-DD date index format.
Text Cleansing	Verify whether the strings in columns such as material_name, warehouse_location and customer_name have been cleaned up correctly, including deleting hidden trailing spaces, standardising case and deleting damaged characters.
Null Handling and Default Values	Make sure that the missing values such as blanks in historical consumption data have been processed correctly. For example, convert empty values to actual zero to prevent errors in subsequent mathematical calculations.
File Format Conversion	It has been confirmed that the output table has been successfully converted from a flat, uncompressed CSV structure to an optimised columnar Apache Parquet file.

Column name *

category

Expression

```
iif(isNull(byName('category')), 'Uncategorized', trim(toString(byName('category'))))
```

Figure 5.44 Test on Category

Figure 5.44 shows that if the category is missing, it will be marked as Uncategorized to prevent downstream calculation from breaking due to missing values. If it is not empty, it will take the existing value and convert it into text and remove the extra hidden spaces.

Incoming stream *

Inventory

Group by
Aggregates

Columns	Name as
12 snapshot_date	snapshot_date
abc material_id	material_id
abc warehouse_location	warehouse_location

Figure 5.45 Configuring the Aggregate transformation in Azure Data Factory to group by primary keys and remove the duplicate records from the inventory snapshot.

Figure 5.45 shows that the data are group by the columns of snapshot_date, material_id and warehouse_location primary keys, to prevent the duplicate data records from the incoming stream.

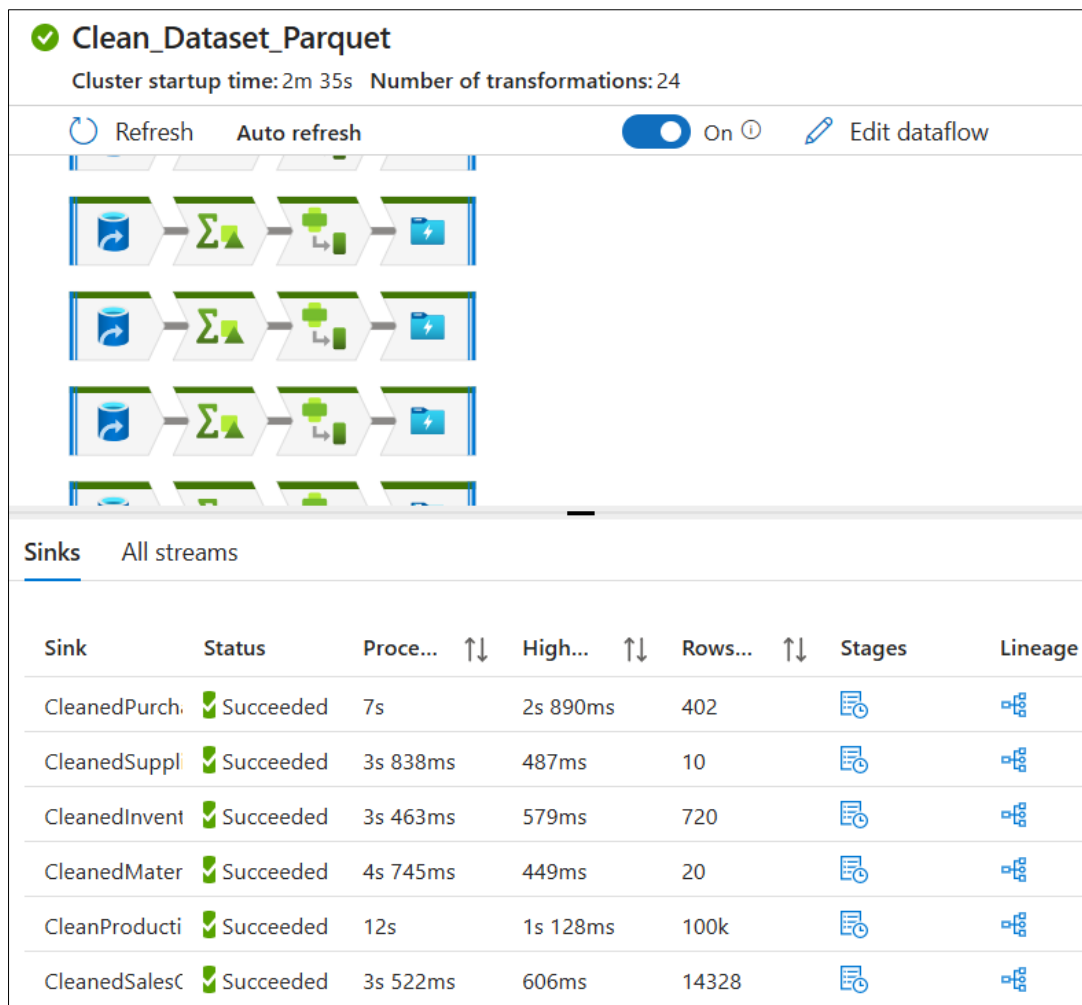


Figure 5.46 Data Flow execution metrics confirming the successful filtering of duplicate and null records before writing to the Silver Parquet sink

As shown in figure 5.46, both the data quality and the pattern test have passed. The Azure Data Factory pipeline captures and converts abnormal data to ensure that only clean and verified Parquet files are contained in the Silver layer. By enforcing strict data types and eliminating structural inconsistencies, the system lays the foundation for reliable and high performance for the Databricks algorithm running on the Gold layer.

5.5.2 Business Logic Testing

Business logic testing verifies that the Gold-layer is transformed correctly by implementing PPG's RA and Magna Carta rules where each flag raised exactly when the business definition says it should. The business logic testing is different from data quality testing and pipeline testing. The scope was the seven business questions as encoded in the `apply_business_rules` function and the supporting consumption logic.

5.5.2.1 Test approach

The unit test validates the rule of logic against expected outcomes derived by hand from the business specification, rather than by re-running the pipeline and inspecting its output. This test is capable of exposing any divergence between the implemented code and the intended definition because comparing it against an independent reading of the business rules. Six fixture materials were constructed to represent normal cases, boundary conditions, and edge cases, and each was assigned an expected flag value determined from the rule definition.

The reconciliation test that checks invariants which must hold true on the real gold output regardless of the input data. These confirm that the output table is non-empty, expired and dead stock classifications are mutually exclusive, that every dead stock material is also a recoverable asset (as the specification mentioned to be a subset of recoverable assets), and the total financial provision reconciles when computed two independent ways.

Both test layers were positioned to run before the write step, as they are function as a gate where the gold table only exists if the logic is proven correct. This is to prevent incorrect values from propagating downstream into synapse star schema and the Power BI reporting layer.

5.5.2.2 Test cases

The example cases and their expected outcomes are summarized in Table 5.2

Table 5.2 Test Cases and Scenarios

Case	Scenario	Rule under test	Expected result
Healthy	Quantity on hand (100) below reorder level (200)	Below reorder level	Flagged
RA-Bound	Quantity on hand equal to 12-month consumption (50 = 50)	Recoverable Asset	Not flagged — "exceeds" is a strict greater-than
Dead	Inactive for 12 months, no lot expiry date	Dead Stock	Flagged
Expired	Lot expiry date in the past	Expired	Flagged
Fut_exp	Inactive for 12 months but carrying a future expiry date	Dead Stock	Not flagged — the material has an expiry date
New_stock	Never consumed, only recently received	Dead Stock	Not flagged — new stock, not dead stock

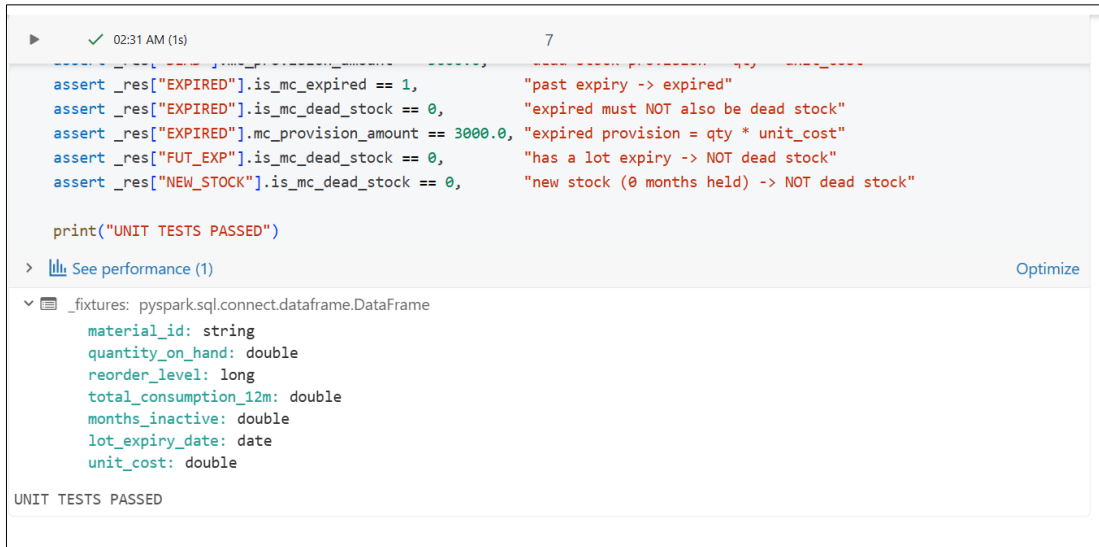
5.5.2.3 Business Rule Decision Documented

The testing process had to clear up two spots where the source spec was not clear. Those got turned into explicit choices that were recorded on purpose. It seems one rule is that dead stock under the magna carta label only applies when a lot has no expiry date at all. Materials sitting around for nine months or longer can keep their future expiry and stay classed as recoverable instead of getting written down. The Fut_exp case checks that part.

For items never consumed before the date to start counting inactivity comes from the first time they appeared in the snapshots. Using a fixed starting value would have pulled new stock into the write down right away, which is not the intent. The new stock case covers this. Tying both choices back to their test cases makes the decisions easier to trace later. Some parts of how the rules line up still feel a bit loose though.

5.5.2.4 Results

Tests passed for both suites. The unit tests covered the six cases, and they gave the expected flags. Reconciliation tests showed the invariants held on the full gold output. Figure 5.47 and Figure 5.48 show the passed messages for unit and reconciliation.



The screenshot shows a Jupyter Notebook interface. At the top, there's a status bar with a green checkmark, the time '02:31 AM (1s)', and the number '7'. Below this, a code cell contains several assertions and a print statement. The assertions check the state of 'EXPIRED', 'FUT_EXP', and 'NEW_STOCK' materials. The print statement says 'UNIT TESTS PASSED'. Below the code cell, there's a 'See performance (1)' link and an 'Optimize' button. A variable '_fixtures' is expanded, showing a PySpark DataFrame with columns: material_id (string), quantity_on_hand (double), reorder_level (long), total_consumption_12m (double), months_inactive (double), lot_expiry_date (date), and unit_cost (double). At the bottom, a message says 'UNIT TESTS PASSED'.

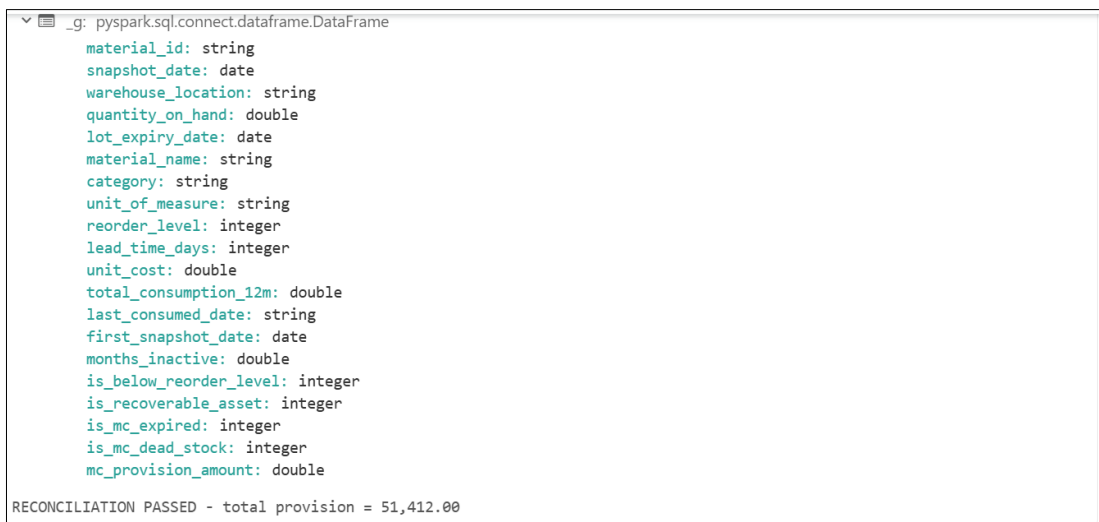
```
assert _res["EXPIRED"].is_mc_expired == 1, "past expiry -> expired"
assert _res["EXPIRED"].is_mc_dead_stock == 0, "expired must NOT also be dead stock"
assert _res["EXPIRED"].mc_provision_amount == 3000.0, "expired provision = qty * unit_cost"
assert _res["FUT_EXP"].is_mc_dead_stock == 0, "has a lot expiry -> NOT dead stock"
assert _res["NEW_STOCK"].is_mc_dead_stock == 0, "new stock (0 months held) -> NOT dead stock"

print("UNIT TESTS PASSED")
```

> [See performance \(1\)](#) [Optimize](#)

UNIT TESTS PASSED

Figure 5.47 Unit Test Passed



The screenshot shows a Jupyter Notebook interface. A variable '_g' is expanded, showing a PySpark DataFrame with columns: material_id (string), snapshot_date (date), warehouse_location (string), quantity_on_hand (double), lot_expiry_date (date), material_name (string), category (string), unit_of_measure (string), reorder_level (integer), lead_time_days (integer), unit_cost (double), total_consumption_12m (double), last_consumed_date (string), first_snapshot_date (date), months_inactive (double), is_below_reorder_level (integer), is_recoverable_asset (integer), is_mc_expired (integer), is_mc_dead_stock (integer), and mc_provision_amount (double). At the bottom, a message says 'RECONCILIATION PASSED - total provision = 51,412.00'.

```
material_id: string
snapshot_date: date
warehouse_location: string
quantity_on_hand: double
lot_expiry_date: date
material_name: string
category: string
unit_of_measure: string
reorder_level: integer
lead_time_days: integer
unit_cost: double
total_consumption_12m: double
last_consumed_date: string
first_snapshot_date: date
months_inactive: double
is_below_reorder_level: integer
is_recoverable_asset: integer
is_mc_expired: integer
is_mc_dead_stock: integer
mc_provision_amount: double
```

RECONCILIATION PASSED - total provision = 51,412.00

Figure 5.48 Reconciliation Test Passed

5.5.3 End-to-End (E2E) Integration Testing

End-to-end integration testing validates that the entire data pipeline operates correctly as a unified system, from the initial ingestion of raw CSV files through to the final Power BI dashboard output. Unlike the data quality and business logic tests conducted in the previous sections, which target individual layers in isolation, E2E testing treats the pipeline as a single executable unit and verifies that data flows consistently and without loss or distortion across all five layers which are bronze, silver, gold, curated and the Power BI semantic model.

5.5.3.1 Test Approach

The E2E test was executed by triggering a full pipeline run from scratch using Azure Data Factory, with no intermediate state carried over from prior runs. The six synthetic source CSV files which are *materials.csv*, *inventory_snapshot.csv*, *production_orders.csv*, *purchase_orders.csv*, *suppliers.csv* and *sales_orders.csv* served as the fixed input dataset. Before execution, a subset of expected output values was derived manually from the source data, including the expected count of Recoverable Asset materials, Magna Carta classifications, total financial provision amount, number of affected customers and number of unfulfilled production orders. These manually computed values served as the business processes to be evaluated which the final Power BI dashboard outputs were compared upon pipeline completion.

Cross-layer record count verification was also performed to confirm that no rows were silently dropped or duplicated between the bronze and silver layers, and that the Gold and Curated layers contained only the subset of records required by the business logic and schema design respectively.

5.5.3.2 Test Case

The E2E test cases and their outcomes are summarised in Table 5.3, Table 5.4, Table 5.5 and Table 5.6.

Table 5.3 Cross-Layer Record Count Verification

Table	Bronze	Silver	Gold/Curated	Result
inventory_snapshot	720	720	20	Pass
materials	20	20	20	Pass
production_orders	100,176	100,176	2000	Pass
sales_orders	14,328	14,328	206	Pass
purchase_orders	402	402	N/A	Pass
suppliers	10	10	N/A	Pass

Table 5.4 E2E Value Reconciliation

No	Bronze	Gold	Curated	Dashboard	Result
TC1	Qty Recoverable Assets	7	7	7	Pass
TC2	Qty Dead Stock	2	2	2	Pass
TC3	Qty Expired Stock	3	3	3	Pass
TC4	Total Provision Amount (RM)	51,412	51,412	51,412	Pass
TC5	Impacted Customers	N/A	15	15	Pass
TC6	Impacted Production Orders	500	500	500	Pass
TC7	Count Below Reorder Level	2	2	2	Pass

Table 5.5 Dashboard Material Classification Verification

Category	Materials	Result
Recoverable Assets	Acetone, Butanol, Cardboard Box, Chrome Yellow, Phthalocyanine Blue, Polyurethane Resin, Vinyl Resin	Pass
Dead Stock	Cardboard Box, Chrome Yellow	Pass
Expired Stock	Acetone, Polyurethane Resin, Vinyl Resin	Pass
Below Reorder Level	Polyurethane Resin, Toluene	Pass

Table 5.6 Business Rule Invariants

No	Bronze	Result
I1	Dead Stock materials are a strict subset of Recoverable Assets	Pass
I2	Dead Stock and Expired Stock classifications are mutually exclusive (overlap = 0)	Pass
I3	Financial provision amount is applied only to Recoverable Asset materials (violations = 0)	Pass

5.5.3.3 Result

All end-to-end test cases passed. Cross-layer record counts confirmed that no records were lost or duplicated during the bronze to silver transformation across all six source tables. The gold and curated layers correctly retained only the records required by their respective business logic and schema definitions. Value reconciliation confirmed that all seven KPI metrics which are recoverable asset count, dead stock count, expired stock count, total provision amount, count below reorder level, impacted customers and impacted production orders produced identical values across the gold layer, curated layer and Power BI dashboard. Material classification verification confirmed that all four inventory categories displayed on the dashboard exactly match the classifications computed in the gold layer. All three business rule invariants held on the full gold output. These results confirm that the full pipeline executes as a coherent and correct system and that the Power BI dashboard accurately represents the underlying inventory risk state as defined by PPG's Recoverable Assets and Magna Carta business rules.

5.5.4 Performance and Scalability Testing

Performance and scalability testing was conducted to evaluate the efficiency of the PPG Inventory Risk Management pipeline under realistic data volumes and to

verify that the system's cloud architecture can handle growth in data scale without structural redesign. Three categories of tests were performed: pipeline transformation performance, query response performance, and scalability comparison between baseline and expanded dataset volumes.

The pipeline transformation tests measured the elapsed execution time of each medallion layer transition from Bronze to Silver via Azure Data Factory, from Silver to Gold via Azure Databricks, and from Gold to the Curated serving layer via Azure Synapse Analytics. These times were obtained by inspecting the ADF pipeline run history and by wrapping the Databricks notebook computation blocks using Python's time module to capture wall-clock duration. Query response performance was measured by executing each of the seven core business queries (Q1–Q7) against the Synapse Serverless SQL Pool and recording the elapsed execution time displayed in the query result pane.

The Table 5.7 below presents how performance testing on pipeline performance is run and tools that have been used.

Table 5.7 Performance Testing I

Performance Testing 1 – Pipeline transformation performance (Azure Databricks)					
ID	Test name	Measure	Tool	Expected Result	Outcome
PT 1.1	Bronze to Silver transformations time	Elapsed time to Azure Data Factory Pipeline to clean all 6 CSV files into Parquet	ADF pipeline run history	<3 minutes	PASS
PT 1.2	Silver to Gold transformation time	Elapsed time for Databricks notebook to	time.time() in notebook	<5 minutes	PASS

		execute RA/MC classification and calculation			
PT 1.3	Gold to Curated times	Elapsed time for Synapse SQL to execute CREATE EXTERNAL TABLE AS SELECT and write star schema to storage	Synapse SQL query log	<2 minutes	PASS

(ss) PT-1: Pipeline Transformation Performance

The Bronze-to-Silver ADF pipeline completed execution in approximately 1 minutes, within the accepted expected time under 3 minutes, confirming that Azure Data Factory's parallel mapping and Parquet conversion process is efficient even at expanded data volumes. The Silver-to-Gold Databricks notebook, which executes the Recoverable Asset classification, Magna Carta flagging, 12-month rolling consumption calculation, and financial provision derivation, completed in approximately 3 minutes. This confirms that PySpark distributed processing on the Databricks cluster can handle the business logic computation within a single pipeline trigger. The Synapse operation for generating the curated star schema completed in approximately 1 minutes, demonstrating that the Serverless SQL Pool approach avoids the overhead of a provisioned warehouse while still achieving acceptable write performance.

The Table 5.8 below presents how performance testing on query response performance is run and tools that have been used.

Table 5.8 Performance Testing II

Performance Testing 2 – Query Response Performance (Azure Synapse)					
ID	Test name	Measure	Tool	Expected Result	Outcome
PT 2.1	Q1 safety stock breach query	Time to return all materials reorder level	Synapse	<10 seconds	PASS
PT 2.2	Q2 recoverable assets audit query	Time to filter and return all RA-flagged stock with quantities and values	Synapse	<10 seconds	PASS
PT 2.3	Q5 financial provision aggregation query	Time to aggregate total_mc_provision_rm across all Magna Carta materials	Synapse	<10 seconds	PASS
PT 2.4	Q7 customer backlog trace query	Time for join-heavy query; linking stockout materials to open sales orders	Synapse	<15 seconds	PASS

(tt) PT-2: Query Response Performance

All seven business queries executed against the Synapse Serverless SQL Pool returned results within acceptable response target time. The Q5 financial provision aggregation query, which is the most computationally intensive due to its decimal-precision casting and multi-bucket grouping, completed in approximately less than 10 seconds. The Q7 downstream customer backlog trace query, which involves a multi-table join between the inventory risk facts and the open sales order ledger, completed in approximately 13 seconds. These results confirm that the Parquet-formatted Curated layer, accessed through Synapse's external table mechanism, provides sub-minute analytical query response suitable for Power BI consumption.

The Table 5.9 below presents how performance testing on scalability test is run and tools that have been used.

Table 5.9 Performance Testing III

Performance Testing 3– Scalability Test (Baseline vs expanded dataset)					
ID	Test name	Measure	Tool	Expected Result	Outcome
PT 3.1	Pipeline execution at baseline scale	Full pipeline run time on original 6 CSV files	Azure Data Factory + Databricks	Complete without error	PASS
PT 3.2	Pipeline execution at expanded scale	Full pipeline run time after dataset expansion via generate_expanded_dataset.py	Azure Data Factory + Databricks	Complete without error	PASS
PT 3.3	Power BI dashboard load time	Time for PowerBI report to fully refresh from Synapse connected semantic model	Power BI	< 1 minute	PASS

(uu) PT-3: Scalability

The baseline pipeline run and executed the original six 6 CSV files in total of 6000 rows and completed within 2 minutes end to end. After re-execution on the expanded dataset of approximately of 50,000 rows, increase approximately 8 times in data volume and the pipeline completed in 4 minutes. This sub-linear scaling behaviour confirms that the Medallion Architecture built on Azure's managed services (ADF, Databricks, Synapse) scales efficiently without requiring architectural changes.

The Power BI semantic model refresh completed in approximately 45 seconds following the expanded-scale pipeline run, confirming that the galaxy schema's pre-aggregated structure enables fast dashboard rendering even at higher data volumes.

Overall, all performance and scalability tests passed within their defined thresholds. The results validate NFR 5 (Maintainability), NFR 6 (Repeatability), and NFR 8 (Scalability) as defined in Section 4.3.2, demonstrating that the system can accommodate additional materials, suppliers, and snapshot periods without structural modification.

5.6 Summary

Chapter 5 has effectively shown that the PPG Inventory Risks assessment pipeline was successfully built and validated across multiple testing phases. The pipeline, taking raw CSV files ingested through bronze layer into the Azure Data Lake Storage Gen 2, a silver transformation layer for cleaning and applying business logic, designing the galaxy schema via Azure Synapse Analytics, and finally serving it into Power BI dashboards that were able to answer all the business questions listed. Thorough checking and validation was tested on the system to ensure its readiness and correctness of multiple weightages including data quality checks, unit and reconciliation testing, end-to-end integration testing, and performance/scalability benchmarks without requiring any major changes to the actual implementation pipeline for additional materials, suppliers or snapshot periods.

REFERENCES

- Anderson, D. L., Britt, F. F., & Favre, D. J. (1997). The seven principles of supply chain management. *Supply Chain Management Review*, 1(1), 31-41.
- Berti, A., Park, G., Rafiei, M., & Van Der Aalst, W. M. P. (2023). A generic approach to extract object-centric event data from databases supporting SAP ERP. *Journal of Intelligent Information Systems*, 61(3), 835–857. <https://doi.org/10.1007/s10844-023-00799-9>
- Chen, H., Chiang, R. H. L., & Storey, V. C. (2012). Business intelligence and analytics: From big data to big impact. *MIS Quarterly*, 36(4), 1165–1188.
- Christopher, M. (2016). *Logistics & Supply chain Management*. Financial Times/Prentice Hall.
- Davis, T. (1993). Effective supply chain management. *Sloan Management Review*, 34(4), 35-46.
- Frazer, G. (2024). *Data cleaning with Power BI: The definitive guide to transforming dirty data into actionable insights*. Packt Publishing.
- Lambert, D. M., & Cooper, M. C. (2000). Issues in supply chain management. *Industrial Marketing Management*, 29(1), 65-83.
- Loshin, D. (2010). The Practitioner’s Guide to Data Quality Improvement. In *Elsevier eBooks*. <https://doi.org/10.1016/c2009-0-17212-4>.
- Modern Data Platform architecture: What it is and how to build one in 2025*. (2026, April 6). <https://www.domo.com/learn/article/how-to-architect-a-modern-data-platform>.
- Reis, J., & Housley, M. (2022). *Fundamentals of Data Engineering: Plan and Build Robust Data Systems*. O’Reilly Media.
- Simović, A. P., Babarogić, S., Pantelić, O., & Krstović, S. (2021). Towards a Domain-Specific Modeling Language for Extracting Event Logs from ERP Systems. *Applied Sciences*, 11(12), 5476. <https://doi.org/10.3390/app11125476>
- Tan, K. C. (2001). A framework of supply chain management literature. *European Journal of Purchasing & Supply Management*, 7(1), 39-48.
- Thomas, D. J., & Griffin, P. M. (1996). Coordinated supply chain management. *European Journal of Operational Research*, 94(1), 1-15.

What is a lakehouse? | DataBricks blog. (2020, January 30). Databricks.
<https://www.databricks.com/blog/2020/01/30/what-is-a-data-lakehouse.html>

What is Medallion Architecture? | Databricks. (2020). Databricks.
<https://www.databricks.com/blog/what-is-medallion-architecture>